

```

public class ORM_Query_SQL {
    public static void main(String[] args) {
        try(Session s = HibernateUtil.getSessionFactory().openSession()) {
            List<Object[]> empleados = s.createNativeQuery("SELECT e.dni, e.id_
                depto, e.nom_emp, dp.categoria FROM empleado e LEFT OUTER JOIN
                empleado_datos_prof dp ON dp.dni=e.dni").list();
            for (Object[] objetos: empleados) {
                System.out.println("Empleado [dni:" + (String) objetos[0]
                    + ", id_depto: " + (Integer) objetos[1]
                    + ", nom_emp: " + (String) objetos[2]
                    + ", categoria: " + (String) objetos[3]
                    + "]");
            }
        } catch (Exception e) {
            e.printStackTrace(System.err);
        }
    }
}

```

Hibernate realiza la consulta de SQL mediante JDBC y consulta los metadatos del `ResultSet` obtenido para averiguar el tipo de cada columna. Para evitar que tenga que hacer esto, o simplemente para explicitar los nombres y los tipos de cada una de las columnas que se espera obtener, lo que redundaría en un código de programa más legible y mantenible, se puede utilizar `addScalar()`. Con ello, la consulta se haría de esta forma:

```

import org.hibernate.type.IntegerType;
import org.hibernate.type.StringType;
(...)
List<Object[]> empleados = s.createNativeQuery("SELECT e.dni, e.id_depto,
    e.nom_emp, dp.categoria FROM empleado e LEFT OUTER JOIN empleado_datos_
    prof dp ON dp.dni=e.dni")
    .addScalar("dni", StringType.INSTANCE)
    .addScalar("id_depto", IntegerType.INSTANCE)
    .addScalar("nom_emp", StringType.INSTANCE)
    .addScalar("categoria", StringType.INSTANCE).list()

```

Resumen

- La persistencia de objetos en bases de datos relacionales plantea un conjunto de problemas que se conocen en conjunto como "desfase objeto relacional".
- ORM (*object-relational mapping*), o "correspondencia objeto-relacional", consiste en el establecimiento de una correspondencia entre clases definidas en un lenguaje de programación orientado a objetos y tablas de una base de datos relacional, y en el uso de mecanismos para que las modificaciones sobre los objetos se registren en la base de datos y, a la inversa, para que se pueda recuperar en objetos la información registrada en la base de datos. La correspondencia objeto-relacional hace posible la persistencia de objetos en bases de datos puramente relacionales.

- Hibernate es un *framework* para ORM en lenguaje Java distribuido bajo licencia LGPL de GNU. Fue la primera herramienta para correspondencia objeto-relacional que logró amplia aceptación y, hoy en día, sigue siendo la más importante. Existe un *porting* de Hibernate para .NET, llamado NHibernate.
- Además de tener su propia API nativa, Hibernate es una implementación certificada de JPA, el estándar de Java para ORM, que es parte de la especificación Java EE.
- Hibernate permite establecer la correspondencia no solo para objetos individuales, sino también para las relaciones entre objetos, ya sean de uno a muchos, de uno a uno o de muchos a muchos. Las relaciones se reflejan en los objetos mediante atributos sencillos y mediante colecciones.
- Hibernate proporciona varios mecanismos alternativos para establecer la ORM para un conjunto de tablas relacionadas por la relación de herencia.
- Con Hibernate, la correspondencia se puede establecer mediante ficheros hbm (*Hibernate mapping files* o “ficheros de correspondencia de Hibernate”) o mediante anotaciones de JPA.
- Una clase para la que se ha definido la correspondencia objeto-relacional es una clase persistente.
- El concepto de sesión es fundamental en Hibernate. Los objetos persistentes lo son en tanto en cuanto están asociados a una sesión. Hibernate sigue la pista de los cambios realizados sobre objetos persistentes asociados a una sesión mediante un gestor de entidades asociado a la sesión y, llegado el momento, se encarga de que esos cambios se reflejen en la base de datos, de manera que se mantenga siempre la consistencia entre el contenido de los objetos persistentes en memoria y en la base de datos. Una sesión y un gestor de entidades asociados constituyen un contexto de persistencia.
- Hibernate tiene su propio lenguaje para manejar objetos persistentes, HQL, similar a SQL, con la diferencia de que maneja objetos y sus propiedades en lugar de filas y columnas de tablas de bases de datos relacionales. JPQL es un subconjunto de HQL y parte del estándar JPA.



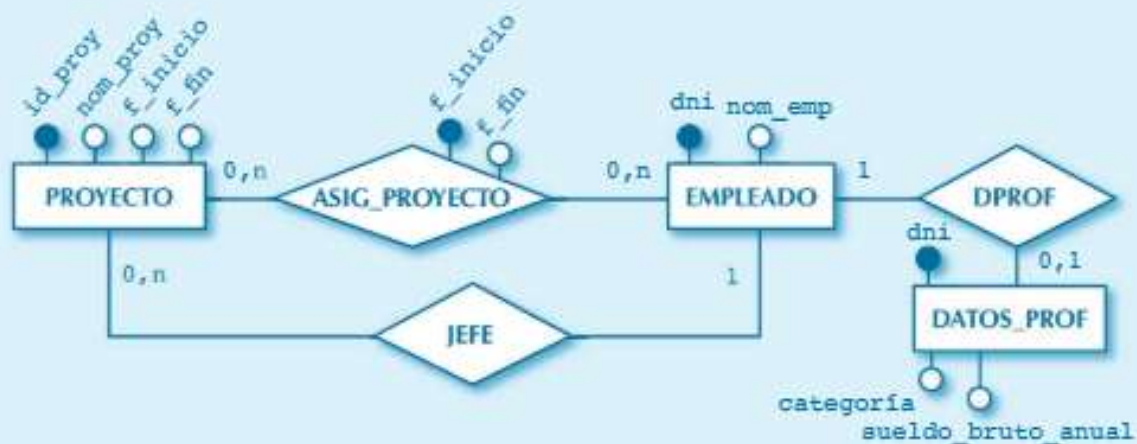
Ejercicios propuestos

Para la realización de estos ejercicios puede ser necesario consultar la documentación de la API de Java SE 8 (<https://docs.oracle.com/javase/8/docs/api/>) o de la de Hibernate (<http://docs.jboss.org/hibernate/orm/5.3/javadocs/>).

Para algunos de los ejercicios debe utilizarse un esquema relacional para representar los proyectos que desarrolla una empresa, sus empleados y las asignaciones de empleados a proyectos. Se da el diagrama entidad-relación para describir las entidades y las relaciones entre ellas, y las sentencias SQL necesarias para crear las tablas en MySQL. No hay que crear todas las tablas y clases desde el principio, sino a medida que vayan siendo necesarias para realizar las actividades. De esa manera, se podrá desarrollar la correspondencia objeto-relacional paso a paso, y centrar la atención en cada momento en cada aspecto que desarrollar.

Para desarrollar los componentes de la aplicación hay que utilizar los asistentes siempre que sea posible, y revisar los ficheros generados, y hacer cambios manualmente si es preciso.

Se puede utilizar otra base de datos relacional distinta de MySQL, preferiblemente Oracle.



Precisiones adicionales:

- La fecha de inicio es obligatoria para proyectos y para asignaciones de empleados a proyectos. Si la fecha de fin es nula, significa que el proyecto no ha concluido. Si no es nula y está en el pasado, significa que el proyecto ha concluido.
- El campo *dni* puede representar también un NIE, es decir, un número de identificación para extranjeros. Por brevedad se le ha llamado sencillamente *dni*.

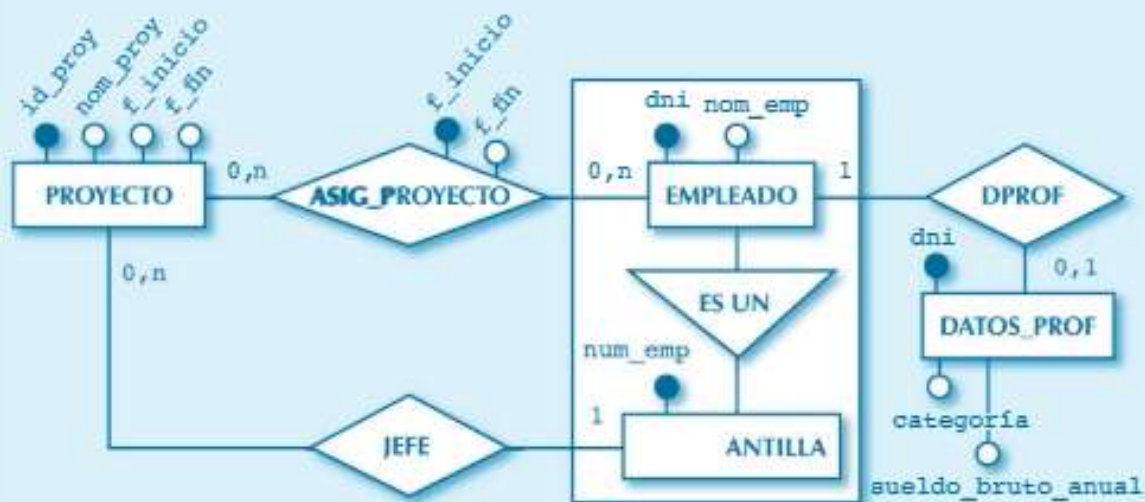
```
create table empleado(
    dni char(9) not null,
    nom_emp varchar(32) not null,
    primary key(dni)
);
create table proyecto(
    id_proy integer auto_increment not null,
    nom_proy varchar(32) not null,
    f_inicio date not null,
    f_fin date,
    dni_jefe_proy char(9) not null,
    primary key(id_proy),
    foreign key fk_proy_jefe(dni_jefe_proy) references
        empleado(dni)
);
create table asig_proyecto(
    dni_emp char(9),
    id_proy integer not null,
    f_inicio date not null,
    f_fin date,
    primary key(dni_emp, id_proy, f_inicio),
```

```

    foreign key f_asig_emp(dni_emp) references empleado(dni),
    foreign key f_asig_proy(id_proy) references proyecto(id_proy)
);
create table datos_prof(
    dni char(9) not null,
    categoria char(2) not null,
    sueldo_bruto_anual decimal(8,2),
    primary key(dni),
    foreign key fk_datosprof_empl(dni) references empleado(dni)
);

```

1. Crea una base de datos en un servidor de bases de datos para las siguientes actividades, llamada `orm_gestion_proyectos`. Crea un usuario sobre la misma base de datos, al que hay que otorgarle los permisos apropiados, como se ha hecho en ejemplos anteriores. Crea una conexión a la base de datos desde el IDE, y verifica que funciona correctamente.
2. Crea las tablas `proyecto` y `empleado`. Crea los POJO y los ficheros de correspondencia utilizando los asistentes que proporciona el IDE. Crea un programa de prueba que cree varios empleados y varios proyectos, y que asigne a cada proyecto su jefe de proyecto. Verifica que no se puede crear un proyecto si no está informado su jefe de proyecto. Verifica que no se puede crear un proyecto si no está informada su fecha de inicio.
3. Crea todo lo necesario para la correspondencia de la relación de muchos a muchos entre proyectos y empleados (incluyendo –y empezando por– la tabla `asig_proyecto`). Crea un programa de prueba que cree varios proyectos y varios empleados y asigne al menos a un empleado a varios proyectos y al menos a un proyecto varios empleados. Verifica que no se puede crear una asignación de un empleado a un proyecto si no se asigna una fecha de inicio.
4. Crea la tabla `datos_prof` y todo lo necesario para la correspondencia de uno a uno entre empleados y datos profesionales. Crea un programa de prueba que genere un nuevo empleado y le asigne sus datos profesionales, y a un empleado ya existente, identificado por su DNI.
5. Crea un programa que, para varias asignaciones cualesquiera de empleados a proyectos, les asigne como fecha de fin una fecha en el pasado (por ejemplo, el día anterior a la fecha actual).
6. Crea un programa que, utilizando una sentencia de HQL, muestre los detalles de todas las asignaciones de empleados a proyectos.
7. Lo mismo que en la actividad anterior, pero solo para asignaciones vigentes. Es decir, no se deben mostrar asignaciones cuya fecha de inicio esté en el futuro ni asignaciones cuya fecha de fin esté informada y esté en el pasado.
8. Considera la distinción de empleados en plantilla frente a empleados no en plantilla, que se contratan para proyectos determinados. Solo los empleados en plantilla pueden ser jefes de proyecto. El siguiente esquema E-R describe este cambio.



Se creará una nueva tabla `emp_plantilla` para los empleados en plantilla, incluyendo un campo `dni` y con una clave foránea por este campo hacia `empleado`. A los empleados en plantilla se les asigna un número de empleado que sirve como identificación para ellos, pero no será una tabla autogenerada. La relación entre empleados y empleados en plantilla es una relación jerárquica, que se implementa en Java con una nueva clase `EmpPlantilla` subclase de la actual `Empleado`. Se pide hacer una copia de la base de datos y una copia del proyecto para ORM desarrollado hasta ahora, que funcionará con la nueva base de datos, y desarrollar la actividad a partir de esta copia. Se pide hacer los cambios necesarios en la estructura de la base de datos, hacer las conversiones de datos necesarias, hacer los cambios necesarios en el proyecto, tanto en las clases como en los ficheros de correspondencia, y crear un programa de prueba en el método `main()` de una nueva clase.

El primer paso será hacer los cambios de estructura y la conversión de datos en la nueva base de datos. Todos los empleados existentes se considerarán empleados en plantilla. Una vez creada la nueva tabla `emp_plantilla`, se tiene que insertar en ella una fila por empleado, con una sentencia `INSERT INTO emp_plantilla(...) SELECT ... FROM empleado`. La clave foránea de `proyecto` a `empleado` debe cambiarse a `emp_plantilla`. Para ello debe borrarse la antigua clave foránea y crear la nueva. Eso es todo en lo que respecta a la conversión de datos. Después habrá que crear la nueva clase `EmpPlantilla` como subordinada de `Empleado`, y la nueva correspondencia para la clase `Empleado` en conjunto con `EmpleadoPlantilla`.

El programa de prueba debe: crear nuevos empleados en plantilla y no en plantilla, crear proyectos nuevos y asignarles como jefe de proyecto empleados en plantilla tanto existentes como empleados creados en el mismo programa, y por último asignar empleados a los nuevos proyectos creados, tanto de plantilla como no de plantilla, y tanto existentes como creados en el mismo programa.

ACTIVIDADES DE AUTOEVALUACIÓN

1. El desfase objeto-relacional es:
 - ☐ a) El conjunto de dificultades conceptuales y técnicas que plantean las bases de datos de objetos.
 - ☐ b) La sobrecarga del sistema que supone la persistencia de objetos en bases de datos relacionales, que se evita con la persistencia en bases de datos de objetos.
 - ☐ c) El conjunto de dificultades que plantea la persistencia de objetos en bases de datos relacionales.
 - ☐ d) El conjunto de correspondencias que es necesario establecer entre clases y tablas de un esquema relacional para hacer posible la persistencia de objetos en bases de datos relacionales.
2. ORM es:
 - ☐ a) El conjunto de extensiones que se pueden añadir a una base de datos relacional para convertirla en una base de datos objeto-relacional, en las que es posible, con ciertas limitaciones, la persistencia de objetos.
 - ☐ b) Un conjunto de extensiones que permiten implementar una base de datos de objetos sobre una base de datos relacional.
 - ☐ c) Un estándar para persistencia de objetos en bases de datos relacionales.
 - ☐ d) La correspondencia objeto-relacional, que hace posible la persistencia de objetos en bases de datos puramente relacionales.
3. Hibernate es:
 - ☐ a) Una implementación de JPA (Java Persistence Architecture).
 - ☐ b) Un *framework* para ORM para el lenguaje Java.
 - ☐ c) Una base de datos objeto-relacional.
 - ☐ d) Nada de lo anterior.
4. Las tablas utilizadas para ORM con Hibernate:
 - ☐ a) No es necesario que tengan clave primaria.
 - ☐ b) Deben tener una clave primaria simple, es decir, consistente en una única columna.
 - ☐ c) Deben tener una clave en la que solo intervengan valores numéricos y auto-generados.
 - ☐ d) Deben tener clave primaria.
5. El fichero de configuración de Hibernate (`hibernate.cfg.xml`):
 - ☐ a) Permite establecer la correspondencia objeto-relacional para un conjunto de clases, independientemente de que para otras clases se pueda establecer mediante ficheros hbm.
 - ☐ b) Guarda todos los datos necesarios para establecer una conexión con el sistema gestor de bases de datos relacional en el que se van a almacenar los objetos persistentes.
 - ☐ c) Normalmente hace referencia a un fichero de ingeniería inversa, cuyo nombre suele ser `hibernate.reveng.xml`.
 - ☐ d) Debe almacenarse en la base de datos.

6. Cuando se hace `s.save(obj)`, siendo `obj` un objeto persistente:
- ☐ a) Hibernate asigna siempre un identificador único automáticamente al objeto.
 - ☐ b) Hibernate asigna un identificador único al objeto solo en el caso en que se haya especificado para el atributo `generator` el valor `assigned`.
 - ☐ c) Hibernate asigna un identificador único al objeto solo si se ha especificado para el atributo `generator` el valor `identity`.
 - ☐ d) Hibernate asigna un identificador único al objeto si se ha especificado para el atributo `generator` el valor `identity`.
7. Las relaciones de uno a muchos entre dos clases:
- ☐ a) Se representan mediante colecciones en las dos clases.
 - ☐ b) Se representan mediante una colección de tipo `HashSet` en una de las clases.
 - ☐ c) Se representan mediante un atributo sencillo en una clase y mediante una colección en la otra clase.
 - ☐ d) No se pueden representar directamente en Hibernate.
8. JPA es:
- ☐ a) Un estándar para el que Hibernate incluye una implementación.
 - ☐ b) Un estándar de ORM para Java.
 - ☐ c) Un estándar que incluye anotaciones en clases de Java para ORM y también el lenguaje JPQL.
 - ☐ d) Todas las opciones anteriores son correctas.
9. Un objeto de una clase persistente puede no tener un identificador único asignado:
- ☐ a) Si está en estado transitorio (*transient*).
 - ☐ b) Si está en estado gestionado (*managed*).
 - ☐ c) Justo después de pasar a estado separado (*detached*).
 - ☐ d) Justo después de pasar a estado eliminado (*removed*).
10. Para establecer la correspondencia objeto-relacional para varias clases entre las cuales hay una relación de herencia:
- ☐ a) Es necesario introducir un campo adicional que funcione como discriminante.
 - ☐ b) Es necesario crear una tabla en la base de datos para todas y cada una de las clases relacionadas mediante la relación de herencia.
 - ☐ c) Si no se crean tablas para las subclases, es necesario crear uno o varios índices únicos en la tabla para la superclase, para evitar duplicidades.
 - ☐ d) Ninguna de las respuestas anteriores es válida.

SOLUCIONES:

1. ☐ a ☐ b ☒ c ☐ d
 2. ☐ a ☐ b ☐ c ☒ d
 3. ☐ a ☒ b ☐ c ☐ d
 4. ☐ a ☐ b ☐ c ☒ d

5. ☐ a ☒ b ☐ c ☐ d
 6. ☐ a ☐ b ☐ c ☒ d
 7. ☐ a ☐ b ☒ c ☐ d
 8. ☐ a ☐ b ☐ c ☒ d

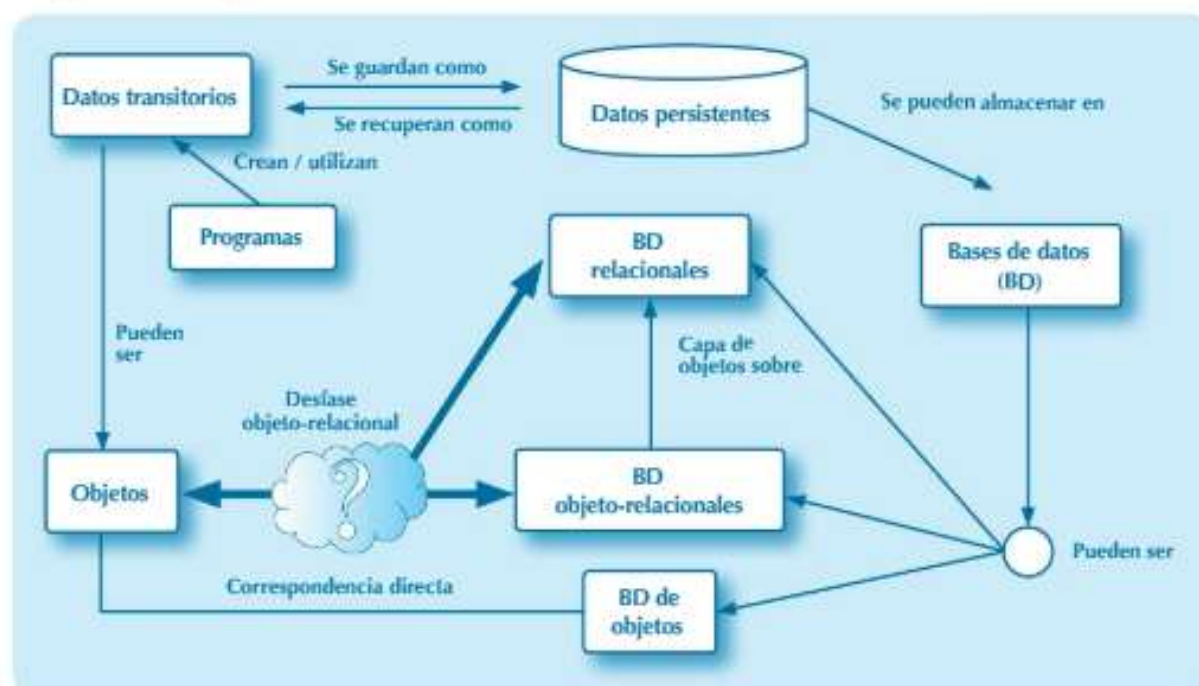
9. ☒ a ☐ b ☐ c ☐ d
 10. ☐ a ☐ b ☐ c ☒ d

Bases de datos de objetos y objeto-relacionales

Objetivos

- ✓ Comprender el propósito y los fundamentos de las BDO.
- ✓ Conocer el estándar ODMG 3.0 para persistencia transparente de objetos desde lenguajes orientados a objetos.
- ✓ Analizar los lenguajes ODL para definición de objetos y OQL para consulta de objetos.
- ✓ Estudiar la BDO Matisse y utilizar su ODL para crear esquemas de objetos.
- ✓ Utilizar el Java *binding* de Matisse para crear, consultar, modificar y borrar objetos persistentes.
- ✓ Entender el propósito y los fundamentos de las BDOR.
- ✓ Asimilar las características objeto-relacionales de la base de datos Oracle.
- ✓ Crear esquemas objeto-relacionales y guardar datos en ellos utilizando el SQL de Oracle.

Mapa conceptual



Glosario

Base de datos de objetos (BDO). Base de datos que puede almacenar directamente objetos complejos.

Base de datos objeto-relacional (BDOR). Base de datos que, aun basándose en el modelo relacional para el almacenamiento de datos, permite trabajar también con objetos.

Language binding. Conjunto de mecanismos implementados en un lenguaje orientado a objetos específico para conseguir persistencia transparente de objetos.

ODL (Object Definition Language). Lenguaje para definición de esquemas de objetos en BDO, parte del estándar ODMG.

ODMG (Object Data Management Group). Grupo formado en 1991 para desarrollar estándares para persistencia de objetos en BDO. También se utiliza para hacer referencia al estándar desarrollado por dicho grupo, cuya última versión es ODMG 3.0.

OQL (Object Query language). Lenguaje para consulta de objetos, parte del estándar ODMG.

Persistencia transparente. Posibilidad de trabajar de la misma forma con objetos persistentes y transitorios, de manera que los cambios realizados en estos últimos durante una transacción se reflejen automáticamente en la base de datos cuando se confirme la transacción.

SQL:99. También llamado SQL3. Revisión del estándar SQL que introduce tipos estructurados definidos por el usuario, entre ellos tipos de objetos.

5.1. Bases de datos de objetos y objeto-relacionales, y correspondencia objeto-relacional

Con el auge de la programación orientada a objetos desde finales de los años ochenta se pusieron de manifiesto las dificultades, tanto conceptuales como técnicas, que planteaba el almacenamiento de objetos complejos en bases de datos relacionales. Para referirse a ellas en conjunto se acuñó el término *desfase objeto-relacional* (en inglés, *object-relational impedance mismatch*) o *desajuste de impedancia objeto-relacional*.

Como solución natural se plantearon bases de datos que permitieran almacenar directamente objetos, a las que se llamó BDO, o en inglés ODB (*object databases*), o también bases de datos orientadas a objetos (BDOO), o en inglés OODB (*object-oriented databases*).

Se intentó repetir la receta del éxito de las bases de datos relacionales unos años antes: el estar basadas en un modelo formal y el temprano desarrollo de estándares, principalmente el lenguaje SQL. Así pues, los primeros esfuerzos se desarrollaron en estos dos ámbitos. En el primero, destaca el manifiesto de Atkinson y otros (1989), con una propuesta para los requisitos que debería cumplir una BDO. Para desarrollar estándares sobre los que basar las BDO, se creó en 1991 el grupo ODMG (Object Database Management Group, en un principio). Este grupo fue creado por R. Cattell (de Sun) y representantes de varios fabricantes de BDO, justo cuando Sun acababa de desarrollar el lenguaje Java. En 1998, ODMG cambió el significado de sus siglas a Object Data Management Group, para reflejar que su objetivo era la persistencia de objetos en general, no necesariamente en BDO.

A pesar del entusiasmo inicial, los resultados a medio y largo plazo no fueron los esperados. No llegó a desarrollarse ningún modelo formal ampliamente aceptado. ODMG desarrolló estándares como ODL para definición de datos, OQL para consulta de datos y, en lugar de un OML o lenguaje específico de manipulación de datos, *language bindings* o vinculaciones con lenguajes orientados a objetos de propósito general ya existentes. La última versión del estándar es ODMG 3.0, publicada en 2000. ODMG se disolvió en 2001, y las compañías que lo integraban centraron sus esfuerzos en JDO (Java data objects). En 2004 se cedieron los derechos para revisar la especificación ODMG 3.0 a OMG (Object Management Group).

Por otra parte, en SQL:99 se incluyeron tipos estructurados definidos por el usuario, entre ellos tipos de objetos, que se han implementado en algunas bases de datos relacionales como PostgreSQL y Oracle, que se pueden considerar por ello BDOR.

Las BDO siguen teniendo un uso muy limitado a fecha de hoy. Existen unas cuantas BDO de *software* libre y privativas, entre ellas Matisse, Objectivity/DB y db4o. Otros planteamientos alternativos para persistencia de objetos han tenido más éxito. Oracle es una BDOR dominante en el segmento de las aplicaciones empresariales, que implementa los objetos en una capa de *software* sobre una base de datos relacional. La correspondencia objeto-relacional (ORM) ha tenido mucho éxito con productos como Hibernate (2001), y el estándar JPA para ORM es parte integrante de Java EE desde Java EE 6 (2009).

5.2. Características de las bases de datos de objetos

Una BDO es una base de datos que puede trabajar directamente con objetos. El manifiesto de Atkinson y otros (1989) propone una lista de características obligatorias que debería cumplir cualquier SGBDO (sistema gestor de BDO), o en inglés ODBMS (*object database management system*). Es muy amplio, pero contiene las ideas fundamentales sobre las que se basan las BDO. En este resumen se omite, por conocido, lo que es similar en las bases de datos relacionales, y conceptos no estrictamente de bases de datos, sino en general de programación orientada a objetos.

1. *Objetos complejos.* Debe ser posible almacenar objetos complejos. Un objeto complejo contiene un conjunto de atributos, cada uno con un nombre y un valor, como una fila de una base de datos relacional. Pero en un objeto complejo los atributos pueden ser no solo de tipos elementales, como números y cadenas de caracteres, sino también:
 - a) Referencias a otros objetos.
 - b) Colecciones desordenadas (conjuntos) y colecciones ordenadas (listas).
2. *Identidad de objetos.* Los objetos tienen una existencia independiente de su valor. Cada objeto tiene un identificador único. Se distingue entre igualdad (dos objetos tienen el mismo valor, o mejor dicho, los mismos valores para sus atributos) e identidad (dos objetos son el mismo objeto, es decir, tienen el mismo identificador único). Identidad implica igualdad, pero no a la inversa.
3. *Tipos o clases.* Un tipo comprende un conjunto de objetos con características comunes, a saber, un conjunto de atributos y un conjunto de operaciones. Una clase es similar, pero es algo más que un conjunto de objetos. Una clase es a su vez un objeto sobre el que se pueden realizar algunas operaciones. Una clase podría mantener su extensión (*extent*), es decir, el conjunto de todos los objetos de la clase, y proporcionar mecanismos para acceder a ellos y realizar operaciones sobre ellos. En adelante se hablará solo de clases, pero todo lo que se diga de ellas se puede hacer extensivo a los tipos.
4. *Extensibilidad.* El sistema de bases de datos tendrá un conjunto de tipos y clases predefinidos. Debe poderse definir nuevos tipos y clases a partir de otros ya existentes, y estos deben poderse utilizar para cualquier cosa para la que se puedan usar los predefinidos.
5. *Complejidad computacional.* Debe poderse realizar cualquier posible cálculo utilizando el DML (lenguaje de manipulación de datos) de la base de datos. Esta es una diferencia fundamental con las bases de datos relacionales. El sublenguaje DML de SQL no es computacionalmente completo. Para realizar muchos o la mayoría de los cálculos con los datos hay que obtener los datos en el contexto de un lenguaje de programación de propósito general. Este requisito se puede satisfacer mediante *bindings* o vinculaciones con lenguajes orientados a objetos de propósito general ya existentes, como por ejemplo Java, en lugar de con un DML separado.
6. *Métodos de consulta sencillos.* Idealmente de alto nivel, eficientes e independientes de la aplicación, es decir, utilizables con cualquier posible base de datos y lenguaje de programación.



Recurso digital

En el anexo web 5.1, disponible en www.sintesis.com y accesible con el código indicado en la primera página del libro, encontrarás el manifiesto de Atkinson, Bancilhon, DeWitt, Dittrich, Maier y Zdonik (1989).

5.3. El estándar ODMG

El estándar ODMG es un estándar para persistencia de objetos en bases de datos, cuya última versión es ODMG 3.0, del año 2000. Los principales componentes de ODMG 3.0 son:

- Modelo de objetos. Basado en el modelo de objetos de OMG.

- ODL o lenguaje de definición de objetos.
- OQL o lenguaje de consulta de objetos. Inspirado en SQL-92.
- *Language bindings* o vinculaciones con los lenguajes C++, Smalltalk y Java.

A diferencia de SQL para bases de datos relacionales, que incluye un DML independiente del lenguaje de programación desde el que se usa, ODMG 3.0 no incluye un DML, sino *language bindings* o vinculaciones con lenguajes de propósito general existentes, a saber: C++, Smalltalk y Java. Este planteamiento se conoce como *persistencia transparente*. Desde el lenguaje de programación de propósito general se crean, modifican y borran los objetos persistentes de la misma forma que los no persistentes, y se utilizan los mecanismos del propio lenguaje. Es en el momento de confirmar los cambios realizados dentro de una transacción cuando se graban en la base de datos los cambios realizados sobre objetos persistentes. Un *language binding* también proporciona mecanismos para realizar consultas de OQL y obtener los resultados como objetos persistentes.

Recurso digital



En el anexo web 5.2, disponible en www.sintesis.com y accesible con el código indicado en la primera página del libro, encontrarás más información acerca de ODMG y del soporte del estándar ODMG 3.0 por parte de diversas BDO (2001).

5.4. ODL

ODL es un lenguaje formal para especificación de objetos. Un SGBDO almacena objetos de tipos definidos en un esquema mediante ODL. Se pueden establecer las analogías con el modelo relacional que se recogen en el cuadro 5.1.

CUADRO 5.1
Analogías entre el modelo ODMG y modelo relacional

Modelo de objetos de ODMG 3.0	Modelo relacional
ODL: lenguaje de definición de objetos	DDL: lenguaje de definición de datos, subconjunto de SQL
Esquema de objetos	Esquema relacional
Clase	Tabla
Instancia de clase	Fila de tabla

5.4.1. Modelo de objetos de ODL

ODL se basa sobre el modelo de objetos de OMG, cuyas principales características son:

1. El estado de un objeto está definido por los valores de sus propiedades. El valor de las propiedades, y por lo tanto el estado de un objeto, puede cambiar a lo largo del tiempo.

2. Las propiedades pueden ser atributos del objeto o relaciones con otros objetos. Las relaciones pueden ser de uno a uno, de uno a muchos o de muchos a muchos.
3. Un objeto es una instancia de una clase y tiene un OID (*object identifier*), que es un identificador único a nivel del sistema, y que no cambia a lo largo del tiempo. Se distingue entre identidad, determinada por el OID (dos objetos son idénticos si y solo si tienen el mismo OID), e igualdad (dos objetos son iguales si y solo si tienen el mismo estado, es decir, los mismos valores para sus atributos).
4. La conducta (*behavior*) de un objeto se define por el conjunto de operaciones que se pueden realizar sobre él. Cada una puede tener una lista de parámetros de entrada y de salida, cada uno de un tipo, y puede devolver un resultado de un tipo determinado.
5. Un tipo (que puede ser una clase o una interfaz) tiene una especificación externa y puede tener una o más implementaciones. La especificación incluye las características visibles externamente del tipo, a saber:
 - a) Operaciones: que se pueden invocar en las instancias del tipo.
 - b) Propiedades: o variables de estado, cuyo valor se puede consultar y cambiar.
 - c) Excepciones: que pueden lanzar sus operaciones.

5.4.2. Clases e interfaces

Un tipo puede ser:

- a) Una interfaz (**interface**). Una interfaz se define por su conducta abstracta, es decir, por sus operaciones.
- b) Una clase (**class**). Una clase se define por su conducta abstracta y por su estado abstracto, es decir, por sus atributos.

La distinción entre interfaces y clases, y su relación con los objetos, es similar a la que hay en Java.

- No se pueden crear instancias de una interfaz, pero sí de una clase. Un objeto es una instancia de una clase.
- Clases e interfaces pueden heredar de interfaces (relaciones de tipo "es un/una").
- Las clases pueden extender otras clases (relaciones de tipo "extiende"). Una clase que extiende a otra, es decir, una subclase, hereda las operaciones y propiedades de la otra, es decir, de la superclase.

Para una clase se puede definir una extensión. Se define con la palabra clave **extent** y se le asigna un nombre. Una extensión proporciona acceso a todas las instancias de una clase. A una extensión (**extent**) se le

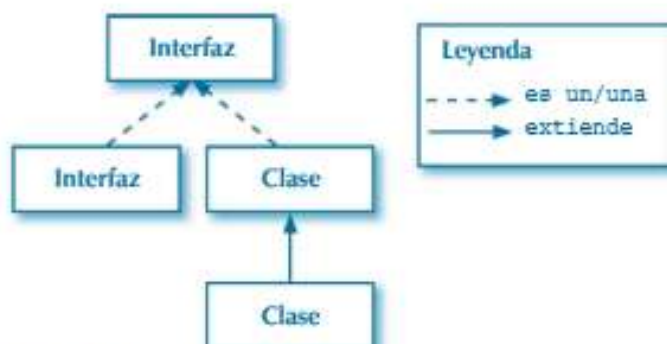


Figura 5.1
Relaciones "es un/una" y "extiende" entre interfaces y clases

puede asociar una clave (*key*), formada por uno o varios atributos. No puede haber dos objetos distintos dentro de la extensión con idéntico valor para la clave. Esta es una restricción de integridad que debe mantener el SGBDO.

5.4.3. Relaciones

Las relaciones son propiedades de las clases. Una relación entre dos clases consiste en la asociación de cada instancia de una clase con una o más instancias de la otra. ODMG 3.0 solo contempla relaciones binarias, es decir, relaciones entre dos clases.

- Pueden ser de uno a uno (1-1), uno a muchos (1-N) y muchos a muchos (N-M).
- Se definen en ambos extremos, es decir, en cada una de las dos clases, como un *traversal path*. La relación entre *Departamento* y *Empleado*, por ejemplo, es de uno a muchos, y está definida por dos *traversal paths* recíprocos el uno del otro: un departamento *emplea_a* uno o varios empleados, y un empleado *trabaja_en* un departamento.

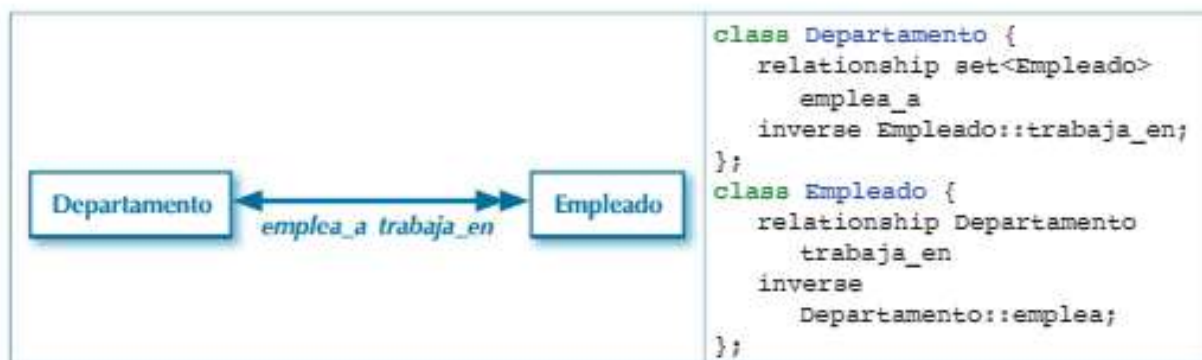


Figura 5.2
Relaciones y *traversal paths* en ODL

- Según la cardinalidad máxima de la relación en un *traversal path*, este se puede implementar mediante un único objeto (cardinalidad máxima uno) o mediante una colección de objetos (cardinalidad máxima muchos). En este último caso, la colección puede ser ordenada (*list*) o desordenada (*set*).
- Las relaciones se gestionan mediante operaciones públicas de las clases involucradas. Si en un *traversal path* la cardinalidad máxima es 1, habrá operaciones para especificar el objeto relacionado, y en caso de que la mínima sea 0, para borrarlo. Si la cardinalidad máxima es mayor que 1, existirán operaciones para añadir y borrar objetos en el conjunto de objetos relacionados. En el ejemplo anterior, existirán operaciones para asignar a un empleado su departamento, así como para añadir un empleado a un departamento.

En ODMG 3.0 se usa una notación propia para diagramas de objetos que representan clases y relaciones entre ellas. Como ejemplo se incluye un diagrama E-R, que incluye los proyectos y los empleados de una empresa, y las relaciones entre ellos, y el correspondiente diagrama de objetos. La traducción de uno a otro es directa.

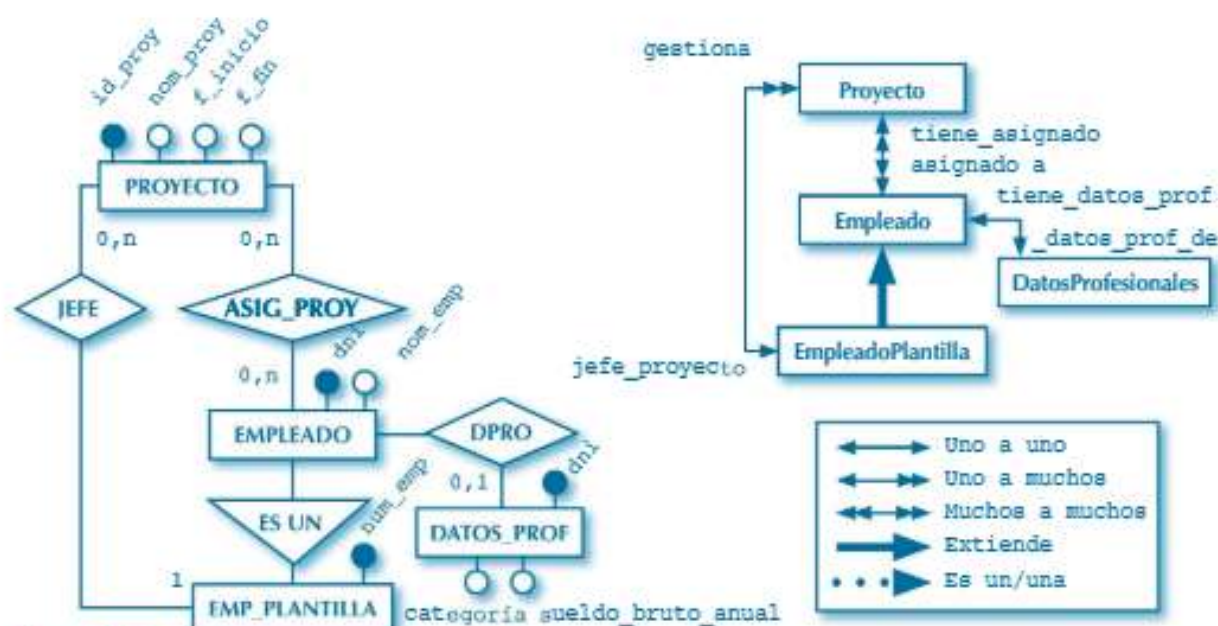


Figura 5.3

Diagrama E-R y de objetos correspondiente para proyectos y empleados de una empresa

Actividad propuesta 5.1



Creo un diagrama E-R, y el correspondiente diagrama de objetos, para representar las siguientes relaciones entre entidades. Una publicación puede ser un libro o una revista. En cualquier caso, tiene un nombre de publicación (*nom_pub*), que en el caso del libro representa el título y en el de la revista, el nombre. Un libro tiene un autor, del que interesa su nombre (*nom_autor*) y su nacionalidad (que se indica en un campo como texto libre). Un libro tiene como identificador un ISBN (*isbn*), y una revista, un ISSN (*issn*). El diagrama E-R debe incluir las entidades PUBLICACIÓN, LIBRO, REVISTA y AUTOR.



Recurso digital

En el anexo web 5.3, disponible en www.sintesis.com y accesible con el código indicado en la primera página del libro, encontrarás una explicación detallada de algunos aspectos del ODL de ODMG 3.0.

5.5. OQL

OQL es un lenguaje de consulta incluido en el estándar ODMG 3.0. Tiene una sintaxis inspirada en la sentencia **SELECT** de SQL, pero adaptada para su uso con objetos. El *language binding* permite realizar consultas en OQL y recuperar los resultados mediante un iterador, como objetos que se pueden utilizar con el lenguaje de programación. OQL no incluye sentencias similares a las sentencias **INSERT**, **UPDATE** o **DELETE** de SQL. En lugar de ello, y según el principio de persistencia transparente de ODMG 3.0, las operaciones de creación, borrado y modificación de objetos persis-

tentes se realizan con el lenguaje de programación, cuyo *language binding* garantiza que los cambios realizados sobre ellos dentro de una transacción se reflejen en la base de datos al confirmarse esta.

OQL tiene el equivalente de casi todas las opciones de la sentencia `SELECT` de SQL y es, por tanto, un lenguaje de consulta muy potente. No se explicará aquí en detalle, sino que se incluirán ejemplos sencillos para ilustrar su funcionamiento general y sus posibilidades.

Las consultas sobre bases de datos se realizan utilizando puntos de entrada. Un punto de entrada da acceso a un objeto o una colección de objetos almacenados. Como punto de entrada puede servir la extensión (*extent*) de una clase que, como ya se ha comentado, permite acceder a todas sus instancias. A continuación, se indica cómo se incluiría en la definición de las clases del esquema de ejemplo la definición de una extensión asociada y de una clave para ella.

```
class Proyecto(extent proyectos key id_proy)
class Empleado(extent empleados key dni)
class EmpleadoPlantilla(extent empleados_plantilla key numEmp) extends
    Empleado
```

La siguiente consulta obtiene los jefes de proyectos con fecha de inicio desde 2017.

```
SELECT p.jefe_proyecto
FROM proyectos p
WHERE p.f_inicio >= date '2017-01-01';
```

Se utiliza como punto de entrada a la base de datos la extensión `proyectos` de la clase `Proyecto`, con alias `p`. El resultado es un objeto de tipo `bag<EmpleadoPlantilla>`. En un `bag` puede haber elementos repetidos. Para evitarlo se puede utilizar `SELECT DISTINCT` en lugar de `SELECT`, y entonces se obtendría un objeto de tipo `set<EmpleadoPlantilla>`.

Se puede acceder a una propiedad de un objeto añadiendo un punto y el nombre de la propiedad. Una propiedad puede ser de un tipo atómico, o un objeto o una relación con otros objetos, en cuyo caso se puede a su vez acceder a sus propiedades de la misma forma. Utilizando este mecanismo repetidamente se puede construir un *path expression* para navegar por la estructura de objetos complejos recuperados a partir del punto de entrada. El siguiente ejemplo recupera el número de empleado de los jefes de proyecto recuperados en el ejemplo anterior, y devolvería un objeto de tipo `set<String>`.

```
SELECT DISTINCT p.jefe_proyecto.num_emp
FROM proyectos p
WHERE p.f_inicio >= date '2017-01-01';
```

Si se quisiera recuperar, además, el nombre de los jefes de proyecto, habría que recuperar ambas cosas en un tipo `struct(String, String)`.

```
SELECT DISTINCT struct(num: p.jefe_proyecto.num_emp, nom: p.jefe_proyecto.
    nom_emp)
FROM proyectos p
WHERE p.f_inicio >= date '2017-01-01';
```

En cada consulta anterior el tipo de objeto devuelto es distinto. En la última hay que utilizar incluso una palabra reservada del OQL (`struct`) para especificar el tipo de objeto devuelto. Todo esto dificulta la composición de consultas, es decir, la creación de nuevas consultas basadas en consultas existentes. En SQL, en cambio, las consultas operan con relaciones y obtienen relaciones. Esto simplifica también el desarrollo de programas de aplicación.

OQL incluye cláusulas análogas a `GROUP BY`, `HAVING` y `ORDER BY` de SQL, y los operadores de agregación `min`, `max`, `count`, `sum`, `avg`.

5.6. Consulta y manipulación de datos con el Java binding

Una implementación del Java *binding*, conforme al estándar ODMG 3.0, debe proporcionar una clase que implemente la interfaz `org.odmg.Implementation`. Esta tiene métodos para crear y recuperar bases de datos (`Database`) y transacciones (`Transaction`). Tiene también un método que permite crear consultas de OQL (`OQLQuery`).

La manipulación de objetos persistentes se realiza según el principio de persistencia transparente. Eso significa que no hay un lenguaje específico para manipulación de objetos persistentes (OML), sino que estos se manipulan desde el lenguaje de programación, y de igual manera que los objetos no persistentes. El *language binding* garantiza que, al realizar una operación *commit* para confirmar los cambios realizados en la transacción actual, se graban en la base de datos todos los cambios realizados sobre los objetos persistentes. También se graban los cambios realizados sobre objetos alcanzables siguiendo las referencias desde cualquier objeto persistente. Esto se conoce como *persistence by reachability* o *transitive persistence* (persistencia por alcanzabilidad o persistencia transitiva).

Las clases persistentes o *persistent-capable* (habilitadas para persistencia) se pueden generar de diversas formas. Una es mediante un preprocesador de ODL que genere su código a partir de definiciones en ODL, lo que no impide que se puedan modificar posteriormente.

Para ilustrar todos estos aspectos se desarrollarán varios ejemplos con el Java *binding* de Matisse, que es en gran medida conforme a ODMG 3.0.

5.7. La base de datos de objetos Matisse

Matisse es una de las BDO más veteranas y con mejor soporte para ODMG 3.0. Es una base de datos de pago pero se puede descargar, previo registro, para su evaluación. Cuenta con una magnífica y muy completa documentación, enlazada desde la ayuda del propio programa.

www

Recursos web

Los siguientes enlaces proporcionan acceso a la descarga de *software* y a toda la documentación para administradores de bases de datos y desarrolladores.

<http://www.matisse.com/developers/downloads>

<http://www.matisse.com/developers/documentation>

La empresa que desarrolla Matisse la define como base de datos posrelacional. Pero, ante todo, y es lo que interesa especialmente aquí, es una potente BDO que soporta en gran medida los diversos aspectos del estándar ODMG 3.0.

- ODL: su implementación es, en gran medida, conforme al planteamiento de ODMG 3.0, aunque usa una sintaxis distinta y presenta algunas discrepancias.
- OQL: Matisse no implementa el OQL de ODMG 3.0. Proporciona como alternativa un *driver* JDBC que permite recuperar colecciones de objetos de la base de datos mediante su lenguaje SQL. Matisse llama SQL a un lenguaje propio que es una adaptación de SQL para BDO, y que incluye no solo una sentencia SELECT como OQL, sino también sentencias **INSERT**, **UPDATE** y **DELETE**.
- *Language bindings*: Matisse proporciona *language bindings* no solo para los incluidos en ODMG 3.0 (C++, Smalltalk y Java, este último con soporte para Java 8 en su versión 9.1.1), sino también para C, C#, Eiffel, Perl, PHP, Python y Visual Basic.

En los siguientes apartados se muestra cómo crear un esquema de objetos en una base de datos de Matisse mediante el ODL de Matisse, y cómo utilizar su Java *binding* para crear programas que se conectan a ella para crear, modificar y borrar tanto objetos como relaciones entre ellos. Para ello será necesario instalar el Enterprise Manager.

5.7.1. Creación de una base de datos mediante ODL con Matisse

Primero, se crea la base de datos y, después, el esquema de objetos en ella mediante ODL. Para crear la base de datos desde Enterprise Manager, se pulsa con el botón derecho del ratón sobre el servidor y se selecciona “New Database”. Como nombre se indica **AcDat_BDO**. Después, se arranca la base de datos pulsando sobre ella y con la opción “Start”. También están disponibles estas opciones en una barra de iconos en la parte de arriba.

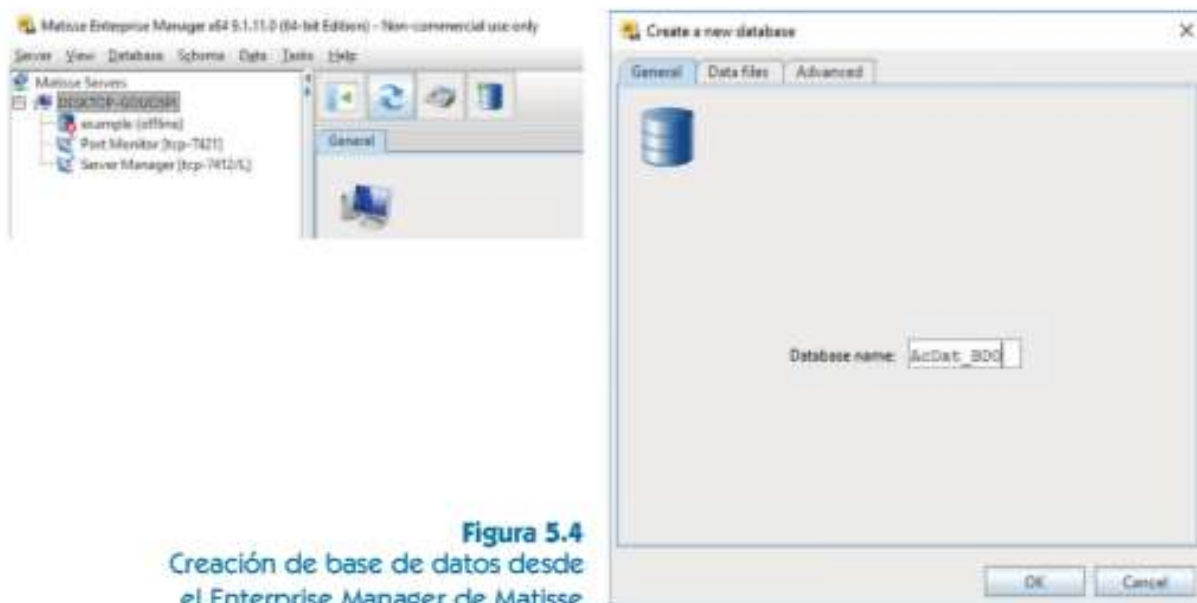


Figura 5.4
Creación de base de datos desde
el Enterprise Manager de Matisse

www

Recurso web

En la siguiente dirección se puede encontrar un documento acerca del ODL de Matisse.
http://www.matisse.com/pdf/developers/matisse_odl.pdf

Las sentencias del ODL de Matisse que permiten crear el esquema de objetos para el modelo de objetos de ejemplo son las siguientes:

```
module gest_proyectos {
    interface Proyecto: persistent
    {
        attribute String<32> nom_proy;
        attribute Date f_inicio;
        attribute Date Nullable f_fin;
        relationship set<Empleado> tiene_asignado[0,-1]
            inverse Empleado::asignado_a;
        relationship EmpleadoPlantilla jefe_proyecto
            inverse EmpleadoPlantilla::gestionada;
    };
    interface Empleado: persistent {
        attribute String<9> dni;
        attribute String<60> nom_emp;
        relationship set<Proyecto> asignado_a[0,-1]
            inverse Proyecto::tiene_asignado;
        relationship DatosProfesionales tiene_datos_prof[0,1]
            inverse DatosProfesionales::datos_prof_de;
        mt_index Empleado_pk unique_key TRUE criteria { dni MT_ASCEND };
        mt_index Empleado_i_nom_emp criteria { nom_emp MT_ASCEND };
    };
    interface DatosProfesionales: persistent {
        attribute String<9> dni;
        attribute String<2> categoria;
        attribute Float sueldo_bruto_anual;
        relationship Empleado datos_prof_de
            inverse Empleado::tiene_datos_prof;
        mt_index Empleado_DatosProf_pk unique_key TRUE criteria { dni MT_ASCEND };
    };
    interface EmpleadoPlantilla: Empleado: persistent {
        attribute String<12> num_emp;
        relationship set<Proyecto> gestionada
            inverse Proyecto::jefe_proyecto;
        mt_index EmpleadoPlantilla_i_dni unique_key TRUE criteria { dni
            MT_ASCEND };
        mt_index EmpleadoPlantilla_i_nom_emp criteria { Empleado::nom_emp
            MT_ASCEND };
    };
};
```

Solo se admiten valores nulos para un atributo si se define con la opción `Nullable`. Por defecto, la cardinalidad mínima de una relación en cualquiera de sus dos extremos o *traversal paths* es 1, y no hay cardinalidad máxima si se define mediante un tipo de colección (`set` o `list`). Se puede cambiar cualquiera de las cardinalidades por defecto indicándolas con `[mín, máx]`. Un valor -1 para `máx` indica que no hay cardinalidad máxima.

Matisse proporciona un identificador único (OID) para cada objeto, único globalmente para todos los objetos existentes en la base de datos y que, por tanto, puede hacer las veces de clave primaria para cualquier clase. Matisse permite definir índices (`mt_index`), sobre un atributo o un conjunto de ellos, que aceleran las búsquedas basadas en sus valores. Los índices pueden ser únicos (de tipo `unique_key`), y entonces sirven como clave primaria, lo que impide que haya más de un objeto con los mismos valores para ellos. En la clase `Empleado` se ha definido un índice único sobre `dni` porque cada persona tiene un DNI distinto, y otro para acelerar búsquedas por nombre. No se ha definido ningún índice único para `Proyecto`. En una base de datos relacional hubiera sido necesario crear en una tabla `proyectos` un atributo para guardar un identificador único, y utilizarlo para la clave primaria.

El esquema de objetos se puede crear importando un fichero con las declaraciones en ODL. El nombre del módulo (en este caso, `gest_proyectos`) es el *namespace* o espacio de nombres, que se creará en la base de datos. El esquema se crea con el botón derecho del ratón sobre la base de datos recién creada, seleccionando "Schema", "Import ODL Schema...". Una vez creado el esquema, se puede utilizar el Data Modeller para obtener diagramas de clases. Este es un programa independiente del Enterprise Manager, y hay que instalarlo aparte.

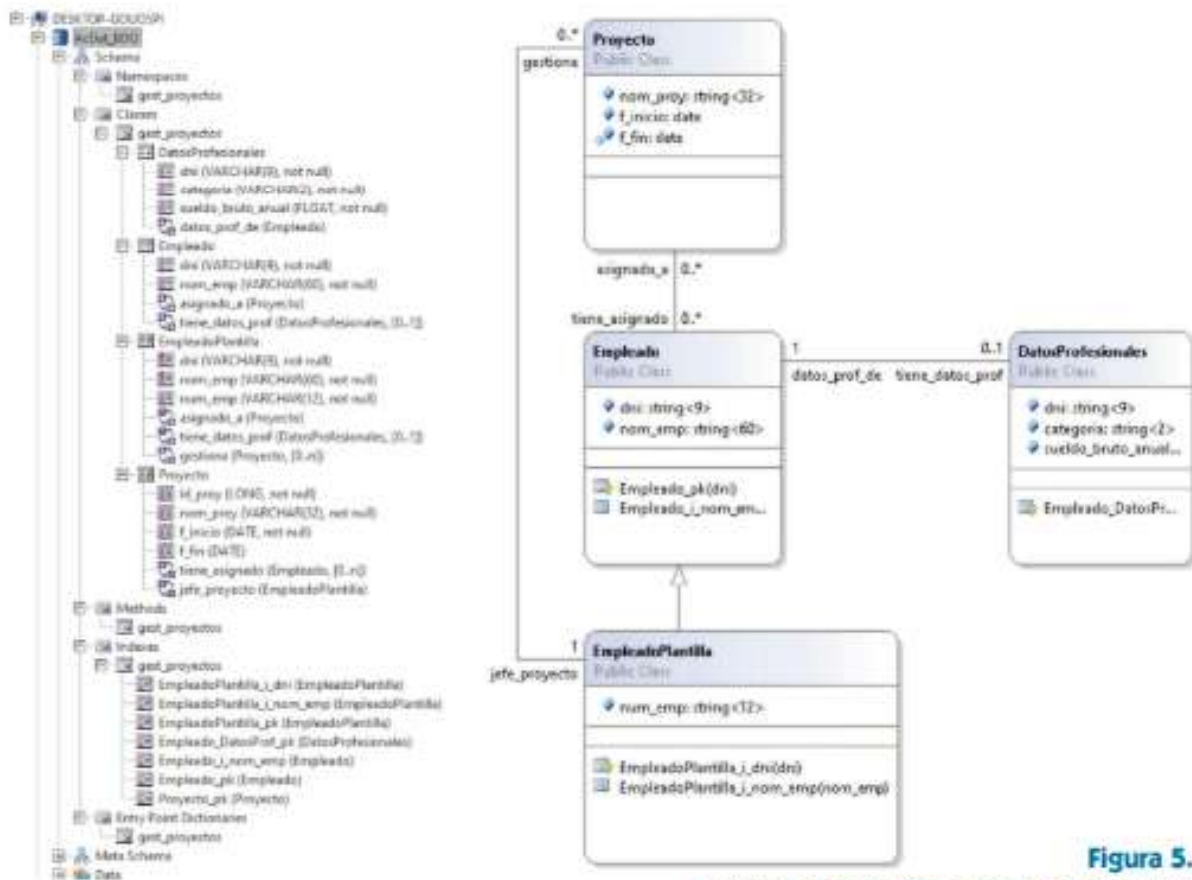


Figura 5.5
Esquema de objetos y diagrama de clases

Actividad propuesta 5.2



Define, utilizando el ODL de Matisse, el esquema de objetos para el modelo de objetos creado en la actividad anterior para publicaciones (libros y revistas), incluyendo todas las clases y los atributos indicados para ellas. Importa el esquema y visualízalo en el Enterprise Manager. Obtén el diagrama de clases con el Data Modeller.

5.7.2. Utilización de la base de datos mediante el Java binding de Matisse

Matisse tiene un Java *binding* muy bien documentado, cuya documentación se puede encontrar en la sección para desarrolladores (*developers*) de su página web, cuyo enlace se ha proporcionado anteriormente.

www

Recursos web

El primero de los siguientes enlaces corresponde a una guía de programación en Java para Matisse con numerosos ejemplos. El segundo proporciona el código fuente completo de estos ejemplos. El tercero es una aplicación para Java EE que se puede desplegar en el servidor de aplicaciones Tomcat, y que permite diversos tipos de consulta sobre los contenidos de una base de datos de Matisse. Se hablará más acerca de Java EE, y del despliegue de aplicaciones en esta plataforma, en un capítulo posterior dedicado a componentes para acceso a datos.

http://www.matisse.com/pdf/developers/java_pg.pdf
http://www.matisse.com/pdf/developers/java_examples.zip
http://www.matisse.com/pdf/developers/jsp_demo.zip



Matisse genera clases de Java para objetos persistentes (*stub classes*) a partir sus definiciones en ODL. Se hace pulsando con el botón derecho sobre la base de datos y seleccionando "Schema", "Generate Code".

Las clases generadas contienen código que no hay que modificar entre comentarios `// BEGIN Matisse SDL Generated Code` y `// END of Matisse SDL Generated Code`. Este código se encarga de la persistencia de objetos. Fuera de ahí, se pueden añadir métodos adicionales a las clases y recompilarlas, y seguirán funcionando

Figura 5.6
Generación de código (*stub classes*)

perfectamente con la base de datos. A continuación, se indican los métodos más importantes disponibles en las *stub classes* generadas por Matisse para Java.

Matisse no proporciona una implementación de la interfaz `org.odmg.Implementation` conforme al estándar ODMG 3.0. En lugar de ello, proporciona una clase `com.matisse.Mt-Database` en `matisse.jar`.

CUADRO 5.2

Métodos más importantes de las *stub classes* generadas por Matisse para Java

	Método(s)	Funcionalidad
Para cada clase. Son de tipo <code>public static</code> . Class es el nombre de la clase. Permiten crear objetos persistentes e iterar sobre los objetos persistentes de la clase.	<code>Class(MtDatabase db)</code>	Crea un objeto de la clase. Class indica el nombre de la clase.
	<code>getInstanceNumber(MtDatabase db)</code> <code>getOwnInstanceNumber (MtDatabase db)</code>	Obtienen el número de instancias de la clase existente en la base de datos. Aquellos métodos en cuyo nombre aparece Own solo tienen en cuenta las instancias de la propia clase, pero no de las subclases.
	<code>instanceIterator(MtDatabase db)</code> <code>ownInstanceIterator(MtDatabase db)</code>	Obtienen iteradores para las instancias de la clase. El primero incluye instancias de subclases, el segundo no.
Para índices. Son de tipo <code>public static</code> . Permiten acceder directamente a objetos conociendo el valor de determinados atributos. Índice es el nombre de un índice.	<code>Class lookupIndice(MtDatabase db, TipoAtrib1 valorAtrib1, TipoAtrib2 valorAtrib2,...)</code> <code>Class[] lookupObjectsIndice (MtDatabase db, TipoAtrib1 valorAtrib1, TipoAtrib2 valorAtrib2,...)</code>	Recuperar objetos dados los valores para atributos incluidos en el índice. Los métodos del primer tipo permiten recuperar un objeto, y son especialmente útiles para índices únicos. Los del segundo tipo permiten recuperar varios objetos, y son de utilidad para índices no únicos.
	<code>IndiceIterator(MtDatabase db, TipoAtrib1 valorAtrib1, TipoAtrib2 valorAtrib2,...)</code>	Devuelve un iterador sobre objetos. Existe otra función que permite especificar opciones adicionales, y que no se describe aquí.
Para cada atributo. Atr es el nombre del atributo.	<code>getAtr()</code> <code>setAtr()</code>	Métodos <i>getter</i> y <i>setter</i> .
	<code>removeAtr()</code>	Elimina el valor del atributo y le asigna su valor por defecto.
	<code>isAtrNull()</code> <code>isAtrDefault()</code>	Comprueban si el valor del atributo es nulo (null) y si es el valor definido por defecto.

[.../...]

CUADRO 5.2 (CONT.)

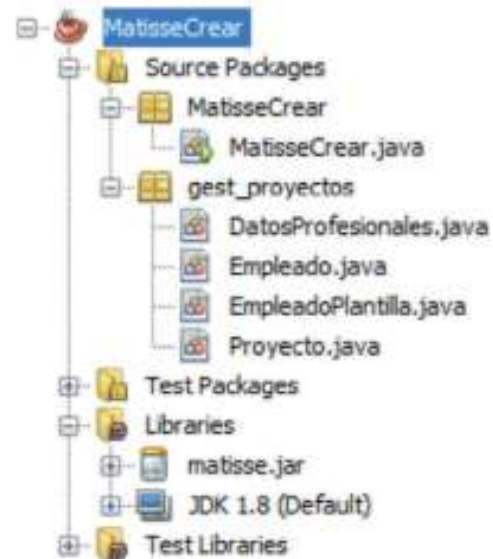
Para todas las relaciones. Rel es el nombre de la relación	<code>clearRel()</code>	Elimina la relación con todos los sucesores según la relación. Un sucesor de un objeto según una relación es un objeto con el que está relacionado de acuerdo a esa relación. No borra los sucesores, solo elimina la relación del objeto en cuestión con ellos.
Para relaciones con cardinalidad máxima 1. TipoSuc es el tipo de los sucesores según la relación	<code>getRel()</code> <code>setRel(TipoSuc succ)</code>	El primer método obtiene el sucesor según la relación. El segundo establece la relación con un objeto dado.
Para relaciones con cardinalidad máxima mayor que 1	<code>TipoSuc[] getRel()</code>	Obtiene en un <i>array</i> sucesores según la relación Rel .
	<code>relIterator()</code>	Obtiene iterador para sucesores según la relación Rel .
	<code>getRelSize()</code>	Obtiene número de sucesores según la relación Rel .
	<code>setRel(TipoSuc[] sucesores)</code>	Asigna los sucesores según la relación Rel .
	<code>prependRel(TipoSuc sucesor)</code>	Añade sucesor según relación Rel , en primera posición, es decir, antes que los ya existentes.
	<code>appendRel(TipoSuc sucesor)</code>	Añade sucesor según relación Rel en última posición, es decir, después de todos los ya existentes.
	<code>appendRel(TipoSuc[] sucesores)</code>	Añade múltiples sucesores según relación Rel después de todos los ya existentes.
	<code>removeRel(TipoSuc sucesor)</code>	Elimina sucesor según la relación Rel .
	<code>removeRel(TipoSuc [] sucesores)</code>	Elimina un sucesor según la relación Rel .

Para construir un programa que utilice el *Java binding* de Matisse hay que incluir en el proyecto `matisse.jar` y las *stub classes*, como se muestra en la figura.

A continuación, se proporcionan ejemplos del uso del *Java binding* de Matisse para abrir una conexión a una base de datos y, dentro de una transacción, y utilizando las funciones anteriores, crear, recuperar, modificar y borrar objetos, y gestionar las relaciones entre ellos.

El siguiente programa abre una conexión con la base de datos y crea un proyecto y varios empleados, tanto de plantilla como no de plantilla, que se asignan al proyecto. Como jefe de proyecto se asigna uno de los empleados de plantilla. Las cardinalidades de las relaciones se verifican al confirmar los cambios en la base de datos. Se produciría una excepción, por ejemplo, si no se asignara un jefe de proyecto a un proyecto. Se resaltan las sentencias para abrir la conexión a la base de datos y crear y confirmar una transacción, y también para gestionar las relaciones entre objetos.

Figura 5.7
Proyecto para Matisse con *stub classes*.



// Creación de empleados y proyectos, y asignación de empleados a proyectos

```

package MatisseCrear;

import com.matisse.MtDatabase;
import com.matisse.MtException;
import gest_proyectos.*;

public class MatisseCrear {

    public static void main(String[] args) {

        try(MtDatabase db = new MtDatabase("localhost", "AcDat_BDO")) {
            db.open();
            db.startTransaction();
            Proyecto p1 = new Proyecto(db);
            p1.setNom_proy("PAPEL ELECTRÓNICO");
            p1.setF_inicio(new java.util.GregorianCalendar(2018,12,01));
            EmpleadoPlantilla jpl = new EmpleadoPlantilla(db);
            jpl.setDni("78901234X");
            jpl.setNom_emp("NADALES");
            jpl.setNum_emp("604202");
            p1.setJefe_proyecto(jpl);
            Empleado el=new Empleado(db);
            el.setDni("56789012B");
            el.setNom_emp("SAMPER");
            p1.appendTiene_asignado(el);
            EmpleadoPlantilla e2=new EmpleadoPlantilla(db);
            e2.setDni("76543210S");
            e2.setNom_emp("SILVA");
            e2.setNum_emp("753014");
            DatosProfesionales dp2 = new DatosProfesionales(db);
            dp2.setDni("76543210S");
            dp2.setCategoria("B1");
            dp2.setSueldo_bruto_anual((float) 45200.00);
            e2.setTiene_datos_prof(dp2);
            p1.appendTiene_asignado(e2);
            Empleado e3=new Empleado(db);

```

```

        e3.setDni("89012345E");
        e3.setNom_emp("ROJAS");
        db.commit();
    }
    catch (MtException mte)
    {
        System.out.println("MtException: " + mte.getMessage());
    }
}
}

```

Actividades propuestas



- 5.3.** Añade un método `float subeSueldoBruto(float incr)` a la clase `Empleado`, al que se le pase el porcentaje de incremento. Si el sueldo bruto está definido, debe devolver el nuevo sueldo, y `null` en otro caso. Escribe y verifica un programa que actualice el sueldo bruto para un empleado utilizando el nuevo método.
- 5.4.** Piensa/investiga/experimenta/verifica: todos los objetos creados por el programa de ejemplo anterior se crean como objetos persistentes. ¿Cómo se crearían objetos transitorios (es decir, no persistentes) de las mismas clases? ¿Es posible que un objeto creado como transitorio se acabe almacenando en la base de datos? ¿En qué casos? Justifica tus respuestas y verifícalas, en su caso, mediante un programa.
- 5.5.** Genera las *stub classes* para el esquema de objetos creado en una actividad anterior para publicaciones. Crea un programa en Java que cree al menos tres libros, dos de ellos del mismo autor, y dos revistas en la base de datos y que, para terminar, muestre toda la información acerca de todos los objetos creados.

El Object Browser del Enterprise Manager de Matisse permite visualizar los objetos creados y las relaciones entre ellos. Se pueden seleccionar objetos mediante consultas en SQL, visualizar sus propiedades y acceder a los objetos relacionados. Como se puede ver, desde un objeto se puede acceder a los objetos relacionados siguiendo los *traversal paths*.

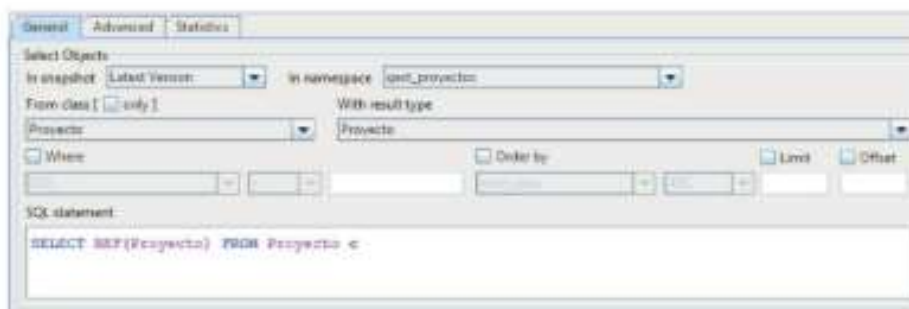


Figura 5.8
Consulta
de objetos
en el Object
Browser
de Matisse

Figura 5.9
Navegación
por los objetos
con el Object Browser
de Matisse

Object	Value	Type
seleccion	(Length=1)	Proyecto[]
[0]	(0x10a8 Proyecto)	Proyecto
nom_proy	PAPEL ELECTRÓNICO	MT_STRING
f_inicio	2018-01-01	MT_DATE
f_fin	MT_NULL	MT_DATE
tiene_asignado	(Length=2)	Empleado[]
[0]	(0x10a8 Empleado)	Empleado
dni	547890123	MT_STRING
nom_emp	SAMPER	MT_STRING
asignado_a	(Length=1)	Proyecto[]
tiene_datos_prof	(Length=0)	DatosProfesionales
[1]	(0x10a8 EmpleadoPlantilla)	EmpleadoPlantilla
dni	765432109	MT_STRING
nom_emp	SILVA	MT_STRING
num_emp	753014	MT_STRING
asignado_a	(Length=1)	Proyecto[]
tiene_datos_prof	(Length=1)	DatosProfesionales
gestiona	(Length=0)	Proyecto[]
jefe_proyecto	(Length=1)	EmpleadoPlantilla
[0]	(0x10a8 EmpleadoPlantilla)	EmpleadoPlantilla
dni	78901234X	MT_STRING
nom_emp	MORALES	MT_STRING
num_emp	604302	MT_STRING
asignado_a	(Length=0)	Proyecto[]
tiene_datos_prof	(Length=0)	DatosProfesionales
gestiona	(Length=1)	Proyecto[]

El siguiente programa borra y modifica algunos objetos y elimina algunas relaciones entre objetos, y utiliza iteradores de varios tipos. Recupera empleados de plantilla a partir de su DNI con `lookupEmpleadoPlantilla_i_dni`, que se ha creado en `EmpleadoPlantilla` porque se ha definido el índice `EmpleadoPlantilla_i_dni`. Recupera empleados a partir de su DNI con `lookup_empleado_pk()`, que se ha creado porque DNI se ha definido como clave primaria de `Empleado` (`unique_key`). Antes de borrar un empleado, es necesario borrar, si existen, sus datos profesionales. Si no, se produce una excepción, dado que la cardinalidad mínima en el *traversal path* `datos_prof_de` es 1. También obtiene y utiliza iteradores. Primero, obtiene todos los proyectos con `instanceIterator()` de la clase `Proyecto`. Para cada proyecto obtiene los empleados asignados a él con un iterador obtenido con el método `tiene_asignadoIterator()` de la clase `Proyecto`, que se ha creado para el *traversal path* `tiene_asignado` de la clase `Proyecto`.

```
// Borrado objetos y relaciones entre ellos
import com.matisse.MtDatabase;
import com.matisse.MtException;
import com.matisse.MtObjectIterator;
import gest_proyectos.*;

public class MatisseModifBorrar {
    public static void muestraProyecto(Proyecto p) {
        System.out.println("Proyecto "+p.getNom_proy()+
            "[OID: "+p.getMtOidToHexString()+""]");
    }
}
```

```

        System.out.println("-----");
        System.out.println("Jefe proyecto: DNI: "+p.getJefe_proyecto().
            getDni()+
            ", Nombre: "+p.getJefe_proyecto().getNom_emp());
        System.out.println("Empleados:");
        MtObjectIterator<Empleado> itEmp = p.tiene_asignadoIterator();
        while(itEmp.hasNext()) {
            Empleado e = itEmp.next();
            System.out.println("DNI: "+e.getDni()+" , Nombre: "+e.getNom_emp());
        }
    }

    public static void main(String[] args) {
        try(MtDatabase db = new MtDatabase("localhost", "AcDat_BDO")) {
            db.open();
            db.startTransaction();
            Proyecto p = new Proyecto(db);
            p.setNom_proy("TINTA HOLOGRÁFICA");
            p.setP_inicio(new java.util.GregorianCalendar(2018,12,28));
            EmpleadoPlantilla ep = // NADALES
            EmpleadoPlantilla.lookupEmpleadoPlantilla_i_dni(db,
                "78901234X");
            p.setJefe_proyecto(ep);
            Empleado e1 = Empleado.lookupEmpleado_pk(db, "89012345E"); //
                ROSAS
            e1.setNom_emp("ROSAS");
            e1.appendAsignado_a(p);
            Empleado e2 = Empleado.lookupEmpleado_pk(db, "76543210S"); //
                SILVA
            e2.getTiene_datos_prof().remove();
            e2.remove();
            Empleado e3 = Empleado.lookupEmpleado_pk(db, "56789012B"); //
                SAMPER
            e3.clearAsignado_a();
            MtObjectIterator<Proyecto> itProy = Proyecto.
                instanceIterator(db);
            while(itProy.hasNext()) {
                Proyecto unProy = itProy.next();
                muestraProyecto(unProy);
            }
            db.commit();
        }
        catch (MtException mte)
        {
            System.out.println("MtException: " + mte.getMessage());
        }
    }
}

```

Actividades propuestas



- 5.6.** Crea un programa que cree un nuevo libro y le asigne como autor uno de los autores ya existentes. El programa debe también crear un nuevo autor, que no tendrá ningún libro

asociado. El programa debe borrar uno de los dos libros del autor para el que había dos libros. Por último, como comprobación, debe mostrar toda la información acerca de todos los objetos existentes. Se sugiere mostrar autor a autor, y para cada autor sus libros.

- 5.7.** Existe un método `deepRemove()` en cada clase que por defecto llama a `remove()`. Este se puede cambiar, cuando tenga sentido, para borrar objetos subordinados antes de llamar a `remove()` para borrar el propio objeto. Cambia `deepRemove()` en la clase `Empleado` para borrar antes los datos profesionales. Modifica el programa de ejemplo anterior para que utilice este método para borrar el empleado.

5.7.3. Consultas mediante el SQL de Matisse

Matisse no proporciona soporte para OQL conforme al estándar ODMG 3.0. En lugar de ello, tiene un lenguaje para consulta, definición y manejo de datos al que se llama SQL, similar al SQL estándar de las bases de datos relacionales, pero adaptado para BDO, y un *driver* JDBC. El lenguaje SQL de Matisse tiene sentencias `SELECT`, `INSERT`, `UPDATE` y `DELETE`. No hay que dejarse confundir por el nombre SQL ni por su similitud al SQL estándar para bases de datos relacionales, ni porque se denomine a Matisse base de datos posrelacional. Matisse es una BDO, y lo que se llama SQL de Matisse no es realmente SQL. En la parte para desarrolladores de la web de Matisse, para la que se ha proporcionado antes un enlace, se puede encontrar documentación acerca del SQL de Matisse, y acerca del Java *binding* de Matisse y programación en Java para Matisse en general, incluyendo el uso de su *driver* de JDBC.

Recursos web

www

En los siguientes enlaces se puede encontrar documentación detallada acerca del lenguaje SQL de Matisse y una guía de programación en Java para Matisse que dedica un capítulo específicamente a JDBC.

http://www.matisse.com/pdf/developers/sql_pg.pdf
http://www.matisse.com/pdf/developers/java_pg.pdf

La clase para el *driver* JDBC es `com.matisse.sql.MtDriver`. Se puede cargar el *driver* a la manera antigua con `Class.forName("com.matisse.sql.MtDriver")` o, lo más sencillo, obtener una conexión JDBC con `getJDBCConnection()` de `MtDatabase`. El siguiente programa crea un nuevo empleado y lo asigna a todos los proyectos de los que es jefe un determinado empleado, identificado por su DNI. Para obtener los proyectos como objetos hay que utilizar en la consulta `REF(p)`, que devuelve una referencia a cada objeto de clase `Proyecto`, que se obtiene con `getObject()`. El planteamiento con una base de datos estrictamente conforme a ODMG 3.0 sería análogo: recuperar los proyectos mediante una consulta en OQL, hacer los cambios pertinentes sobre ellos y confirmar los cambios.

```
// Recuperación de datos mediante JDBC y modificaciones sobre datos obtenidos
package MatisseConsultaConJDBCModifConJavaBinding;

import com.matisse.MtDatabase;
import com.matisse.MtException;
import java.sql.Connection;
import java.sql.Statement;
import java.sql.ResultSet;
import java.sql.SQLException;
import gest_proyectos.*;

public class MatisseConsultaConJDBCModifConJavaBinding {

    public static void muestraErrorSQL(SQLException e) {
        System.err.println("SQL ERROR mensaje: " + e.getMessage());
        System.err.println("SQL Estado: " + e.getSQLState());
        System.err.println("SQL código específico: " + e.getErrorCode());
    }

    public static void main(String[] args) {
        try {
            MtDatabase db = new MtDatabase("localhost", "AcDat_BDO") {
                db.open();
                db.startTransaction();
                Empleado e = new Empleado(db);
                e.setDni("65432109F");
                e.setNom_emp("LUQUE");
                try {
                    Connection jdbcCon = db.getJDBCConnection();
                    Statement stmt = jdbcCon.createStatement(); {
                        String commandText = "SELECT REF(p) FROM gest_proyectos.
                        Proyecto p WHERE p.jefe_proyecto.dni='78901234X'";
                        ResultSet rset = stmt.executeQuery(commandText);
                        Proyecto p;
                        while (rset.next()) {
                            p = (Proyecto) rset.getObject(1);
                            System.out.println("Proyecto: "+p.getNom_proy()+
                            ", jefe: ["+p.getJefe_proyecto().getDni()+"] "+
                            p.getJefe_proyecto().getNom_emp());
                            if(p.getJefe_proyecto().getDni().equals("78901234X")) {
                                p.appendTiene_asignado(e);
                                System.out.println("Asignado nuevo empleado.");
                            }
                        }
                    }
                    rset.close();
                    stmt.close();
                    db.commit();
                } catch (SQLException sqle) {
                    muestraErrorSQL(sqle);
                }
            } catch (MtException mte) {
                System.out.println("MtException: " + mte.getMessage());
            }
        }
    }
}
```

5.8. SQL:99 y bases de datos objeto-relacionales

En SQL:99, también conocido como SQL3, la cuarta revisión del estándar SQL, se introdujeron los tipos estructurados definidos por el usuario. Esto significa que los atributos de una tabla pueden tener tipos no atómicos o estructurados, además de tipos atómicos como `CHAR`, `VARCHAR2`, `INTEGER`, etc. En una tabla puede haber columnas que contengan objetos, *arrays* o incluso tablas (tablas anidadas), y también referencias a objetos, de manera que en varios lugares diferentes de la base de datos puedan existir referencias al mismo objeto, almacenado en un único lugar en la base de datos. Estas características, por supuesto, se salen completamente del modelo relacional. Pero si no se utilizan en una base de datos, esta sigue siendo relacional.

Las bases de datos que implementan, en mayor o menor medida, estas características se pueden calificar como objeto-relacionales. Entre ellas cabe destacar Oracle.

Oracle implementa todas estas características en una capa de *software* sobre una base de datos relacional.

Recursos web

www

Las últimas versiones de la base de datos Oracle son 11g, 12c y 18c. Existen versiones XE de 11g y 18c. Son versiones simplificadas y limitadas que pueden utilizarse de manera gratuita y sin soporte técnico de Oracle. Para un primer contacto puede ser buena opción la versión 11g XE. El siguiente enlace dirige a la página de descargas para las distintas versiones de Oracle:

<https://www.oracle.com/technetwork/database/enterprise-edition/downloads/index.html>

El siguiente enlace proporciona la documentación más importante de la versión 12c:

<https://docs.oracle.com/en/database/oracle/oracle-database/12.2/books.html>

Es de especial interés la documentación de SQL, una vez que se haya completado el proceso de instalación y configuración inicial:

<http://docs.oracle.com/en/database/oracle/oracle-database/12.2/sqlqr/sql-language-quick-reference.pdf>

<http://docs.oracle.com/en/database/oracle/oracle-database/12.2/sqlrf/sql-language-reference.pdf>

5.9. Características objeto-relacionales de Oracle

Oracle permite definir tipos de objetos (clases), tablas de objetos (que almacenan un objeto de una clase determinada en cada fila), tablas con columnas de objetos (que contienen objetos de una clase determinada en una columna), y tablas con columnas que contienen colecciones, tanto *arrays* como tablas (tablas anidadas). Pero en última instancia se trata de una base de datos relacional y todos estos datos se almacenan en una base de datos puramente relacional.



Recurso digital

En el anexo web 5.4, disponible en www.sintesis.com y accesible con el código indicado en la primera página del libro, encontrarás la “Guía del desarrollador objeto-relacional”.

A continuación, se explicará la forma de definir, utilizando el SQL de Oracle, tipos estructurados definidos por el usuario, tales como los introducidos en SQL:99, entre ellos tipos de objetos, y se utilizarán para implementar el modelo de objetos de ejemplo que se ha venido utilizando hasta ahora y para introducir algunos datos de ejemplo.

5.9.1. Tipos de objetos

Se crean con `CREATE TYPE ... AS OBJECT`. Un tipo de objeto es un tipo estructurado que tiene atributos y métodos, es decir, una clase. Hay que terminar las declaraciones de tipo con el carácter `/`. Se pueden crear tipos de objeto, que se utilizarán para la definición de las tablas, como sigue:

```
CREATE TYPE DatosProfesionales AS OBJECT (
    categoria CHAR(2),
    sueldo_bruto_anual NUMBER(8, 2)
);
/
CREATE TYPE Empleado AS OBJECT (
    dni CHAR(9),
    nom_emp VARCHAR2(60),
    datos_prof DatosProfesionales
) NOT FINAL;
/
```

El propósito de la opción `NOT FINAL` en la definición de `Empleado` se verá en breve.

En estas definiciones de tipos no tienen sentido restricciones sobre el valor que puedan tomar los atributos, como por ejemplo que no puedan tomar valor nulo. Estas se introducen como restricciones de integridad en tablas basadas en estos tipos, como se verá en breve.

5.9.2. Herencia

Se pueden definir clases como subclases de otras. No se admite herencia múltiple, solo simple. Solo se pueden definir subclases de una clase dada si se define como `NOT FINAL`. Se puede definir la clase `EmpleadoPlantilla` como subclase de `Empleado` como sigue:

```
CREATE TYPE EmpleadoPlantilla UNDER Empleado(
    num_emp CHAR(12)
);
/
```

Si no se hubiera definido la clase Empleado como `NOT FINAL`, se podría cambiar con:

```
ALTER TYPE Empleado NOT FINAL;
```

5.9.3. Objetos de fila y objetos de columna

Se puede definir una tabla con la misma estructura de una clase (es decir, de un *object type* o tipo de objeto). Cada fila de la tabla contendrá un objeto de la clase, lo que se conoce como *row object* u objeto de fila. La base de datos genera un OID o identificador único para cada objeto de fila.

Un atributo de una tabla puede ser de un tipo de objeto. Esto es lo que se conoce como *object column* u objeto de columna. La base de datos no genera OID para objetos de columna.

Por ejemplo, se puede definir una tabla `empleados` cuyas filas contengan objetos del tipo de objeto `Empleado`, cuyo atributo `datos_prof` es del tipo `DatosProfesionales`, por lo que cada uno de estos objetos de fila contendrá un objeto de columna en `datos_prof`.

En la definición de la tabla se puede incluir la definición de clave primaria y restricciones sobre los valores que puedan tomar los atributos.

```
CREATE TABLE empleados OF Empleado(
    dni PRIMARY KEY,
    nom_emp NOT NULL
);
```

Se pueden insertar filas en la tabla de empleados de la siguiente forma:

```
INSERT INTO empleados VALUES (Empleado('56789012B', 'SAMPER', null));
INSERT INTO empleados VALUES (Empleado('76543210S', 'SILVA',
    DatosProfesionales('B1', 45200.00)));
INSERT INTO empleados VALUES (EmpleadoPlantilla('78901234X', 'NADALES', null,
    '604202'));
```

El ejemplo anterior muestra cómo en la tabla `empleados` se pueden introducir objetos de la clase `Empleado` y de su subclase `EmpleadoPlantilla`.

El operador `REF` permite obtener referencias a objetos de fila, es decir, sus OID.

```
SELECT REF(e) FROM empleados e;
```

Para objetos de columna no se crean referencias. La siguiente consulta es errónea:

```
SELECT REF(e.datos_prof) FROM empleados e;
```

5.9.4. Tipos de objetos con referencias a otros tipos de objetos

Las referencias permiten utilizar un mismo objeto en la definición de varios objetos. Por ejemplo: un mismo empleado de plantilla puede ser el jefe de proyecto de varios proyectos. Para reflejar esto en la base de datos, un objeto de tipo `EmpleadoPlantilla` se almacena una sola vez en la base de datos, en la tabla `empleados`, y en la tabla `proyectos` se almacenan varios objetos de tipo `Proyecto` que contienen una referencia a él. Guardar copias del mismo objeto de tipo `EmpleadoPlantilla` en varios objetos de tipo `Proyecto` no solo es ineficiente, sino erróneo. Una copia de un objeto es igual, pero no idéntica, no es el mismo objeto. Una referencia contiene el OID del objeto referenciado. El tipo `Proyecto` y la tabla `proyectos` se podrían definir de esta manera:

```
CREATE TYPE Proyecto AS OBJECT (
    nom_proy VARCHAR2(32),
    f_inicio DATE,
    f_fin DATE,
    jefe_proy REF EmpleadoPlantilla
);
/
CREATE TABLE proyectos OF Proyecto(
    nom_proy NOT NULL,
    f_inicio NOT NULL,
    jefe_proy SCOPE IS empleados
);
```

El jefe de proyecto de un proyecto se almacena como una referencia (`REF`) a un objeto de tipo `EmpleadoPlantilla` almacenado en la tabla `empleados` (`SCOPE IS empleados`).

Ahora se puede introducir un proyecto, incluyendo la referencia a su jefe de proyecto.

```
INSERT INTO proyectos
SELECT Proyecto('PAPEL ELECTRÓNICO', to_date('01/12/2018', 'dd/mm/YYYY'),
    NULL, TREAT(REF(e) AS REF EmpleadoPlantilla)) FROM empleados e WHERE
DNI='78901234X';
```

El operador `TREAT` hace posible tratar la referencia recuperada como una referencia a un objeto de tipo `EmpleadoPlantilla` en lugar de como un objeto de su superclase `Empleado`.

Se puede obtener un objeto a partir de una referencia a él mediante el operador `DEREF`.

```
SELECT DEREF (p.jefe_proy)
FROM proyectos p WHERE p.nom_proy='PAPEL ELECTRÓNICO';
```

5.9.5. Tipos de datos de colección: `VARRAY` y tablas anidadas

Aparte de tipos de objeto, es posible definir distintos tipos de colecciones, y se pueden definir atributos con esos tipos. En particular, se pueden definir tipos de *arrays* y de tablas.

Los tipos de *array* (`VARRAY`) son similares a los *arrays* de Java. Los elementos guardados en un `VARRAY` se pueden recuperar indicando un índice, siendo 1 el índice del primer elemento. Por

ejemplo, para cada empleado se podría guardar hasta un máximo de tres teléfonos, modificando la definición del tipo `Empleado` como sigue:

```
CREATE TYPE TelefonosEmpleado AS VARRAY(2) OF CHAR(14);
/
ALTER TYPE Empleado ADD ATTRIBUTE telefonos TelefonosEmpleado CASCADE;
```

La palabra reservada `CASCADE` se utiliza para que los cambios realizados en el tipo se propaguen a todos los tipos y tablas en los que este se utiliza. Esto implica, aparte del cambio de los tipos, la actualización de los contenidos de las tablas para adaptar los datos existentes a la nueva definición de los tipos.

Se podría añadir un nuevo empleado en la tabla de objetos `empleados` definida anteriormente de la siguiente forma:

```
INSERT INTO empleados VALUES( Empleado('89012345E', 'ROJAS', null,
    TelefonosEmpleado('654321098', '959456789'))
);
```

Se pueden definir tipos de tablas. Si se define una columna de una tabla con un tipo de tabla, cada fila de la tabla contendrá en esa columna una tabla (tabla anidada o *nested table*). Por ejemplo, se podría añadir a la tabla `proyectos` una columna para la lista de empleados asignados al proyecto.

```
CREATE TYPE TablaAsigEmpleados AS TABLE OF REF Empleado;
/
ALTER TYPE Proyecto ADD ATTRIBUTE asig_empleados TablaAsigEmpleados CASCADE;
```

Se puede añadir un nuevo proyecto con la lista de empleados asignados de la siguiente manera:

```
UPDATE proyectos
SET asig_empleados=TablaAsigEmpleados
(
(SELECT REF(e) FROM empleados e WHERE DNI='56789012B'),
(SELECT REF(e) FROM empleados e WHERE DNI='76543210S')
)
WHERE nom_proy='PAPEL ELECTRÓNICO';
```



Actividad propuesta 5.8

Se quiere tener para cada asignación de un empleado a un proyecto la fecha de inicio y de fin. Elimina la columna `asig_empleados` de la tabla `proyectos` y añade una nueva columna con igual nombre que contenga también una tabla anidada con los empleados asignados al proyecto, pero en cada fila de esta tabla, además de la referencia al empleado, debe haber columnas con las fechas de inicio y de fin de asignación al proyecto (`f_ini` y `f_fin`). La referencia al empleado no puede ser nula, ni la fecha de inicio. Su clave primaria debe estar compuesta por la referencia

al empleado y la fecha de inicio. Se trata de definir un nuevo tipo `AsigEmpleado` que incluya la referencia al empleado y las dos fechas, y utilizarlo para definir el tipo `TablaAsigEmpleados`.

Las características objeto-relacionales de Oracle ofrecen muchas posibilidades. Con esta breve introducción se ha tratado solo de introducir las características fundamentales de las BDOR basadas en SQL:99, y de dar una idea de cómo se puede crear con Oracle un esquema objeto-relacional con diversos tipos de objetos y diversos tipos de relaciones entre ellos.

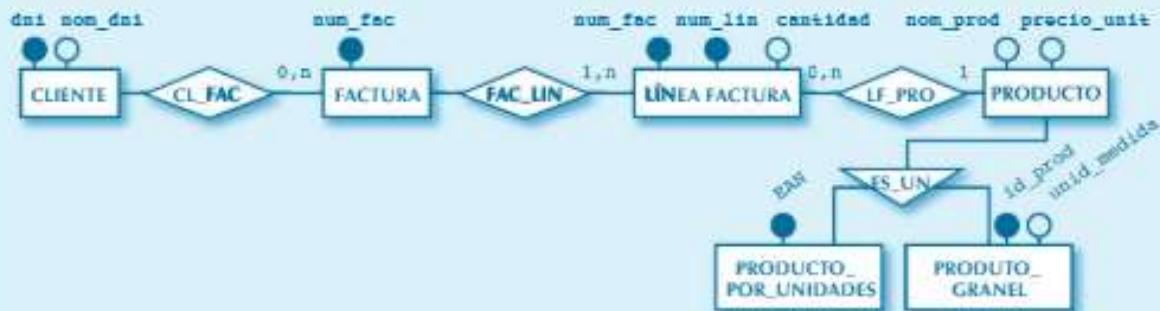
Resumen

- Como solución al desfase objeto-relacional, es decir, al conjunto de dificultades que plantea la persistencia de objetos en bases de datos relacionales, han surgido diversas soluciones. Entre ellas, las BDO, las BDOR y la correspondencia objeto-relacional.
- Las BDO permiten almacenar directamente objetos.
- La organización ODMG desarrolló estándares para BDO, entre los que están ODL y OQL. No se incluye entre ellos ningún lenguaje para manipulación de objetos, sino *language bindings* para lenguajes de programación orientados a objetos –entre ellos Java–, que hacen posible la persistencia transparente. Es decir, se gestionan de igual manera los objetos persistentes y transitorios, y en el momento de confirmar una transacción se reflejan los cambios realizados sobre los objetos en la base de datos.
- ODL permite especificar los atributos de los objetos, sus métodos y las relaciones entre objetos, que pueden ser de tipo uno a uno, uno a mucho o muchos a muchos.
- En SQL:99 se introdujeron tipos estructurados definidos por el usuario, entre los que están los tipos objetos (clases), además de tipos de colecciones, tablas anidadas y referencias.
- Las BDOR, como Oracle, se basan en SQL:99 y amplían la funcionalidad de una base de datos relacional con tipos estructurados definidos por el usuario, entre los que están los objetos.



Ejercicios propuestos

1. En el siguiente diagrama E-R se representan las relaciones entre clientes, facturas y productos incluidos en las facturas. Cada línea de factura viene identificada por la línea de factura y un número secuencial dentro de la propia factura. Interesa guardar ambos, dado que aparecen en las facturas impresas y el orden de las líneas es relevante. Los productos son de dos tipos: los que se venden por unidades, identificados por su EAN, y los que se venden a granel (por kilos, litros, etc.), identificados por un código que puede contener letras y números. Para estos últimos interesa guardar la unidad de medida a la que se aplica el precio unitario. Crea un esquema de objetos para él, la definición del esquema de objetos en ODL de Matisse, e importa el esquema en una nueva base de datos en Matisse.



2. Crea un programa en Java que cree varios clientes y productos y varias facturas para varios clientes, cada una de ellas con varias líneas. Debe haber alguna factura que contenga líneas para productos tanto por unidades como a granel. Será necesario generar previamente las *stub classes*.
3. Añade un método a la clase `Factura` que devuelva su importe neto, que será el resultado de sumar el importe para cada línea, que a su vez será el resultado de multiplicar el precio unitario del producto por la cantidad. Escribe un programa que lo utilice para mostrar todos los datos de todas las facturas, incluyendo el importe neto.
4. Escribe un programa que borre una factura, que debe recuperar a partir de su número de factura. Antes de borrar la factura, debe borrar sus líneas.
5. Cambia el método `deepRemove()` de la clase `Factura` para que borre una factura, borrando previamente todas sus líneas y, por último, la propia factura.
6. Escribe un programa que, utilizando el `driver JDBC` de Matisse, obtenga todas las facturas de un cliente identificado por su DNI, y muestre para cada una su número de factura y su importe total, que se obtendrá con el método desarrollado en una actividad anterior.
7. Crea un esquema objeto-relacional en Oracle para el diagrama E-R anterior. Las relaciones de uno a muchos se implementarán mediante referencias a objetos en lugar de mediante claves foráneas. Las líneas de factura se almacenarán en una tabla anidada dentro de la propia tabla de facturas, y en cada línea de factura habrá una referencia al producto.
8. Crea para Oracle, mediante sentencias de SQL, los mismos datos que se crearon en actividades anteriores para Matisse.

Borra en Oracle, mediante sentencias de SQL, la misma o las mismas facturas que se borraron en una actividad anterior mediante un programa en Java en Matisse.

ACTIVIDADES DE AUTOEVALUACIÓN

1. El objeto del estándar ODMG 3.0 es:
 - ☐ a) Definir las características que debe cumplir una BDO.
 - ☐ b) La persistencia de objetos utilizados en lenguajes orientados a objetos.
 - ☐ c) La persistencia de objetos en BDO u objeto-relacionales.
 - ☐ d) Ninguna de las opciones anteriores es correcta.

2. En SQL:99 se introdujeron:
 - ☐ a) Tipos de objetos definidos por el usuario que son clases e interfaces.
 - ☐ b) Mecanismos para facilitar la transición de las actuales bases de datos relacionales a futuras BDO.
 - ☐ c) Tipos estructurados definidos por el usuario, entre ellos objetos, arrays y tablas anidadas.
 - ☐ d) Mecanismos para implementar la persistencia de objetos en bases de datos relacionales mediante el establecimiento automático de correspondencias objeto-relacionales.
3. Un *language binding* tal como los especificados en ODMG 3.0:
 - ☐ a) Interactúa con un DML con unas funcionalidades mínimas.
 - ☐ b) Permite gestionar de la misma forma objetos persistentes y transitorios, utilizando los mecanismos ya existentes en un lenguaje de programación.
 - ☐ c) Hace innecesario un DML, pero requiere determinadas extensiones en la sintaxis de un lenguaje orientado a objetos de propósito general.
 - ☐ d) No garantiza la completitud computacional propuesta en el manifiesto de Atkinson y otros en 1989.
4. Un *traversal path* en ODL:
 - ☐ a) Solo se define para relaciones de uno a muchos o de muchos a muchos.
 - ☐ b) Se define en extremos de una relación con multiplicidad mayor que 1.
 - ☐ c) Se definen en ambos extremos de una relación binaria.
 - ☐ d) Se implementan siempre mediante colecciones, ordenadas o desordenadas.
5. El lenguaje OQL:
 - ☐ a) Tiene las sentencias UPDATE y DELETE de SQL, adaptadas para objetos.
 - ☐ b) Permite hacer consultas sobre objetos persistentes almacenados en una BDO.
 - ☐ c) Solo se puede utilizar desde un *language binding*.
 - ☐ d) Es completamente independiente de los *language binding*, porque estos solo sirven para crear, modificar y borrar objetos, pero no para consultarlos.
6. Una implementación del *language binding* de Java de ODMG 3.0 garantiza que, al realizarse una operación commit para confirmar una transacción:
 - ☐ a) Se graben todos los cambios realizados en objetos persistentes, pero se ignoren los cambios realizados en cualquier objeto transitorio.
 - ☐ b) Se graben los cambios realizados sobre cualquier objeto persistente modificado, y también sobre cualquier objeto transitorio alcanzable siguiendo las referencias desde ellos.
 - ☐ c) Si siguiendo las referencias desde cualquier objeto persistente modificado se alcanza un objeto no persistente, se produzca un error y se deshagan todos los cambios realizados.
 - ☐ d) Se graben todos los cambios realizados durante ella, tanto en objetos persistentes como transitorios, y todos estos últimos se convierten en persistentes.
7. La base de datos Matisse:
 - ☐ a) Es una base de datos posrelacional, y solo hasta cierto punto una BDO.

- ☐ b) Es en última instancia una base de datos relacional sobre la que se implementan los objetos en una capa de *software*.
 - ☐ c) Es una BDO que tiene su propio *driver* JDBC, pero en una transacción no se pueden mezclar cambios hechos con JDBC y con los métodos de las *stub classes* del *Java binding*.
 - ☐ d) Es una BDO que tiene además un *driver* JDBC.
8. El lenguaje SQL de Matisse:
- ☐ a) Proporciona una implementación completa de SQL:99.
 - ☐ b) Solo proporciona operaciones de consulta, dado que las operaciones de creación, modificación y borrado se realizan con el *Java binding*.
 - ☐ c) No es realmente SQL, aunque sí es similar a SQL en muchos aspectos.
 - ☐ d) No es SQL, pero incluye una implementación de OQL y, al igual que OQL, es similar a SQL.
9. Oracle 12c:
- ☐ a) Es una BDOR, es decir, es a la vez una BDO y una base de datos relacional.
 - ☐ b) Es una base de datos relacional sobre la que se implementan objetos en una capa de *software*.
 - ☐ c) Es conforme a ODMG 3.0.
 - ☐ d) Todo lo anterior es cierto.
10. La base de datos Oracle:
- ☐ a) Permite tablas de objetos y columnas de objetos.
 - ☐ b) Permite tablas de objetos, pero no columnas de objetos, aunque sí columnas que contengan referencias a objetos.
 - ☐ c) Permite tablas de objetos siempre que no contengan referencias a otros objetos.
 - ☐ d) Permite tablas anidadas, siempre que no sean tablas de objetos.

SOLUCIONES:

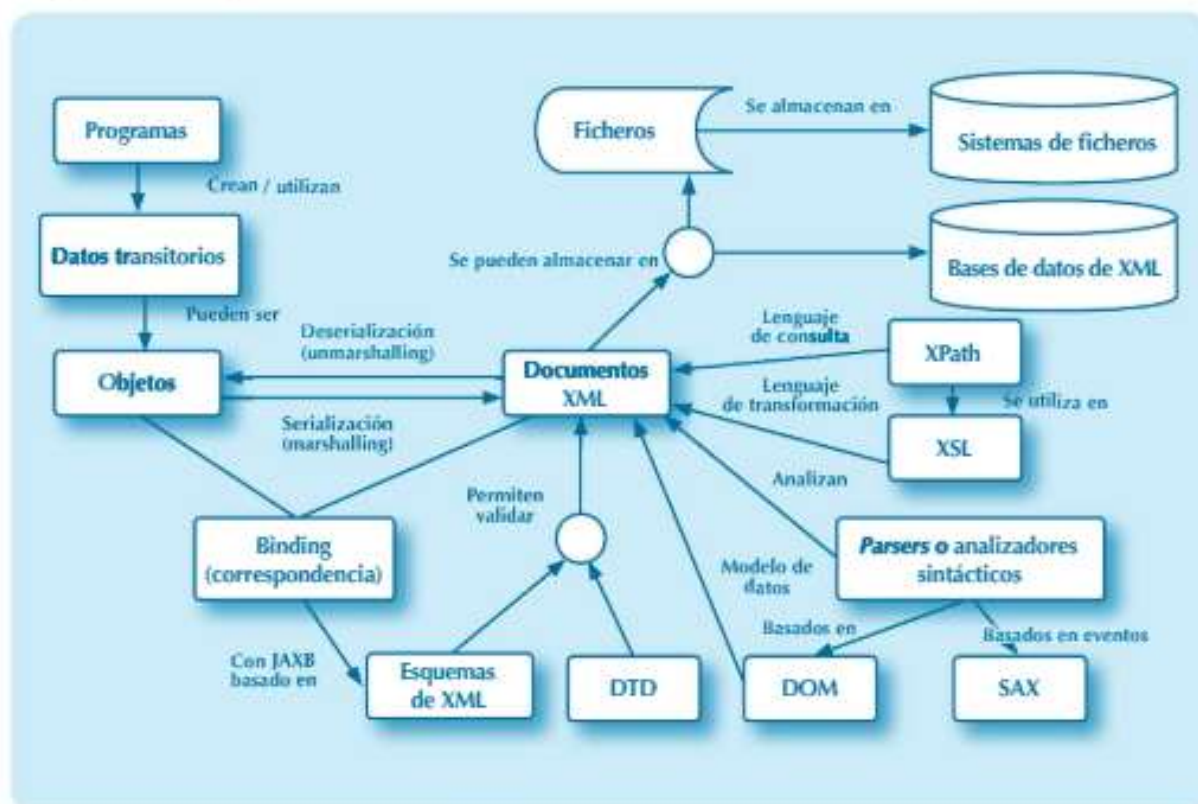
- | | | |
|---|---|--|
| 1. ☐ a ☐ b ☒ c ☐ d | 5. ☐ a ☒ b ☐ c ☐ d | 9. ☐ a ☒ b ☐ c ☐ d |
| 2. ☐ a ☐ b ☒ c ☐ d | 6. ☐ a ☒ b ☐ c ☐ d | 10. ☒ a ☐ b ☐ c ☐ d |
| 3. ☐ a ☒ b ☐ c ☐ d | 7. ☐ a ☐ b ☐ c ☒ d | |
| 4. ☐ a ☐ b ☒ c ☐ d | 8. ☐ a ☐ b ☒ c ☐ d | |

XML

Objetivos

- ✓ Entender la sintaxis del lenguaje XML.
- ✓ Conocer el modelo DOM para almacenamiento en memoria de documentos de XML.
- ✓ Utilizar *parsers* o analizadores sintácticos para XML basados en DOM y SAX.
- ✓ Analizar los mecanismos de verificación y validación de XML tales como DTD y esquemas.
- ✓ Trabajar de forma efectiva con la funcionalidad de *binding* proporcionada por JAXB para serialización (*marshalling*) y deserialización (*unmarshalling*) de objetos con documentos de XML.
- ✓ Aprender la sintaxis de XPath y utilizarlo para realizar consultas en documentos de XML.
- ✓ Comprender XSL y desarrollar plantillas sencillas para obtener documentos de diverso tipo a partir de un documento de XML, al menos documentos en formato HTML y CSV.

Mapa conceptual



Glosario

DOM. Del inglés *document object model* o "modelo de objetos para documentos". Modelo de datos para los contenidos de documentos de XML.

Esquema de XML. Documento que especifica la estructura y contenidos para un formato particular de documento de XML.

Parser o analizador sintáctico. Programa o componente de *software* que analiza un documento de texto y verifica que es sintácticamente correcto de acuerdo a la sintaxis de un lenguaje formal, como por ejemplo XML, y que permite acceder a sus contenidos, una vez analizado.

SAX. Del inglés *simple API for XML* o "analizador sencillo para XML". Analizador o *parser* de XML basado en eventos, que no genera una estructura de datos, sino que va lanzando determinados eventos conforme va identificando componentes de un documento de XML.

Serialización y deserialización de documentos XML (*unmarshalling* y *marshalling*). Generación de una estructura de objetos a partir de un documento de XML y viceversa, basada en la correspondencia entre los elementos de un esquema de XML y una colección de clases.

Validación. Proceso que permite verificar si un documento de XML sintácticamente correcto es, además, válido, es decir, conforme a una especificación para su estructura y contenidos.

XML. Lenguaje de marcas que permite representar información en documentos de texto con una estructura jerárquica.

XPath. Lenguaje de consulta sobre los contenidos de documentos XML.

XSL. Lenguaje para transformación de documentos XML a otros formatos.

6.1. El lenguaje XML

El nombre XML viene de *extensible markup language* o “lenguaje de marcado extensible”. XML es un estándar del W3C (World Wide Web Consortium), que es la organización que tiene como objetivo fundamental desarrollar y mantener el amplio cuerpo de estándares que hacen posible la web. XML ha jugado un papel fundamental en el desarrollo de nuevos servicios y aplicaciones para internet desde su introducción en 1998.

El origen del lenguaje XML está ligado a la *world wide web*, WWW o simplemente “la web”. HTML (*hypertext markup language* o “lenguaje de marcado para hipertexto”) es el lenguaje de marcado (*markup language*, en inglés) que se inventó para especificar tanto los contenidos como el aspecto visual o presentación de las páginas webs. El significado original de *markup* en inglés hace referencia a anotaciones que se hacen sobre un texto para indicar qué formato debe tener una vez publicado. Un documento de HTML sería como un texto con anotaciones (*markups*) que indican el formato o la presentación, es decir, aspectos tales como el tamaño de letra, negritas, cursivas, etc. XML surgió como un lenguaje con sintaxis similar a HTML, pero para representar exclusivamente datos de cualquier tipo. Esto fue en paralelo con la tendencia creciente a la separación entre la presentación, formato o aspecto visual de las páginas webs, por una parte, y los contenidos o datos que mostraban, por otra. Finalmente, HTML quedó como un subconjunto de XML para presentación, denominado con frecuencia XHTML, y XML, como lenguaje universal para representar cualquier tipo de datos.

Entre los principios de diseño de XML está el que sea fácil de leer e interpretar tanto por humanos como por máquinas, y esta es una de las claves de su éxito. Esto se ha conseguido a cambio de una gran *verbosidad*. Los documentos de XML ocupan mucho espacio en relación con la cantidad efectiva de información que contienen.

6.2. Estructura de un documento de XML

Un documento de XML es un documento de texto que está escrito conforme a la sintaxis del lenguaje XML. De la misma forma, por ejemplo, un fichero de programa en Java es un fichero cuyos contenidos son conformes a la sintaxis del lenguaje Java. De un fichero así se dice que está bien formado, lo que significa simplemente que es sintácticamente correcto. El concepto de “XML bien formado” no deja de ser redundante. Si no estuviera bien formado, no sería XML.

Como ejemplo de documento de XML, valga el siguiente, con datos de diferentes clientes:

```
<?xml version="1.0" encoding="UTF-8"?>
<clientes>
  <cliente DNI="78901234X">
    <apellidos>NADALES</apellidos>
    <CP>44126</CP>
  </cliente>
  <cliente DNI="89012345E">
    <apellidos>ROJAS</apellidos>
    <validez estado="borrado" timestamp="1528286082"/>
  </cliente>
  <cliente DNI="56789012B">
    <apellidos>SAMPER</apellidos>
    <CP>29730</CP>
  </cliente>
</clientes>
```

Aparte de la línea de cabecera, el documento está formado por bloques dentro de los cuales hay otros bloques, con lo que tiene una estructura jerárquica. A continuación, se describen sus distintos componentes, empleando la terminología habitual para XML:

1. *Cabecera*. En ella se identifica el documento como un documento de XML y se proporcionan algunos detalles como la versión de XML (1.0) y la codificación empleada para el texto (UTF-8). UTF-8 es la codificación por defecto, en caso de que no se indique.
2. *Elementos*. Aparte de la cabecera, todo el documento está comprendido entre una etiqueta de apertura `<clientes>` y una etiqueta de cierre `</clientes>`. Un bloque de este tipo es un elemento. Hay que tener en mente que, aunque la palabra *elemento* tiene un significado muy amplio y genérico, cuando se habla de documentos de XML es un término con un significado muy específico. Por lo tanto, en lo sucesivo se debe entender con este significado y se evitará utilizar esta palabra si no es con ese significado específico. Un elemento puede tener una única etiqueta, como es el caso de la etiqueta con nombre `validez`. Se sabe que no hay etiqueta de cierre porque al final de la etiqueta de apertura se encuentra `/>` y no `>`. Dentro de un elemento puede haber:
 - Un texto. En el caso de la primera etiqueta con nombre `apellidos`, es `NADALES`.
 - Otros elementos, y por eso los documentos de XML tienen estructura jerárquica.

En ese ejemplo no hay ningún elemento que contenga a la vez un texto y otros elementos, pero puede haberlos, como en el siguiente ejemplo:

```
<?xml version="1.0" encoding="UTF-8"?>
<comunidades>
  <comunidad>EXTREMADURA
    <provincia CAPITAL="CÁCERES">CÁCERES</provincia>
    <provincia CAPITAL="BADAJOZ">BADAJOZ</provincia>
  </comunidad>
  <comunidad>ASTURIAS
    <provincia CAPITAL="OVIEDO">ASTURIAS</provincia>
  </comunidad>
</comunidades>
```

3. *Atributos*. Aparecen en las etiquetas de apertura de los elementos, y consisten en la asignación de un valor a un nombre. En el elemento `<cliente DNI="78901234X">`, el atributo con nombre `DNI` tiene valor `78901234X`. El elemento `validez` tiene dos atributos con nombres `estado` y `timestamp`.

6.3. DOM

DOM es un modelo que define interfaces de programación para acceder a los contenidos de un documento de XML y modificarlos. En el modelo DOM, un documento de XML se representa como un árbol, y los distintos componentes de un documento de XML son nodos de dicho árbol. La figura 1.7 es el documento DOM para el documento de XML del ejemplo anterior.

En el árbol DOM hay distintos tipos de nodos. Los hay para los elementos (`clientes`, `cliente`, `apellidos`, `CP`, `validez`). También los hay para los textos, identificados como `#text`, que contienen el valor para los elementos, incluyendo los elementos para los que no hay ningún valor, a saber, `clientes`, `cliente` y `validez`. Los atributos, en el modelo DOM, no son nodos del árbol. Están en una lista asociada al nodo de su elemento. Cada entrada de esta lista contiene el nombre de un atributo y su valor.

Para concluir, existen nodos de tipo `#text` que parecen no contener ningún texto. En realidad, contienen saltos de línea y espacios en blanco, incluidos normalmente en los ficheros de XML para facilitar a las personas su lectura. Nada impediría que el documento de XML estuviera en una única línea de texto, aunque serían más incómodos de leer para las personas. Más adelante veremos la manera de ignorar estos nodos de texto innecesarios e *ignorables*.

En breve se verá cómo se puede obtener un documento DOM a partir de un documento de XML con un programa en Java.

6.4. *Parsers* o analizadores sintácticos, serialización y deserialización

Un *parser* o analizador sintáctico, para un lenguaje formal como por ejemplo XML, Java o C++, es un programa que permite verificar que un documento de texto es sintácticamente correcto, es decir, conforme a la sintaxis del lenguaje. En ese caso, además, una vez analizado el documento, permite acceder a sus contenidos. Existen *parsers* para XML como para cualquier otro lenguaje formal. Un *parser* DOM permite verificar que un documento de XML es sintácticamente correcto y si lo es, obtener un documento DOM.

El proceso de *parsing*, incluyendo la construcción de una estructura de datos que representa sus contenidos, se denomina a veces deserialización, porque a partir de un documento de texto, que es una serie o secuencia de caracteres, se obtiene una estructura de datos no secuencial. El proceso inverso, es decir, obtener una estructura secuencial, como por ejemplo un fichero, a partir de una no secuencial, se suele denominar serialización.



Figura 6.1
Parser DOM

Un *parser* DOM es muy sencillo de utilizar en Java. Con pocas líneas de código se obtiene un documento DOM a partir de un fichero XML. Disponer de todo el documento en memoria permite un acceso rapidísimo a sus contenidos. Las interfaces de programación de DOM permiten no solo consultar los contenidos de un documento, sino también modificarlos. También se puede serializar un árbol DOM, es decir, generar un fichero con sus contenidos.

Pero construir un documento DOM en memoria puede suponer un grave inconveniente para documentos muy grandes. XML se utiliza hoy en día para almacenar enormes volúmenes de datos. Por ejemplo, remesas con decenas de miles de recibos bancarios, o corpus lingüísticos con millones de entradas. Para aplicaciones que necesitan consultar los contenidos de documentos muy grandes, pero no modificarlos, conviene utilizar otro tipo de *parser* que permita acceder a ellos sin almacenar el documento en memoria, como por ejemplo un *parser* SAX.

Java incluye soporte para XML y varias tecnologías relacionadas en su API JAXP (Java API for XML Processing). La mayoría de ellas (DOM, SAX y XSLT) se verán en el resto de este capítulo.

www

Recurso web

Tutorial de Oracle sobre JAXP: <http://docs.oracle.com/javase/tutorial/jaxp>.

6.5. DOM con Java

Todo lo necesario para trabajar con los distintos componentes del modelo DOM está en el paquete `org.w3c.dom`. Se puede ver que es de la organización W3C (su nombre empieza por `org.w3c`, no por `java` o `javax`).

Un documento DOM puede crearse a partir de un documento de XML, mediante un proceso de *parsing*. O también puede crearse directamente en memoria, empezando con un documento vacío y añadiendo elementos.

www

Recurso web

Se pueden consultar los JavaDocs del paquete `org.w3c.dom` en <http://docs.oracle.com/javase/8/docs/api>.

Las principales interfaces para los distintos elementos del modelo DOM son las siguientes:

- **Document**: representa un documento de XML.
- **Node**: representa un nodo de un documento de XML. La interfaz **Document** implementa la interfaz **Node**. Eso representa el hecho de que un documento DOM está definido por su nodo raíz y algunas cosas más.
- **Element**: representa un elemento.
- **NodeList**: representa una lista de nodos. Es útil para recuperar la lista de nodos de un nodo, como por ejemplo la lista de elementos de un **Element**.

- **NamedNodeMap**: al igual que **NodeList**, representa una lista de nodos, pero con algunas diferencias. Se utiliza para listas de atributos (interfaz **Attr**). En ellas, los atributos se identifican por el nombre. No puede haber dos con el mismo nombre, y el orden en que aparecen es irrelevante.

En lo sucesivo se describirán brevemente estas interfaces y sus principales métodos, para hacerse una idea general de su funcionalidad y comprender los programas de ejemplo. Para realizar algunas de las actividades propuestas será necesario consultar su documentación.

En el cuadro 6.1 se muestran algunos métodos de la interfaz **Document** para consulta.

CUADRO 6.1
Métodos de la interfaz **Document** para consulta

Método	Funcionalidad
<code>String getXmlEncoding()</code>	Devuelve la codificación. Podría ser, por ejemplo, "UTF-8".
<code>String getXmlVersion()</code>	Devuelve la versión de XML. Podría ser, por ejemplo, "1.0".

En el cuadro 6.2 se muestran algunos métodos de la interfaz **Node** que son útiles para consulta de un documento DOM.

CUADRO 6.2
Métodos de la interfaz **Node** para consulta

Método	Funcionalidad
<code>short getNodeType()</code>	Devuelve el tipo de nodo. Algunos de los posibles valores que puede devolver son: DOCUMENT_NODE : se trata del nodo raíz del documento. ELEMENT_NODE : se trata de un elemento. TEXT_NODE : se trata de un texto.
<code>String getNodeName()</code>	Devuelve el nombre del nodo. Si es un elemento, devuelve el nombre del elemento. Si es un texto, devuelve "#text". Si es un atributo, devuelve el nombre del atributo.
<code>String getNodeValue()</code>	Devuelve el valor asociado al nodo. Si es un texto, devuelve el texto. Si es un atributo, devuelve el valor del atributo.
<code>NodeList getChildNodes()</code>	Devuelve la lista de nodos hijos. Relevante para nodos de tipo Element .
<code>NamedNodeMap getAttributes()</code>	Si el nodo es de tipo Element , devuelve una lista con sus atributos. Si no, devuelve null . Devuelve un NamedNodeMap y no un NodeList , porque entre los atributos de un Element el orden no es relevante, y se identifican solo por su nombre.

[.../...]

CUADRO 6.2 (CONT.)

<code>Node getParentNode()</code>	Devuelve el nodo padre si existe, o <code>null</code> en caso contrario. Los nodos de algunos tipos de nodos, como <code>Document</code> o <code>Attribute</code> , no tienen nodo padre.
-----------------------------------	---

6.5.1. Parsing DOM

El paquete `javax.xml.parsers` proporciona toda la funcionalidad para el *parser* DOM y también para el *parser* SAX, que se verá más adelante.

La clase `DocumentBuilder` implementa un *parser* DOM. Se obtiene una instancia suya con la clase `DocumentBuilderFactory`, mediante su método de clase `newInstance()`. A su vez, se obtiene una instancia de `DocumentBuilderFactory` mediante su método de clase `newDocumentBuilder()`. En los cuadros siguientes se indican los métodos más importantes de ambas clases. Algunos métodos de `DocumentBuilderFactory` especifican opciones por defecto para los *parsers* que construye.

CUADRO 6.3

Algunos de los principales métodos de la clase `DocumentBuilderFactory`

Método	Funcionalidad
<code>static DocumentBuilderFactory newInstance()</code>	Crea una instancia de la clase.
<code>DocumentBuilder newDocumentBuilder()</code>	Devuelve un <i>parser</i> DOM.
<code>void setIgnoringComments (boolean ignoreComm)</code>	Ignora comentarios. A no ser que se ignoren, se incluyen como nodos dentro del árbol DOM.
<code>void setIgnoringElementContentWhitespace (boolean whitespace)</code>	Ignora textos vacíos en el contenido de los elementos si el <i>parser</i> está en modo de validación, que se verá más adelante. En caso contrario, no tiene ningún efecto.

En el cuadro 6.4 se explican los métodos más importantes de `DocumentBuilder`. Esta clase no solo sirve para obtener documentos DOM mediante el *parsing* de un documento DOM, sino también para crearlos desde cero. Existe un método para crear un documento DOM vacío y métodos para añadir distintos tipos de elementos, como se verá más adelante.

CUADRO 6.4

Métodos de la clase `DocumentBuilder`

Método	Funcionalidad
<code>abstract Document newDocument()</code>	Devuelve un nuevo documento DOM vacío inicialmente.

[.../...]

CUADRO 6.4 (CONT.)

```
Document parse(File f)
Document parse(InputStream is)
Document parse(String uri)
```

Construye un documento DOM a partir de los contenidos de un fichero (**File**), a partir de un **InputStream**, o a partir de un URI. El URI puede ser, por ejemplo, una dirección desde la que obtener el documento mediante HTTP, y en ese caso empezaría por `http://` y tendría la forma habitual para ese tipo de URI.

El siguiente programa obtiene un documento DOM a partir de un fichero XML y muestra sus contenidos. El *parsing* en sí es sencillo, cinco líneas de código para crear y configurar un **DocumentBuilderFactory**, crear un **DocumentBuilder** e invocar su método **parse()**. El grueso del programa es la parte que muestra los contenidos del documento, a saber, el método **muestraNodo**. Para mostrar el árbol entero se le pasa el nodo raíz. El parámetro **nivel** (0 para el nodo raíz) se utiliza para mostrar los contenidos del nodo con la indentación apropiada. Una vez se han mostrado los detalles del nodo, se obtiene la lista de nodos hijos y se muestran sus contenidos llamando de nuevo a **muestraNodo** para cada uno de ellos, incrementando **nivel** en uno. Se hace una verificación previa para ignorar los nodos de texto que solo tienen espacios en blanco o saltos de línea, **muestraNodo** escribe a un **PrintStream** que se le pasa como parámetro. Esto permite, de manera muy sencilla, dirigir la salida a un fichero cualquiera, por ejemplo, en lugar de a **System.out**.

```
// Parsing DOM y visualización de documento DOM generado
package dom_parser;
import java.io.File;
import java.io.InputStream;
import java.io.FileNotFoundException;
import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.parsers.DocumentBuilder;
import javax.xml.parsers.ParserConfigurationException;
import org.w3c.dom.Document;
import org.w3c.dom.Node;
import org.w3c.dom.NodeList;
import org.w3c.dom.NamedNodeMap;
import org.xml.sax.SAXException;
public class DOM_Parser {
    private static final String INDENT_NIVEL = " "; // Para indentación
    public static void muestraNodo(Node nodo, int nivel, PrintStream ps) {
        if (nodo.getNodeType() == Node.TEXT_NODE) { // Ignora textos vacíos
            String text = nodo.getNodeValue();
            if (text.trim().length() == 0) {
                return;
            }
        }
        for (int i = 0; i < nivel; i++) { // Indentación
            ps.print(INDENT_NIVEL);
        }
        switch (nodo.getNodeType()) { // Escribe información de nodo según
            tipo
            case Node.DOCUMENT_NODE: // Documento
                Document doc=(Document) nodo;
                ps.println("Documento DOM, versión: "+doc.getXmlVersion()+
                    ", codificación: "+doc.getXmlEncoding());
                break;
```

```

        case Node.ELEMENT_NODE: // Elemento
            ps.print("<" + nodo.getNodeName());
            NamedNodeMap listaAtr = nodo.getAttributes(); // Lista atributos
            for (int i = 0; i < listaAtr.getLength(); i++) {
                Node atr = listaAtr.item(i);
                ps.print(" @" + atr.getNodeName() + "[" + atr.getNodeValue()
                    + "]");
            }
            ps.println(">");
            break;
        case Node.TEXT_NODE: // Texto
            ps.println(nodo.getNodeName() + "[" + nodo.getNodeValue() +
                "]");
            break;
        default: // Otro tipo de nodo
            ps.println("(nodo de tipo: " + nodo.getNodeType() + ")");
    }
    NodeList nodosHijos = nodo.getChildNodes(); // Muestra nodos hijos
    for (int i = 0; i < nodosHijos.getLength(); i++) {
        muestraNodo(nodosHijos.item(i), nivel + 1, ps);
    }
}

public static void main(String[] args) {
    String nomFich;
    if (args.length < 1) {
        System.out.println("Indicar por favor nombre de fichero");
        return;
    } else {
        nomFich = args[0];
    }
    DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();
    dbf.setIgnoringComments(true);
    dbf.setIgnoringElementContentWhitespace(true);
    try {
        DocumentBuilder db = dbf.newDocumentBuilder();
        Document domDoc = db.parse(new File(nomFich));
        muestraNodo(domDoc, 0, System.out);
    } catch (FileNotFoundException | ParserConfigurationException |
        SAXException e) {
        System.err.println(e.getMessage());
    } catch (Exception e) {
        e.printStackTrace();
    }
}
}
}

```

A continuación, se muestra la salida del programa anterior al lado del fichero XML. Se puede ver que no se muestran nodos de texto que solo tengan espacios en blanco.

<pre> <?xml version="1.0" encoding="UTF-8"?> <clientes> <cliente DNI="78901234X"> <apellidos>NADALES</apellidos> <CP>44126</CP> </cliente> </pre>	<pre> Documento DOM, versión: 1.0, codificación: UTF-8 <clientes> <cliente @DNI[78901234X]> <apellidos> #text[NADALES] </pre>
---	---

<pre> <cliente DNI="89012345E"> <apellidos>ROJAS</apellidos> <validez estado="borrado" timestamp="1528286082"/> </cliente> <cliente DNI="56789012B"> <apellidos>SAMPER</apellidos> <CP>29730</CP> </cliente> </clientes> </pre>	<pre> <CP> #text[44126] <cliente @DNI[89012345E]> <apellidos> #text[ROJAS] <validez @estado[borrado] @ timestamp[1528286082]> <cliente @DNI[56789012B]> <apellidos> #text[SAMPER] <CP> #text[29730] </pre>
---	--



Actividad propuesta 6.1

Modifica el programa anterior para que escriba la salida a un fichero con nombre `parsing_dom.txt`. Como mejora, puedes utilizar la clase `SimpleDateFormat` para hacer que en el nombre de fichero se incluyan la fecha y la hora, de manera similar a como se hacía en un programa de ejemplo del capítulo anterior dedicado a ficheros.

6.5.2. Creación de documentos con DOM

Ahora se mostrará cómo se puede crear desde cero un documento DOM, y cómo se puede escribir un documento DOM a un fichero de XML.

Se pueden crear nodos de distintos tipos con algunos métodos de la interfaz `Document`. Se usarán estos para crear los nodos, y métodos de la interfaz `Node` para añadirlos al documento.

CUADRO 6.5

Métodos de la interfaz `Document` para crear nuevos nodos para un documento DOM

Método	Funcionalidad
<code>Element createElement(String nombreEtiqueta)</code>	Crea nuevo elemento con el nombre dado.
<code>Text createTextNode(String texto)</code>	Crea nuevo nodo de tipo texto.

Los atributos se añaden con métodos de la interfaz `Element`.

CUADRO 6.6

Métodos de la interfaz `Element`

Método	Funcionalidad
<code>void setAttribute(String name, String value)</code>	Añade atributo con el valor dado.

A continuación, algunos métodos de la interfaz `Node` que permiten añadir elementos a un documento DOM. Existen más métodos para eliminar o reemplazar elementos, y para cambiar

valores y propiedades, que no se explican aquí. Si alguna de estas operaciones no se puede realizar, típicamente lanzará una excepción de tipo `DOMException`.

CUADRO 6.7

Métodos de la interfaz `Node` para añadir elementos a un documento DOM

Método	Funcionalidad
<code>Node appendChild(Node nuevoHijo)</code>	Añade nodo como último nodo hijo, y devuelve el nodo añadido.
<code>Node insertBefore(Node nuevoHijo, Node refHijo)</code>	Añade nuevo hijo antes del indicado.

Antes de poner un programa de ejemplo para construir un documento DOM, se verá cómo serializar un documento DOM para poder verificarlo fácilmente una vez creado. Serializar un documento DOM es generar un documento de XML textual a partir de él.

6.5.3. Serialización de documentos DOM

La clase `Transformer` permite serializar documentos DOM, es decir, generar documentos de texto XML a partir de ellos.

Se obtienen instancias de `Transformer` mediante el método `newTransformer()` de la clase `TransformerFactory`. A su vez, se obtienen instancias de `TransformerFactory` mediante su método de clase `newInstance()`.

Por lo demás, se pueden controlar las opciones para la serialización mediante el método `setOutputProperty`, y se realiza la serialización mediante el método `transform(Source xmlSource, Result outputTarget)`.

Las opciones para serialización son muy numerosas y se pueden consultar en la documentación de la clase. Son básicamente las recogidas en el estándar XSL. Más adelante en este capítulo se verá algo de XSL.

El siguiente programa crea un documento DOM con parte de los contenidos del documento de XML de ejemplo, y después lo serializa en un `string` mediante un `StringWriter`. Se indica codificación UTF-8 para la serialización.

```
// Creación de árbol DOM añadiendo elementos y serialización una vez creado
package creardocumentodom;

import java.io.StringWriter;
import org.w3c.dom.Document;
import org.w3c.dom.Element;
import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.parsers.DocumentBuilder;
import javax.xml.parsers.ParserConfigurationException;
import javax.xml.transform.dom.DOMSource;
import javax.xml.transform.TransformerFactory;
import javax.xml.transform.Transformer;
import javax.xml.transform.OutputKeys;
```

```

import javax.xml.transform.stream.StreamResult;

public class CrearDocumentoDOM {

    public static void main(String[] args) {

        try {
            DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();
            DocumentBuilder db = dbf.newDocumentBuilder();
            Document doc = db.newDocument();
            doc.setXmlVersion("1.0");

            Element elClientes = doc.createElement("clientes");
            doc.appendChild(elClientes);

            Element elCliente = doc.createElement("cliente");
            elCliente.setAttribute("DNI", "89012345E");
            Element elApell = doc.createElement("apellidos");
            elApell.appendChild(doc.createTextNode("ROJAS"));
            elCliente.appendChild(elApell);
            Element elValidez = doc.createElement("validez");
            elValidez.setAttribute("estado", "borrado");
            elValidez.setAttribute("timestamp", "1528286082");
            elCliente.appendChild(elValidez);

            elClientes.appendChild(elCliente);

            DOMSource domSource = new DOMSource(doc);
            Transformer transformer = TransformerFactory.newInstance().
                newTransformer();
            transformer.setOutputProperty(OutputKeys.METHOD, "xml");
            transformer.setOutputProperty(OutputKeys.ENCODING, "UTF-8");
            transformer.setOutputProperty(OutputKeys.INDENT, "yes");
            transformer.setOutputProperty(
                "{http://xml.apache.org/xslt}indent-amount", "4");

            StringWriter sw = new StringWriter();
            StreamResult sr = new StreamResult(sw);
            transformer.transform(domSource, sr);
            System.out.println(sw.toString());
        } catch (ParserConfigurationException e) {
            System.err.println(e.getMessage());
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

La salida del programa anterior es la siguiente:

```

<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<clientes>
  <cliente DNI="89012345E">
    <apellidos>ROJAS</apellidos>
    <validez estado="borrado" timestamp="1528286082"/>
  </cliente>
</clientes>

```

Actividad propuesta 6.2



Crea un programa que realice el *parsing* DOM del fichero de ejemplo con los datos de varios clientes. Una vez hecho esto, debe buscar un cliente cualquiera por DNI y cambiarle sus apellidos. Después, debe hacer una copia del fichero añadiendo al nombre ".bak". Seguido, debe utilizar la clase `Transformer` para serializar el documento DOM al fichero, sobrescribiendo sus contenidos. Tanto el DNI como el nuevo valor para `apellidos` se pueden asignar en variables del programa.

6.6. SAX

Ya se ha comentado que un *parser* DOM crea un documento DOM en memoria, y esto puede ser un inconveniente importante cuando se trabaja con documentos muy grandes.

En ese caso, puede ser más conveniente utilizar un *parser* SAX, que no construye una estructura de datos en memoria. En lugar de ello, cada vez que sucede un determinado tipo de evento durante el proceso de *parsing*, llama a determinados métodos de una clase especial manejadora de eventos. Estos se definen a conveniencia para la aplicación que se quiera desarrollar. También llama a métodos manejadores de errores y avisos cuando se da alguna situación que así lo requiera.

Se puede considerar que un *parser* SAX es un *parser* mínimo. Se limita a identificar eventos y, para cada evento, invocar un método manejador para el tipo de evento. Para utilizar un *parser* SAX hay que proporcionar una o varias clases que implementen distintas interfaces para manejar distintos tipos de eventos. Lo más sencillo es gestionarlos todos mediante un `DefaultHandler`, porque esta clase implementa todas estas interfaces, y esto es lo que se hará aquí. En `DefaultHandler` los métodos manejadores de eventos no hacen nada. Hay que redefinirlos a conveniencia en una subclase suya.

La clase `XMLReader` implementa un *parser* SAX. Se pueden construir instancias suyas mediante la clase `XMLReaderFactory`. Una vez creado un `XMLReader` se le asocian instancias de clases manejadoras de eventos y, por último, se invoca su método `parse()`. A continuación, se explican las clases y métodos necesarios para el siguiente ejemplo.



Figura 6.2
Parser SAX

CUADRO 6.8
Métodos de la clase XMLReader

Método	Funcionalidad
<pre>void parse(InputSource input) void parse(String idSistema)</pre>	Realiza el <i>parsing</i> de un documento de XML. Estos métodos permiten obtener fácilmente el documento a partir de un identificador de sistema, un <code>InputStream</code> o un <code>Reader</code> . Un identificador de sistema puede ser un nombre de fichero o un URI. En este último caso, se podría obtener, por ejemplo, de un servidor web mediante http.
<pre>void setContentHandler(ContentHandler handler)</pre>	Asocia al <i>parser</i> un manejador de eventos para contenidos. Hay varias clases que implementan <code>ContentHandler</code> , pero lo más sencillo es utilizar <code>DefaultHandler</code> para gestionar todos los eventos.

Los métodos de `DefaultHandler` suelen ir por parejas: uno para el inicio de un componente y otro para el final. Ya se ha dicho que en `DefaultHandler` no hacen nada.

CUADRO 6.9
Métodos de la clase DefaultHandler

Método	Funcionalidad
<pre>void startDocument() void endDocument()</pre>	Inicio y fin del documento (<code>Document</code>).
<pre>void startElement(String uri, String nombreLocal, String nombreCualif, Attributes atributos) void endElement(String uri, String nombreLocal, String nombreCualif)</pre>	Inicio y fin de un elemento (<code>Element</code>). Si no se usan <i>namespaces</i> , el nombre del elemento está en <code>localName</code> .
<pre>void characters(char[] chars, int inicio, int longitud)</pre>	Texto dentro de un elemento (<code>Element</code>). Se proporciona el texto del documento de XML, y la posición inicial y la posición final del texto. Se puede obtener el texto para el elemento con <code>String(chars, inicio, fin)</code> .

El siguiente programa de ejemplo utiliza un *parser* SAX para mostrar la misma información que mostraba el anterior programa de ejemplo que utilizaba un *parser* DOM. Los métodos que se han desarrollado son muy sencillos. Solo es necesario dar respuesta adecuada a cada evento redefiniendo los métodos apropiados en `GestorEventos`, clase hija de `DefaultHandler`. No hay nada en este programa realmente complicado, es incluso más sencillo que el programa anterior basado en DOM. Pero puede costar más entender cómo encajan todas las piezas en el funcionamiento global del *parser* SAX. Se utiliza la anotación `@Override` para indicar que se redefine un método de la clase padre. Esto no es estrictamente necesario, pero sí aconsejable.

```
// Parsing con SAX
package sax_parser;
import java.io.PrintStream;
import org.xml.sax.helpers.DefaultHandler;
import org.xml.sax.helpers.XMLReaderFactory;
import org.xml.sax.XMLReader;
import org.xml.sax.SAXException;
import org.xml.sax.Attributes;
import javax.xml.parsers.SAXParserFactory;
import javax.xml.parsers.SAXParser;
class GestorEventos extends DefaultHandler {
    private static final String INDENT_NIVEL = " "; // Para indentación
    private final PrintStream ps;
    private int nivel;
    private void indenta() {
        for (int i = 0; i < nivel; i++) { // Indentación
            ps.print(INDENT_NIVEL);
        }
    }
    public GestorEventos(PrintStream ps) {
        this.ps = ps;
    }
    @Override
    public void startDocument() {
        nivel = 0;
    }
    @Override
    public void startElement(String uri, String nombreLocal, String
        nombreCualif, Attributes atributos) {
        nivel++;
        indenta();
        ps.print("<" + nombreCualif);
        for (int i=0; i<atributos.getLength(); i++) {
            ps.print(" @" + atributos.getLocalName(i) + "[" + atributos.
                getValue(i) + "]");
        }
        ps.println(">");
    }
    @Override
    public void endElement(String uri, String nombreLocal, String
        nombreCualif) {
        nivel--;
    }
    @Override
    public void characters(char[] cars, int inicio, int longitud) {
        String texto = new String(cars, inicio, longitud);
        if (texto.trim().length() == 0) {
            return;
        }
        nivel++;
        indenta();
        nivel--;
        ps.println("#text[" + texto + "]");
    }
}
public class SAX_Parser {
    public static void main(String[] args) {
        String nomFich;
    }
}
```

```

if (args.length < 1) {
    System.out.println("Indicar por favor nombre de fichero");
    return;
} else {
    nomFich = args[0];
}
try {
    XMLReader parserSAX = XMLReaderFactory.createXMLReader();
    GestorEventos gestorEventos = new GestorEventos(System.out);
    parserSAX.setContentHandler(gestorEventos);
    parserSAX.parse(nomFich);
} catch (SAXException e) {
    System.err.println(e.getMessage());
} catch (Exception e) {
    e.printStackTrace();
}
}
}

```

6.7. Validación de documentos de XML

Un documento de XML bien formado es, además, válido si satisface un conjunto de restricciones adicionales sobre su estructura y contenidos. La estructura se refiere a qué tipo de elementos puede tener, dónde y con qué atributos. El contenido se refiere a qué valores puede tener el contenido de cada tipo de elemento y cada atributo. Hay distintos mecanismos para validación de documentos de XML, de los que los más importantes son:

1. *DTD*. Son las siglas de *data type definition* o definición de tipos de datos. Fue el primer mecanismo de validación para XML. De hecho, es previo a XML y se creó para SGML, un precedente de HTML y XML. Hoy en día su uso ha decaído en favor de los esquemas de XML.
2. *Esquemas de XML*. Es un mecanismo mucho más potente. Los esquemas se almacenan en ficheros XSD (XML Schema Definition), que son documentos de XML.

6.7.1. Validación con DTD

DTD permite especificar la estructura y el contenido de un documento de XML. En lo referente a estructura, qué elementos (identificados por su nombre) pueden aparecer, dónde y cuántas veces, y qué atributos pueden tener. En lo referente a contenido, qué valores se admiten para cada elemento y atributo. Su funcionalidad es mucho más limitada que los esquemas de XML y tiene una sintaxis diferente, al contrario que los esquemas, que son documentos de XML.

Recurso digital



En el anexo web 6.1, disponible en www.sintesis.com y accesible con el código indicado en la primera página del libro, encontrarás más información sobre DTD.

6.7.2. Validación con esquemas de XML

Los esquemas de XML son documentos de XML que permiten verificar o validar los contenidos de un documento de XML.

A continuación, se incluye un esquema de XML para el documento de ejemplo con datos de clientes. Al final del documento se puede ver cómo los esquemas de XML permiten especificar todo tipo de restricciones sobre los valores aceptables para cada elemento o atributo.

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema version="1.0"
  xmlns:xs="http://www.w3.org/2001/XMLSchema"
  elementFormDefault="qualified">
  <xs:element name="clientes">
    <xs:complexType>
      <xs:sequence>
        <xs:element ref="cliente" minOccurs="0" maxOccurs="unbounded"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
  <xs:element name="cliente">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="apellidos" type="xs:string"/>
        <xs:element name="CP" minOccurs="0" type="tipo_CP"/>
        <xs:element ref="validez" minOccurs="0"/>
      </xs:sequence>
      <xs:attribute name="DNI" use="required" type="tipo_DNI"/>
    </xs:complexType>
  </xs:element>
  <xs:element name="validez">
    <xs:complexType>
      <xs:attribute name="estado" use="required" type="tipo_estado"/>
      <xs:attribute name="timestamp" use="optional" type="tipo_timestamp"/>
    </xs:complexType>
  </xs:element>
  <xs:simpleType name="tipo_CP">
    <xs:restriction base="xs:string">
      <xs:pattern value="\d{5}"/>
    </xs:restriction>
  </xs:simpleType>
  <xs:simpleType name="tipo_timestamp">
    <xs:restriction base="xs:string">
      <xs:pattern value="\d{10}"/>
    </xs:restriction>
  </xs:simpleType>
  <xs:simpleType name="tipo_DNI">
    <xs:restriction base="xs:string">
      <xs:pattern value="[0-9XYZ]\d{7}[\dA-Z]"/>
    </xs:restriction>
  </xs:simpleType>
  <xs:simpleType name="tipo_estado">
    <xs:restriction base="xs:string">
      <xs:enumeration value="borrado"/>
      <xs:enumeration value="vigente"/>
    </xs:restriction>
  </xs:simpleType>
</xs:schema>
```

Se puede especificar el esquema para validación dentro del propio documento de XML y añadir una referencia al esquema dentro de la etiqueta de apertura del elemento `clientes`, con lo que quedaría así:

```
<?xml version="1.0" encoding="UTF-8"?>
<clientes xmlns:xsi='http://www.w3.org/2001/XMLSchema-instance'
  xsi:noNamespaceSchemaLocation='clientes.xsd'>
</clientes>
```

Esto puede ser necesario o cómodo para validar el XML con algunas herramientas, porque automáticamente se localiza y utiliza el esquema. Pero puede plantear problemas con otras herramientas, y en particular con los programas de ejemplo que se desarrollarán en breve. En un sistema real el esquema estaría normalmente situado en un servidor web y se accedería a él no por un nombre de fichero local, sino mediante un URI de tipo `http://`.



Actividad propuesta 6.3

Dado que un esquema de XML es un tipo de documento de XML, ¿piensas que puede tener sentido o utilidad un esquema de XML para verificar los contenidos de esquemas de XML? ¿En qué situaciones? Justifica tu respuesta.

6.8. Parsing con validaciones con Java

Las clases `DocumentBuilderFactory` para *parsing* DOM y `SAXParserFactory` para *parsing* SAX tienen métodos para que el *parser* valide los XML con un DTD o esquema. De esta manera, se producirán excepciones si el XML no es válido.

CUADRO 6.10
Métodos de la clase `DocumentBuilderFactory` y `SAXParserFactory` para validación

Método	Funcionalidad
<code>setValidating(boolean validating)</code>	Realiza validación utilizando un DTD si se llama con valor <code>true</code> para <code>validating</code> . El DTD debe estar referenciado desde o incluido en el mismo XML.
<code>void setSchema(Schema schema)</code>	Especifica el esquema que se va a utilizar para validar los documentos.
<code>void setIgnoringElementContentWhitespace(boolean whitespace)</code>	Solo en el caso de que se realice validación, hará que se ignoren los textos vacíos o <i>ignorables</i> en el contenido de los elementos.

Aparte de utilizar estos métodos, debe hacerse una apropiada gestión de los errores de validación que se puedan producir. No hay que confundir estos con los errores de sintaxis, que se producen si el documento de XML no está bien formado, es decir, no es correcto sintácticamente. Aunque pueda parecer un tanto extraño, para gestionar los errores de validación hay que utilizar el sistema de gestión de eventos de SAX, aunque se utilice un *parser* DOM. Esto significa, por cierto, que para la implementación del *parser* DOM se utiliza internamente un *parser* SAX.

CUADRO 6.11

Métodos de la clase `DefaultHandler` para gestión de errores y avisos de validación

Método	Funcionalidad
<pre>void error(SAXParseException e) throws SAXException void fatalError(SAXParseException e) throws SAXException</pre>	Error recuperable o no recuperable durante el <i>parsing</i> . En el primer caso, se puede continuar el <i>parsing</i> . En el segundo, no se puede.
<pre>void warning(SAXParseException e) throws SAXException</pre>	Aviso durante el <i>parsing</i> .

A continuación se muestra un ejemplo de *parsing* DOM con validación. El fichero XML se le pasa mediante el primer parámetro de línea de comandos. Si se le pasa un fichero con un esquema de XML en el segundo parámetro, la validación se hace con ese esquema, y en otro caso, con un DTD que debe estar referenciado desde el propio fichero XML.



TOMA NOTA

Los atributos que se pueden añadir a elementos de un documento de XML relativos a esquemas, como los del siguiente ejemplo, no son necesarios para los programas de ejemplo que siguen, porque en ellos se valida el documento entero con un esquema de XML indicado en línea de comandos.

```
<clientes xmlns:xsi=
    'http://www.w3.org/2001/XMLSchema-instance'
    xsi:noNamespaceSchemaLocation='clientes.xsd'>
```

Además, harán que falle siempre la validación. Si están, hay que eliminarlos, y dejar:

```
<clientes>
```

La validación con DTD se consigue con `setValidating(true)`. Con esquemas de XML habría que utilizar, en cambio, `setSchema(Schema schema)`. Los errores y avisos se gestionan con un objeto de una clase manejadora de eventos, hija de `DefaultHandler`, la misma que se usó para el ejemplo de *parsing* SAX. A sus métodos manejadores de errores (y avisos) se les pasa un `SAXParseException` que se lanza de nuevo con `throws`. En el programa principal se capturan las excepciones de este tipo y no se hace nada con ellas. Para avisos y errores recuperables, podría no relanzarse la excepción para seguir con el *parsing*.

```
// Parsing DOM con validación basada en DTD
package DOMParserValid;
import java.io.File;
import java.io.PrintStream;
import org.xml.sax.SAXException;
import org.xml.sax.SAXParseException;
import org.xml.sax.helpers.DefaultHandler;
import org.w3c.dom.Document;
import org.w3c.dom.Node;
import org.w3c.dom.NodeList;
import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.parsers.DocumentBuilder;
import javax.xml.parsers.ParserConfigurationException;
import javax.xml.validation.SchemaFactory;
import javax.xml.XMLConstants;

public class DOMParserValid {
    private static final String INDENT_NIVEL = " "; // Para indentación
    public static void muestraNodo(Node nodo, int nivel, PrintStream ps) {
        for (int i = 0; i < nivel; i++) {
            ps.print(INDENT_NIVEL);
        }
        switch (nodo.getNodeType()) { // Escribe información de nodo según
            tipo
            case Node.DOCUMENT_NODE: // Documento
                ps.println("DOCUMENTO");
                break;
            case Node.ELEMENT_NODE: // Elemento
                ps.println("ELEMENTO(" + nodo.getNodeName() + ")");
                break;
            case Node.TEXT_NODE: // Texto
                ps.println(nodo.getNodeName() + "[" + nodo.getNodeValue() +
                    "]");
                break;
        }
        NodeList nodosHijos = nodo.getChildNodes(); // Nodos hijos
        for (int i = 0; i < nodosHijos.getLength(); i++) {
            muestraNodo(nodosHijos.item(i), nivel + 1, ps);
        }
    }
    public static void main(String[] args) {
        File f = null, fEsq = null;
        if (args.length < 1) {
            System.out.println("Indicar por favor nombre de fichero");
            return;
        } else {
            String nomFich = args[0];
            f = new File(nomFich);
            if (!f.isFile()) {
                System.err.println("Fichero " + nomFich + " no existe.");
                return;
            }
            if (args.length >= 2) {
                String nomFichEsquema = args[1];
                fEsq = new File(nomFichEsquema);
                if (!fEsq.isFile()) {

```

```

        System.err.println("Fichero " + nomFichEsquema + " no
                           existe.");
        return;
    }
}
}
DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();
dbf.setIgnoringComments(true);
dbf.setIgnoringElementContentWhitespace(true);
System.out.println("Fichero XML: " + f.getAbsolutePath());
if (fEsq != null) { // Validar con esquema
    System.out.println("Validación con esquema: "+fEsq.getAbsolutePath());
    try {
        dbf.setSchema(
            SchemaFactory.newInstance(
                XMLConstants.W3C_XML_SCHEMA_NS_URI).newSchema(fEsq)
        );
    } catch (SAXException e) {
        System.err.println(e.getMessage());
        return;
    }
} else { // Validar con DTD
    System.out.println("Validación con DTD");
    dbf.setValidating(true);
}
try {
    DocumentBuilder db = dbf.newDocumentBuilder();
    db.setErrorHandler(new GestorEventos());
    Document domDoc = db.parse(f);
    muestraNodo(domDoc, 0, System.out);
} catch (ParserConfigurationException e) {
    System.err.println(e.getMessage());
} catch (SAXParseException e) { // Ya gestionada en GestorEventos
} catch (Exception e) {
    e.printStackTrace();
}
}
}
class GestorEventos extends DefaultHandler {
    @Override
    public void error(SAXParseException e) throws SAXParseException {
        System.err.println("Error recuperable: " + e.toString());
        throw (e);
    }
    @Override
    public void fatalError(SAXParseException e) throws SAXParseException {
        System.err.println("Error no recuperable: " + e.toString());
        throw (e);
    }
    @Override
    public void warning(SAXParseException e) throws SAXParseException {
        System.err.println("Aviso: " + e.toString());
        throw (e);
    }
}
}

```

Para *parsing* SAX se haría de manera análoga, pero utilizando `SAXParserFactory` y `SAXParser` en lugar de `DocumentBuilderFactory` y `DocumentBuilder`. Hay, además, alguna otra pequeña variación. Por ejemplo, es necesario proporcionar dos clases manejadoras distintas: una para contenidos y otra para errores y avisos. Pero nada impide usar la misma, siempre que en ella estén definidos los métodos manejadores de errores y eventos. Así se ha hecho en el siguiente programa de ejemplo. Como en el anterior programa, el fichero XML se pasa mediante el primer argumento de línea de comandos. Si se pasa un fichero con un esquema de XML en el segundo, la validación se hace con ese esquema, y en otro caso con un DTD que debe estar referenciado desde el propio fichero XML.

```
// Parsing SAX con validación basada en DTD
package SAXParserValid;
import java.io.File;
import java.io.PrintStream;
import javax.xml.XMLConstants;
import org.xml.sax.XMLReader;
import org.xml.sax.SAXException;
import org.xml.sax.Attributes;
import org.xml.sax.SAXParseException;
import org.xml.sax.helpers.DefaultHandler;
import javax.xml.parsers.SAXParserFactory;
import javax.xml.parsers.SAXParser;
import javax.xml.validation.SchemaFactory;

class GestorEventos extends DefaultHandler {
    private static final String INDENT_NIVEL = " "; // Para indentación
    private final PrintStream ps;
    private int nivel;
    private void indenta() {
        for (int i = 0; i < nivel; i++) {
            ps.print(INDENT_NIVEL);
        }
    }
    public GestorEventos(PrintStream ps) {
        this.ps = ps;
    }
    @Override
    public void startDocument() {
        nivel = 0;
        ps.println("(DOCUMENTO)");
    }
    @Override
    public void startElement(String uri, String nombreLocal, String
        nombreCualif, Attributes atributos) {
        nivel++;
        indenta();
        ps.println("ELEMENTO(" + nombreCualif + ")");
    }
    @Override
    public void endElement(String uri, String nombreLocal, String
        nombreCualif) {
        nivel--;
    }
}
```

```

@Override
public void characters(char[] cars, int inicio, int longitud) {
    String texto = new String(cars, inicio, longitud);
    nivel++;
    indenta();
    nivel--;
    ps.println("TEXTO(" + texto + ")");
}
@Override
public void error(SAXParseException e) throws SAXParseException {
    System.err.println("Error recuperable: " + e.toString());
    throw (e);
}
@Override
public void fatalError(SAXParseException e) throws SAXParseException {
    System.err.println("Error no recuperable: " + e.toString());
    throw (e);
}
@Override
public void warning(SAXParseException e) throws SAXParseException {
    System.err.println("Aviso: " + e.toString());
    throw (e);
}
}
public class SAXParserValid {
    public static void main(String[] args) {
        File f = null, fEsq = null;
        if (args.length < 1) {
            System.out.println("Indicar por favor nombre de fichero");
            return;
        } else {
            String nomFich = args[0];
            f = new File(nomFich);
            if (!f.isFile()) {
                System.err.println("Fichero " + nomFich + " no existe.");
                return;
            }
            if (args.length >= 2) {
                String nomFichEsquema = args[1];
                fEsq = new File(nomFichEsquema);
                if (!fEsq.isFile()) {
                    System.err.println("Fichero " + nomFichEsquema + " no existe.");
                    return;
                }
            }
        }
        try {
            SAXParserFactory spf = SAXParserFactory.newInstance();
            System.out.println("Fichero XML: " + f.getAbsolutePath());
            if (fEsq != null) { // Validar con esquema
                System.out.println("Validación con esquema: " + fEsq.getAbsolutePath());
            }
        }
    }
}

```

```

    try {
        spf.setSchema(
            SchemaFactory.newInstance(
                XMLConstants.W3C_XML_SCHEMA_NS_URI).newSchema(fEsq)
            );
    } catch (SAXException e) {
        System.err.println(e.getMessage());
        return;
    }
} else { // Validar con DTD
    System.out.println("Validación con DTD");
    spf.setValidating(true);
}
SAXParser parserSAX = spf.newSAXParser();
XMLReader xmlReader = parserSAX.getXMLReader();
GestorEventos gestorEventos = new GestorEventos(System.out);
xmlReader.setContentHandler(gestorEventos);
xmlReader.setErrorHandler(gestorEventos);
xmlReader.parse(f.getAbsolutePath());
} catch (SAXException e) {
    System.err.println(e.getMessage());
} catch (Exception e) {
    e.printStackTrace();
}
}
}

```



Actividad propuesta 6.4

Prueba los dos programas anteriores (parsers DOM y SAX con validación). Utiliza la validación mediante esquema de XML. Puedes utilizar el documento de prueba proporcionado con datos de clientes o cualquier otro documento de XML, siempre que dispongas de su correspondiente esquema o que lo crees tú mismo. Cambia el documento de XML para introducir diversos errores de sintaxis y de validación en el documento de XML, y comprueba qué errores se detectan. Verifica qué sucede si se cambia el esquema de XML para introducir errores en él. Modifica los manejadores de eventos para que solo se relancen las excepciones en caso de aviso o de error recuperable, de manera que se pueda seguir validando para mostrar más avisos o errores. Recuerda: CTRL+Z para deshacer cambios, CTRL+Y para rehacer cambios.

6.9. Binding con JAXB

JAXB proporciona una API que permite realizar de manera muy sencilla la traducción de un documento de XML a una colección de objetos (*unmarshalling* o deserialización), y a la inversa, la traducción de una colección de objetos a un documento de XML (*marshalling* o serialización).



Figura 6.3

Binding con JAXB para persistencia de objetos en documentos de XML

El paso previo para utilizar JAXB es la generación —a partir de un esquema de XML y utilizando el compilador de *binding* `xjc`— de una colección de clases de Java que representan sus elementos y las relaciones entre ellos, y de interfaces y clases que implementan los procesos de *marshalling* y *unmarshalling* (incluyendo este último el *parsing* de los ficheros XML y su validación con el esquema de XML).

6.9.1. Esquemas de XML para *binding*

Para que se pueda navegar sin problemas desde cualquier objeto creado mediante el proceso de *unmarshalling* a cualesquiera otros relacionados, es aconsejable que cualquier elemento (`<xs:element>`) que pueda contener otros elementos, o bien repetirse varias veces (en cualquiera de estos dos casos se define con `<xs:complexType>`), tenga una definición aparte y no incluida dentro de la de otro elemento. Siempre que se necesite se puede hacer referencia a su definición mediante `<xs:element ref= "... ">`. Así se garantiza que el compilador de *binding* crea las clases apropiadas para ellos. Se puede verificar que el esquema de ejemplo anterior es conforme a estas recomendaciones.

```

<?xml version="1.0" encoding="UTF-8"?>
<xs:schema version="1.0"
  xmlns:xs="http://www.w3.org/2001/XMLSchema"
  elementFormDefault="qualified">
  <xs:element name="clientes">
    <xs:complexType>
      <xs:sequence>
        <xs:element ref="cliente" minOccurs="0" maxOccurs="unbounded"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
  <xs:element name="cliente">
    <xs:complexType>
      <xs:sequence>
        (...)
      </xs:sequence>
      <xs:attribute name="DNI" use="required" type="tipo_DNI"/>
    </xs:complexType>
  </xs:element>
  (...)
</xs:schema>

```

Figura 6.4

Esquema para un documento de XML apropiado para JAXB

6.9.2. Compilador de *binding*

El compilador de *binding* es un programa llamado `xjc` que se puede usar directamente desde línea de comandos o indirectamente desde un IDE (entorno de desarrollo integrado) como NetBeans o Eclipse. Se desarrollará ahora un programa de ejemplo para el esquema de ejemplo `clientes.xsd`, empezando por la generación de las clases a partir del esquema mediante el compilador de *binding*.

Recurso digital



En el anexo web 6.2, disponible en www.sintesis.com y accesible con el código indicado en la primera página del libro, encontrarás un aviso para los usuarios de Windows.

- Desde línea de comandos:

```
xjc clientes.xsd
```

Se pueden utilizar opciones para, entre otras cosas, indicar el nombre de paquete y el directorio donde se quiere que se generen las clases e interfaces.

- Desde el entorno de desarrollo NetBeans: se añade el esquema de XML (fichero `xsd`) al proyecto desde el que se quiere utilizar JAXB. Después se selecciona el menú “File”, opción “New File”, se selecciona “XML” y después “JAXB Binding”, o bien se pulsa con el botón derecho y se selecciona “New” y después “JAXB Binding”.

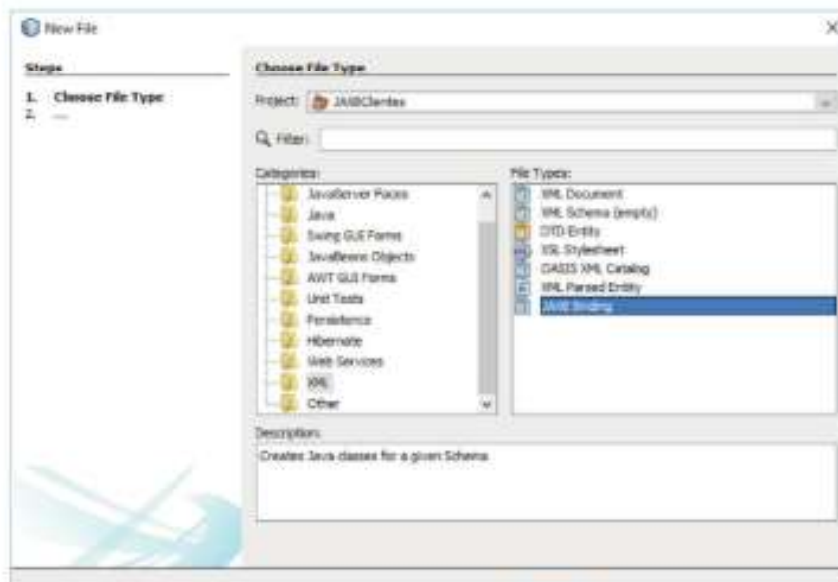


Figura 6.5
Creación de *binding* para esquema de XML con Netbeans

En el siguiente diálogo hay que indicar el fichero de esquema, el nombre para el contexto de *binding* y el nombre de un paquete para incluir las clases e interfaces creadas por el compila-

lador de *binding* `xjc`. Para estos dos últimos se ha especificado `clientes_JAXB`. Se muestran estas opciones y cómo queda el proyecto.

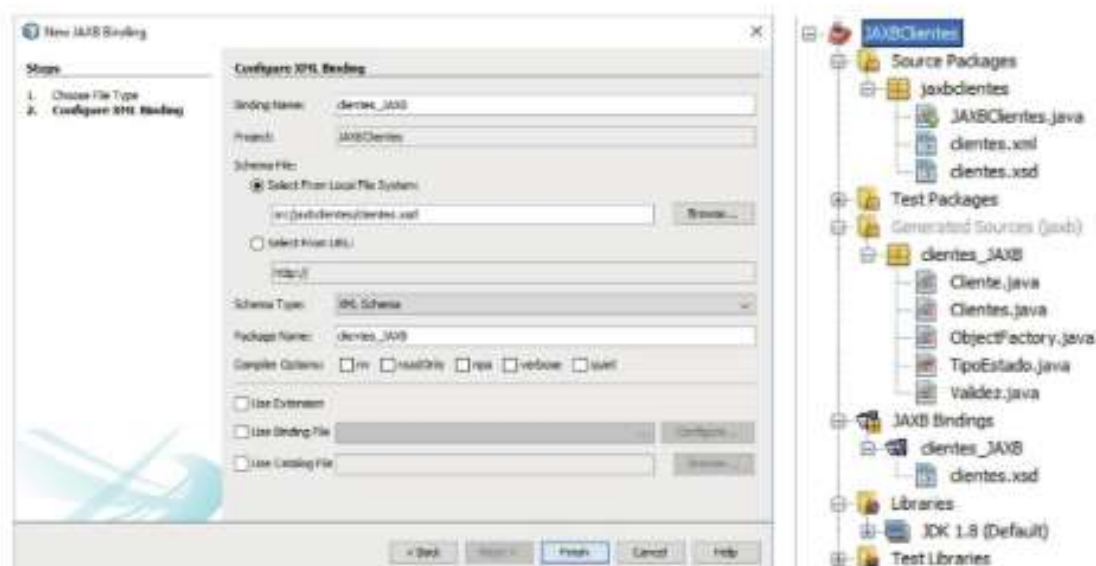


Figura 6.6
Configuración del *binding* y proyecto final

6.9.3. Clases generadas por el compilador de *binding*

Todas las clases generadas por el compilador de *binding* disponen de métodos `get` y `set` (*getters* y *setters*) para obtener y asignar los valores de cada atributo y de cada elemento que no sea de un tipo complejo `<xs:complexType>`. Si dentro del elemento `<xs:element>` al que está asociada una clase hay otro elemento `<xs:element>` que se puede repetir muchas veces (atributo `maxOccurs` de `<xs:element>` con valor mayor que uno o `unbounded`), habrá un método `get` que devuelve una lista de objetos de la clase correspondiente al otro elemento, no existirá el correspondiente método `set`, y los contenidos de la lista deberán modificarse con los métodos de la interfaz `List`. En el cuadro 6.12 se muestran todos los métodos.

CUADRO 6.12
Getters y setters generados por el compilador de *binding*

Clase	Métodos
Clientes	<code>public List<Cliente> getCliente()</code>
Cliente	<code>public String getApellidos()</code> <code>public void setApellidos(String value)</code> <code>public String getCP()</code> <code>public void setCP(String value)</code> <code>public Validez getValidez()</code> <code>public void setValidez(Validez value)</code> <code>public String getDNI()</code> <code>public void setDNI(String value)</code>

[.../...]

CUADRO 6.12 (CONT.)

Validez	<pre> public TipoEstado getEstado() public void setEstado(TipoEstado value) public String getTimestamp() public void setTimestamp(String value) </pre>
---------	--

Se ha generado otra clase `TipoEstado` para el elemento `validez` de tipo simple (`<x:simpleType>`) basado en una enumeración (`<x:enumeration>`). Su uso es muy sencillo y se verá en un ejemplo posterior.

6.9.4. Unmarshalling (deserialización) con JAXB

Ahora se está en condiciones de escribir programas que utilicen la funcionalidad de *binding* para acceder a los datos en ficheros de XML conformes al esquema de XML. La API JAXB está disponible en el paquete `javax.xml.bind`.

A continuación, se muestra un programa de ejemplo para ilustrar la funcionalidad de JAXB. Primero se crea un `JAXBContext` asociado a la clase creada por el compilador de *binding* para el elemento al nivel más alto del esquema, `clientes_JAXB.Clientes.class` (`clientes_JAXB` es el paquete creado por el compilador de *binding*). A continuación, se crea un `Unmarshaller` asociado a este contexto. Después, se le asocia un esquema para validación y un manejador de eventos para errores y avisos. Por último, se invoca el método `unmarshal`, que devuelve un objeto de tipo `clientes_JAXB.Clientes` para todos los contenidos del fichero XML, a los que se puede acceder mediante los métodos `get` de las clases creadas por el compilador de *binding*. Los eventos que se van a gestionar pueden ser de tres tipos: error no recuperable, error recuperable o aviso. Todos se gestionan en un único método, y si devuelve `true`, se continúa con el proceso. En este programa se continúa a menos que se trate de un error no recuperable. En caso de error recuperable o aviso, se devuelve `true` para poder continuar con el proceso e informar de más errores o avisos, en caso de que los haya.

```

// Unmarshalling con JAXB para mostrar contenidos de un documento de XML
package jaxbclientes;

import java.util.List;
import java.io.File;
import javax.xml.XMLConstants;
import javax.xml.validation.SchemaFactory;
import javax.xml.bind.JAXBContext;
import javax.xml.bind.Unmarshaller;
import javax.xml.bind.JAXBException;
import javax.xml.bind.UnmarshalException;
import javax.xml.bind.ValidationEvent;
import javax.xml.bind.ValidationEventHandler;

public class JAXBClientes {

    public static void main(String[] args) {
        File f = null, fEsq = null;
        if (args.length < 1) {
            System.out.println("Indicar por favor nombre de fichero");
            return;
        }
    }
}

```

```

    } else {
        String nomFich = args[0];
        f = new File(nomFich);
        if (!f.isFile()) {
            System.err.println("Fichero " + nomFich + " no existe.");
            return;
        }
        if (args.length >= 2) {
            String nomFichEsquema = args[1];
            fEsq = new File(nomFichEsquema);
            if (!fEsq.isFile()) {
                System.err.println("Fichero " + nomFichEsquema + " no
                    existe.");
                return;
            }
        }
        System.out.println("Fichero XML: " + f.getAbsolutePath());
        if (fEsq != null) { // Validar con esquema
            System.out.println("Valid. con esquema: " + fEsq.
                getAbsolutePath());
        }
        try {
            JAXBContext contextoClientes = JAXBContext.newInstance(
                clientes_JAXB.Clientes.class // Contexto para clase Clientes
            );
            // Unmarshaller para contexto
            Unmarshaller u = contextoClientes.createUnmarshaller();
            u.setSchema(SchemaFactory.newInstance( // Esquema para validación
                XMLConstants.W3C_XML_SCHEMA_NS_URI).newSchema(fEsq));
            u.setEventHandler(new GestorEventos()); // Gestor de eventos
            clientes_JAXB.Clientes clientes = (clientes_JAXB.Clientes)
                u.unmarshal(f); // unmarshalling, que incluye parsing y
                validación
            // Iteración sobre objeto para mostrar los contenidos
            List<clientes_JAXB.Cliente> listaClientes = clientes.getClientes();
            for (clientes_JAXB.Cliente cliente: listaClientes) {
                System.out.println("=> DNI: " + cliente.getDNI());
                System.out.println("Apellidos: " + cliente.getApellidos());
                String cp = cliente.getCP();
                if (cp != null) {
                    System.out.println("CP: " + cp);
                }
                clientes_JAXB.Validez validez = cliente.getValidez();
                if (validez != null) {
                    System.out.println("Validez: " + validez.getEstado());
                    String timestamp = validez.getTimestamp();
                    if (timestamp.length() > 0) {
                        System.out.println("timestamp: " + timestamp);
                    }
                }
            }
        } catch (UnmarshallerException e) {
            System.err.println(e.getMessage());
        }
    }
}

```

```

    } catch (JAXBException e) {
        System.err.println(e.getMessage());
    } catch (Exception e) {
        e.printStackTrace();
    }
}

class GestorEventos implements ValidationEventHandler {
    @Override
    public boolean handleEvent(ValidationEvent e) {
        System.err.println("Evento de validación (" + e.getSeverity() + ") "
            + "[línea: " + e.getLocator().getLineNumber() + ", "
            + "columna: " + e.getLocator().getColumnNumber() + "]: "
            + e.getMessage());
    }
    return (e.getSeverity() != ValidationEvent.FATAL_ERROR);
}
}

```



Actividad propuesta 6.5

Creas un nuevo esquema de XML a partir del esquema de ejemplo para clientes, y añades para cada cliente la posibilidad de especificar en un elemento opcional `max_descuento` una cantidad entera entre 1 y 60 que represente el máximo porcentaje de descuento para el cliente (si no sabes cómo, debes investigar acerca de XML Schema). Vuelves a generar las clases a partir del esquema y cambias el programa para que muestre este porcentaje para los clientes en los que esté informado.

6.9.5. *Marshalling* (serialización) con JAXB

A continuación se muestra un programa que utiliza JAXB para crear un fichero XML con los datos de dos clientes. Para construir la estructura de objetos se utilizan los *setters* de las clases generadas por el compilador de *binding*. Para añadir elementos en una secuencia se utilizan los métodos de la interfaz *List*, típicamente `add()`, para añadir al final de la lista. Para asignar los posibles valores del atributo `estado` se utilizan constantes definidas en `clientes_JAXB.TipoEstado`. Para serializar la estructura de objetos se crea un contexto `JAXBContext` para la clase `clientes_JAXB.Clientes`. Después, se crea un `Marshaller` para este contexto y, por último, se genera el fichero con el método `marshal()`.

```

// Marshalling con JAXB para generar un documento de XML
package jaxbclientescrear;

import java.io.File;
import java.util.List;
import javax.xml.bind.JAXBContext;
import javax.xml.bind.Marshaller;
import javax.xml.bind.JAXBException;

public class JAXBClientesCrear {

    public static void main(String[] args) {

```

```

File f = null;
if (args.length < 1) {
    System.out.println("Indicar por favor nombre de fichero");
    return;
} else {
    String nomFich = args[0];
    f = new File(nomFich);
    if (f.isFile()) {
        System.err.println("Fichero "+nomFich+" existe, no se hace nada");
        return;
    }
}

clientes_JAXB.Clientes clientes = new clientes_JAXB.Clientes();
List<clientes_JAXB.Cliente> listaClientes = clientes.getClientes();

clientes_JAXB.Cliente cliente = new clientes_JAXB.Cliente();
cliente.setDNI("78901234X");
cliente.setApellidos("NADALES");
cliente.setCP("44126");
listaClientes.add(cliente);

cliente = new clientes_JAXB.Cliente();
cliente.setDNI("89012345E");
cliente.setApellidos("ROJAS");
clientes_JAXB.Validez validez = new clientes_JAXB.Validez();
validez.setEstado(clientes_JAXB.TipoEstado.VIGENTE);
validez.setTimestamp("1528286082");
cliente.setValidez(validez);
listaClientes.add(cliente);

try {
    JAXBContext contextoClientes = JAXBContext.newInstance(
        clientes_JAXB.Clientes.class // Crea contexto para objeto
        Clientes
    );

    Marshaller m=contextoClientes.createMarshaller(); // Crea
        marshaller

    m.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true);
    m.marshal(clientes, f);

    System.out.println("Generado fichero XML: " + f.getAbsolutePath());
} catch (JAXBException e) {
    System.err.println(e.getMessage());
} catch (Exception e) {
    e.printStackTrace();
}
}
}

```

Actividad propuesta 6.6



Modifica el programa anterior para que muestre el documento de XML en pantalla, en vez de crear un fichero.

6.10. El lenguaje de consulta XPath

XPath es un lenguaje de consulta para documentos de XML. Fue desarrollado inicialmente para el lenguaje XSLT, el cual se verá en el siguiente apartado. En él se basan otros estándares de XML que también se verán más adelante, como XQuery, que se estudiará en un capítulo posterior dedicado a bases de datos de XML.

Una expresión (o consulta) de XPath permite recuperar contenidos de un documento de XML representado en forma de árbol. Una expresión de XPath devuelve un conjunto de nodos. Esto recuerda al modelo DOM. La especificación de XPath incluye su propio modelo de datos, muy similar al modelo DOM, pero con algunas diferencias. Las implementaciones de XPath en Java y, en general, en todos los lenguajes de programación se basan en DOM.

Una expresión de XPath consta de una secuencia de pasos de búsqueda, separados por el carácter “/”. La búsqueda comienza por el nodo raíz del árbol, al que se le aplica el primer paso, lo que proporciona una primera lista de nodos como resultado. Cada paso sucesivo se aplica sobre la lista de resultados del paso anterior, lo que proporciona una nueva lista de resultados. Como resultado del último paso se obtiene la lista de resultados de la expresión de XPath.

Se muestra un ejemplo del funcionamiento de XPath. Supongamos que se quieren recuperar todos los DNI de clientes que tengan informado el código postal. Esto se podría hacer con la expresión `/clientes/cliente/CP/../../@DNI`. El paso “..” se utiliza para obtener el nodo padre, “@” se utiliza para obtener el valor de un atributo.

<pre><?xml version="1.0" encoding="UTF-8"?> <!DOCTYPE clientes SYSTEM "clientes. dtd"> <clientes> <cliente DNI="78901234X"> <apellidos>NADALES</apellidos> <CP>44126</CP> </cliente> <cliente DNI="89012345E"> <apellidos>ROJAS</apellidos> <validex estado="borrado" timestamp="1528286082"/> </cliente> <cliente DNI="56789012B"> <apellidos>SAMPER</apellidos> <CP>29730</CP> </cliente> </clientes></pre>	<pre>/clientes [<clientes>] /clientes/cliente [<cliente DNI="78901234X">...</cliente>, <cliente DNI="89012345E">...</cliente>, <cliente DNI="56789012B">...</cliente>] /clientes/cliente/CP [<CP>44126</CP>, <CP>29730</CP>] /clientes/cliente/CP/.. [<cliente DNI="78901234X">...</cliente>, <cliente DNI="56789012B">...</cliente>] /clientes/cliente/CP/../../@DNI ["78901234X", "56789012B"]</pre>
---	--

Figura 6.7
Resultados de una consulta de XPath, explicada paso a paso

A continuación se muestra un programa en Java que permite obtener los resultados de una expresión de XPath sobre un documento de XML. Después, se incluirán varios ejemplos de consultas de XPath, cuyos resultados se podrán obtener utilizando este programa.

A este programa se le pasa un nombre de fichero XML y una expresión de XPath como argumentos de línea de comandos, o bien desde consola (`BufferedReader` construido sobre

`System.in`), y muestra la lista de resultados obtenidos. Primero, se hace un *parsing* DOM del documento para obtener un `Document` con sus contenidos. Después se crea un `XPathExpression` a partir de un `String` que contiene la expresión de XPath. Por último, `evaluate` devuelve en un `NodeList` la lista de resultados de aplicar el XPath sobre el `Document`. Hay otras maneras de obtener los resultados, pero en este programa se ha seleccionado la opción `XPathConstants.NODESET`. Para visualizar el contenido de los nodos, incluyendo el de sus nodos hijos, se utiliza el método `getTextContent` de la clase `Node`.

```
// Consulta en un documento de XML mediante XPath
package XPathConsulta;

import java.io.File;
import java.io.BufferedReader;
import java.io.InputStreamReader;
import org.w3c.dom.Document;
import org.w3c.dom.NodeList;
import org.xml.sax.SAXException;
import javax.xml.xpath.XPathFactory;
import javax.xml.xpath.XPathExpression;
import javax.xml.xpath.XPathException;
import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.xpath.XPathConstants;

public class XPathConsulta {

    public static void main(String[] args) {

        String nomFich, xPath = "";
        File fXML;

        if (args.length < 1) {
            System.out.println("Por favor, indicar nombre de fichero.");
            return;
        } else {
            nomFich = args[0];
            System.out.println("Fichero XML: " + nomFich);
            if (args.length < 2) {
                System.out.print("Introducir XPath: ");
                BufferedReader br = new BufferedReader(
                    new InputStreamReader(System.in));
                try {
                    xPath = br.readLine();
                } catch (Exception e) {}
            } else {
                xPath = args[1];
            }
        }
        fXML = new File(nomFich);
        if (!fXML.isFile()) {
            System.err.println("Fichero " + nomFich + " no existe.");
            return;
        }
        System.out.println("Fichero XML: " + fXML.getAbsolutePath() + " XPath: "
            + xPath);
    }
}
```

```

try {
    Document docXML = DocumentBuilderFactory.newInstance().
        newDocumentBuilder().parse(fXML);
    XPathExpression xPathExp = XPathFactory.newInstance().newXPath().
        compile(xPath);
    NodeList resultados = (NodeList) xPathExp.evaluate(
        docXML, XPathConstants.NODESET);
    for (int i = 0; i < resultados.getLength(); i++) {
        System.out.printf("[%d]: %s\n", i + 1,
            resultados.item(i).getTextContent());
    }
    if (resultados.getLength() < 1) {
        System.out.println("No se obtuvo ningún resultado.");
    }
} catch (SAXException | XPathException e) {
    System.err.println(e.getMessage());
} catch (Exception e) {
    e.printStackTrace();
}
}
}

```

En el cuadro 6.13 se recogen algunos ejemplos más de consultas con XPath.

CUADRO 6.13
Varios ejemplos de XPath

<code>//@DNI</code>	El paso "vacío" obtiene todos los nodos por debajo del nodo actual. Este XPath muestra todos los valores de cualquier atributo con nombre "DNI", en cualquier elemento.
<code>/clientes/*/@DNI</code>	El paso * obtiene todos los nodos por debajo del nodo actual.
<code>/clientes/cliente[apellidos="SAMPER"]</code>	Obtiene elemento cliente para cliente que tiene como apellidos "SAMPER".
<code>/clientes/cliente[@DNI="89012345E"]</code>	Obtiene elemento cliente para el cliente con DNI 89012345E.
<code>/clientes/cliente[CP=44126]/apellidos</code>	Obtiene apellidos de cliente con código postal 44126.
<code>//validez/@*</code>	Obtiene valor de todos los atributos de cualquier elemento de nombre validez.
<code>/clientes/cliente[CP>=29000 and CP<30000]</code>	Devuelve elementos cliente de clientes con código postal mayor o igual que 29000 y menor que 30000.
<code>/clientes/cliente[position()=2]</code>	Devuelve el nombre del segundo cliente.
<code>/clientes/cliente[position()=last()]/@DNI</code>	Devuelve el DNI del último cliente.

Actividades propuestas



- 6.7.** Modifica el programa anterior para que muestre todos los contenidos de los nodos devueltos por cada consulta, utilizando un método como `muestraNodo`, utilizado en los programas de ejemplo anteriores para *parsing* DOM. Se trata, simplemente, de copiar el método y hacerlo funcionar en este programa. Prueba distintas opciones para devolver los resultados de la búsqueda con XPath. En particular, otros valores en lugar de `XPathConstants.NODESET`, como `XPathConstants.NODE` o `XPathConstants.STRING`. Pueden ser necesarios cambios adicionales en el programa, al cambiar el tipo de objeto devuelto (ya no sería un `NodeList`). Piensa en qué situaciones podrían ser útiles estas opciones.
- Prueba con los XPath anteriores y experimenta con otros nuevos, y con otros ficheros de XML.
- 6.8.** Investiga la forma de definir una clave primaria para un elemento en un esquema de XML con `<xs:key>` y XPath. Aplícalo sobre el esquema de ejemplo para datos de clientes para verificar que no hay DNI duplicados.

6.11. El lenguaje XSL

XSL (del inglés *extended stylesheet language*) es un lenguaje de transformación que permite generar nuevos documentos a partir de un documento de XML. A los documentos en lenguaje XSL se les suele llamar plantillas XSL, y son documentos de XML.



Figura 6.8
El lenguaje de transformación XSL

El nombre *lenguaje de hojas de estilo* viene del uso para el que fue concebido originalmente: la generación, a partir de unos mismos datos (en un documento de XML), de distintos documentos en lenguaje HTML o similares, según la plantilla utilizada, para adaptar la presentación

a distintos dispositivos, desacoplando así los contenidos (en XML) de la presentación (en XSL). Pero XSL permite generar documentos en cualquier formato de texto, además de en HTML o en XML, como por ejemplo en formato CSV.

Sería muy largo explicar todas las posibilidades del lenguaje XSL. Baste indicar que es un lenguaje muy potente con sentencias de asignación, condicionales, bucles, posibilidad de definir funciones, etc. Se muestran dos plantillas de ejemplo para el documento de XML con datos de clientes, una al lado de la otra para facilitar su comparación. Una genera HTML, que se puede mostrar en un navegador web. La otra genera un documento de tipo CSV, pero con campos, y por ende registros, de longitud fija (y rellena con espacios por la derecha con `concat` y `substring`), y con el carácter “|” como separador de campos y con un salto de línea como separador de registros. O, visto de otra manera, un fichero con campos y registros de longitud fija como los vistos en el capítulo anterior dedicado a ficheros, pero con separador de campos y de registros.

```
<?xml version="1.0" encoding="UTF-8"?>
<!--
Plantilla para generar HTML
-->
<xsl:stylesheet xmlns:xsl="http://
www.w3.org/1999/XSL/Transform"
version="1.0">
  <xsl:output method="html"
encoding="UTF-8"/>
  <xsl:template match="/">
    <html>
      <head>
        <title>Lista de clientes
        </title>
      </head>
      <body>
        <xsl:apply-templates
select="clientes"/>
      </body>
    </html>
  </xsl:template>
  <xsl:template match="clientes">
    <h1>Clientes</h1>
    <table border="1">
      <thead>
        <tr>
          <th>DNI</th>
          <th>Apellidos</th>
          <th>CP</th>
        </tr>
      </thead>
      <tbody>
        <xsl:apply-templates
select="cliente"/>
      </tbody>
    </table>
  </xsl:template>
```

```
<?xml version="1.0" encoding="UTF-8"?>
<!--
Plantilla para generar CSV
-->
<xsl:stylesheet xmlns:xsl="http://
www.w3.org/1999/XSL/Transform"
version="1.0">
  <xsl:output method="text"
encoding="UTF-8"/>
  <xsl:strip-space elements="*" />
  <xsl:variable
name="salto_linea">
    <xsl:text>&#xa;</xsl:text>
  </xsl:variable>
  <xsl:variable
name="separador_campos">
    <xsl:text>|</xsl:text>
  </xsl:variable>
  <xsl:template match="/">
    <xsl:apply-templates
select="clientes"/>
  </xsl:template>
  <xsl:template match="clientes">
    <xsl:apply-templates
select="cliente"/>
  </xsl:template>
  <xsl:template match="cliente">
    <xsl:if test="not(validez/@
estado='borrado')">
      <xsl:value-of select="@
DNI"/>
      <xsl:apply-templates/>
      <xsl:value-of
select="$salto_linea"/>
    </xsl:if>
  </xsl:template>
```

<pre> <xsl:template match="cliente"><xsl:if test="not(validez/@ estado='borrado')"> <tr><td><xsl:value-of select="@DNI"/> </td> <xsl:apply-templates/> </tr> </xsl:if> </xsl:template> <xsl:template match="apellidos CP"> <td><xsl:apply-templates/> </td> </xsl:template> </xsl:stylesheet> </pre>	<pre> <xsl:template match="apellidos"> <xsl:value-of select="\$separador_campos"/> <xsl:value-of select="substring(concat(., ' ', 1, 20)"/> </xsl:template> <xsl:template match="CP"> <xsl:value-of select="\$separador_campos"/> <xsl:value-of select="substring(concat(., ' ', 1, 5)"/> </xsl:template> </xsl:stylesheet> </pre>
--	--

Figura 6.9

Plantillas XSL para generar HTML y CSV a partir de documento de XML con datos de clientes

Algunas opciones de XSL controlan aspectos del documento generado. Por ejemplo, en `<xsl:output>` el atributo `method`, con valores `html` y `text` utilizados aquí (otro posible valor es `xml`). Para generar texto sin que se añadan separadores de relleno, se utiliza `<xsl:strip-space elements="*" />`. Para generar texto, se utilizan dos variables: `separador_campo` y `salto_linea`. Esta última se define como `
`, es decir, un *byte* con valor 10 (`a` en hexadecimal). Esto muestra que con XSL se pueden escribir directamente *bytes* y, por tanto, generar ficheros binarios. El resto está basado en el uso conjunto de `<xsl:template match="...">`, al que se le indica un XPath (se puede ver que se empieza con el nodo raíz `/`) y `<xsl:apply-templates select="...">`, que selecciona mediante XPath nodos sobre los que aplicar otras plantillas. Hay muchas más posibilidades que no se ilustran aquí, como por ejemplo la posibilidad de definir parámetros a los que se les pasa un valor en el momento de utilizar la plantilla. Esto se podría usar, por ejemplo, para especificar el separador de campos y de líneas al generar un documento en formato CSV.

Por último, un programa en Java que aplica las dos plantillas anteriores al documento de XML de ejemplo con datos de clientes para generar un fichero HTML y otro CSV. Utiliza la misma clase `Transformer` que se utilizó en un programa anterior, pero de una manera muy diferente. Esta es una clase muy versátil, pero en general relacionada con la transformación de documentos de XML. En el programa anterior se construía un `Transformer` sobre un documento DOM para serializarlo en un fichero. En este se construye un `Transformer` sobre una plantilla XSL para aplicarla a un documento de XML y generar un fichero.

```

// Plantillas XSL para generar HTML y CSV a partir de un documento de XML
package plantillasxsl;

import java.io.FileOutputStream;
import java.io.FileNotFoundException;
import javax.xml.transform.Source;
import javax.xml.transform.Result;
import javax.xml.transform.TransformerFactory;
import javax.xml.transform.Transformer;
import javax.xml.transform.stream.StreamSource;
import javax.xml.transform.stream.StreamResult;

```

```

import javax.xml.transform.TransformerException;

public class PlantillasXSL {
    public static void main(String[] args) {
        String fXML = "clientes.xml";
        String fXSLHTML = "clientes_HTML.xsl";
        String fXSLCSV = "clientes_CSV.xsl";
        String fHTML = "clientes.html";
        String fCSV = "clientes.csv";

        try {
            Source sXML = new StreamSource(fXML); // Documento fuente XML
            Source sXSLHTML = new StreamSource(fXSLHTML); // Plantilla para
            HTML
            Result osHTML = new StreamResult(new FileOutputStream(fHTML));
            Transformer tHTML = TransformerFactory.newInstance().
                newTransformer(sXSLHTML);
            tHTML.transform(sXML, osHTML);

            Source sXSLCSV = new StreamSource(fXSLCSV); // Plantilla para CSV
            Result osCSV = new StreamResult(new FileOutputStream(fCSV));

            Transformer tCSV = TransformerFactory.newInstance().
                newTransformer(sXSLCSV);
            tCSV.transform(sXML, osCSV);
        } catch (FileNotFoundException | TransformerException e) {
            System.err.println(e.getMessage());
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

Resumen

- XML es un lenguaje formal. XML es un estándar del W3C. Los documentos en lenguaje XML son documentos de texto con estructura jerárquica que se utilizan para guardar información de todo tipo.
- DOM es un modelo que define interfaces de programación para acceder a los contenidos de un documento de XML y para modificarlos. En el modelo DOM un documento de XML se representa como un árbol con distintos tipos de nodos.
- Un *parser* o analizador sintáctico para un lenguaje formal como XML permite verificar que un documento es conforme a la sintaxis del lenguaje y, si es el caso, proporciona acceso a sus contenidos.
- Un *parser* DOM permite obtener un árbol DOM a partir de un documento de XML.
- Un *parser* SAX es un tipo de *parser* para XML basado en eventos. No devuelve una estructura de datos como un *parser* DOM. En lugar de ello, genera diversos tipos de eventos conforme va analizando el documento, y estos eventos pueden gestionarse en métodos manejadores de eventos.

- La validación de un documento de XML consiste en verificar que es conforme a un conjunto de restricciones adicionales relativas a los elementos que contiene, sus atributos y los valores para todos ellos. Estas restricciones se pueden especificar en DTD y en esquemas de XML. Se puede validar un documento de XML durante el *parsing*, y en ese caso el *parser* lanzará excepciones ante cualquier error de validación.
- El *binding* es un mecanismo sencillo para traducir de documentos XML a objetos y viceversa, proporcionado por la API JAXB (Java Architecture for XML Binding). Para utilizarla es necesario un esquema de XML. A partir de este, un compilador de *binding* genera clases que se corresponden con los elementos del esquema de XML. En la terminología de JAXB, *marshalling* es serialización de objetos de Java en documentos de XML, y *unmarshalling* es deserialización de documentos de XML en objetos de Java.
- XPath es un lenguaje de consulta para documentos de XML. Se creó para XSL, y es fundamental para el lenguaje de consulta XQuery.
- XSL es un lenguaje que permite convertir documentos de XML a otros tipos, que pueden ser documentos XML de otros tipos, otros tipos de documentos de texto e incluso documentos binarios.



Ejercicios propuestos

Como en el capítulo anterior, para la realización de estos ejercicios puede ser necesario consultar la documentación de Java SE 8 (<http://docs.oracle.com/javase/8/docs/api/>).

1. Crea un programa en Java que, a partir de los contenidos del fichero XML como el del ejemplo, con los datos de varios clientes, muestre un nuevo documento de XML que sea el resultado de añadir los datos de un nuevo cliente al principio, antes de todos los clientes que puedan existir. Debe utilizarse un *parser* DOM y las interfaces de DOM para añadir los datos del nuevo cliente, incluyendo DNI, apellidos y CP. La salida debe ser a salida estándar (`System.out`) y en formato XML, y debe generarse utilizando la clase `Transformer`. El fichero se debe especificar mediante un argumento de línea de comandos. Si el fichero no es un fichero XML correcto, o si lo es pero no tiene un único elemento con nombres de clientes en el nivel más alto, se debe terminar la ejecución inmediatamente, y mostrar mensajes de error apropiados.
2. Cambia el programa anterior para que la salida se realice al mismo fichero, y reemplaze sus contenidos. La salida se debe generar también con la clase `Transformer`. Antes de nada hay que hacer una copia del fichero en un fichero con el mismo nombre pero incluyendo una marca de tiempo y al final ".bak". Se aconseja generar la salida en un fichero temporal, borrar el fichero original y renombrar el fichero temporal con el nombre del fichero original. En los programas de ejemplo del capítulo anterior dedicado a ficheros se utiliza esta técnica. Se recomienda consultarlos y copiar y adaptar su código de programa. Si, como es lógico, no funciona a la primera, se tiene una copia del fichero original que se puede renombrar para intentarlo de nuevo.

3. Crea un programa como el anterior, pero que no permita añadir el nuevo cliente si en el fichero ya existe un cliente con su DNI. En ese caso, debe mostrarse un mensaje de error y terminarse inmediatamente la ejecución del programa. Además, el cliente debe añadirse al final, en lugar de al principio.
4. Crea un programa en Java que haga todo lo anterior, pero basándose en una clase `clientesXML` para encapsular convenientemente todas las operaciones. El método `main()` debe crear una instancia de esa clase y utilizarla para todas las operaciones. Su constructor debe tomar como argumento un nombre de fichero y obtener el documento DOM en una variable de clase de tipo `Document`. El constructor no debe gestionar ninguna excepción, de manera que debe añadirse en su declaración una cláusula `throws` con todos los tipos de excepción que sea necesario incluir. La inserción de un nuevo cliente debe realizarse mediante un método público de clase `insertaCliente(String DNI, String apellidos, String CP)`, y debe insertarse al final, después de todos los clientes que puedan existir. Este método debe devolver un `Node` con el nuevo nodo creado, o `null` si por la razón que fuera no se crea el nodo. No debe reflejar los cambios en el fichero, solo en el `Document`. No debe gestionar las excepciones que se puedan producir al insertar los datos del nuevo cliente en el `Document`, de manera que hay que añadir la cláusula `throws` apropiada. Debe tener un método `grabar()` que permita reemplazar los contenidos del fichero con los contenidos del documento DOM. La generación del fichero (serialización del documento DOM) se debe realizar con la clase `Transformer`. Este método debe, antes que nada, hacer una copia del fichero en un fichero cuyo nombre esté basado en el nombre original del fichero y que incluya una marca de tiempo con la fecha y hora. El programa de prueba debe insertar un cliente con un DNI ya existente en el momento en que se intenta la inserción y dos clientes cuyos DNI no existan. Si el DNI ya existe, debe mostrarse un mensaje de error y seguir adelante. El método `main()` debe quedar más o menos así:

```
ClientesXML cXML = new ClientesXML(nombreFichero);
cXML.insertaCliente(dni1_nuevo, nombrel, CP1);
cXML.insertaCliente(dni2_repetido, nombre2);
cXML.insertaCliente(dni3_nuevo, nombre3, CP3);
cXML.grabar();
```

5. El formato `tmx` es un tipo de fichero XML que se utiliza para almacenar memorias de traducción. Una memoria de traducción es una colección de traducciones de un idioma a otro. Hay muchos ficheros de este tipo en el sitio <http://opus.nlpl.eu>. Utilizar un parser SAX para escribir las primeras quinientas entradas de la memoria de traducción disponible en <http://opus.nlpl.eu/download.php?f=EMEA/en-es.tmx.gz>. Este es un documento `tmx`, comprimido en formato `zip`, que hay que descargar y descomprimir. El principio de este fichero tiene el siguiente aspecto:

```
<?xml version="1.0" encoding="UTF-8" ?>
<tmx version="1.4">
<header creationdate="Sun Oct 23 00:51:26 2011">
```

```

srolang="en"
(...)
/>
<body>
  <tu>
    <tuv xml:lang="en"><seg>(texto en inglés)</seg></tuv>
    <tuv xml:lang="es"><seg>(texto en español)</seg></tuv>
  </tu>
  <tu>
    <tuv xml:lang="en"><seg>(texto en inglés)</seg></tuv>
    <tuv xml:lang="es"><seg>(texto en español)</seg></tuv>
  </tu>
  (...)

```

El programa debe mostrar primero la lengua original y después los textos de cada segmento, y resaltar el que esté en la lengua original (en este caso, en inglés). La salida del programa debe ser similar a la siguiente:

```

Lengua original: en
---
[1]
*en=>(Texto segmento 1 en inglés)
es=>(Texto segmento 1 en español)
---
[2]
*en=>(Texto segmento 2 en inglés)
es=>(Texto segmento 2 en español)
---
[500]
*en=>(Texto segmento 500 en inglés)
es=>(Texto segmento 500 en español)

```

6. Escribe una clase similar a la desarrollada en la actividad 4 anterior, pero que utilice JAXB para traducir los contenidos del fichero en objetos. No hay que utilizar DOM para nada, las modificaciones deben realizarse sobre objetos. Las operaciones de serialización y deserialización deben realizarse con los métodos que JAXB proporciona para *marshalling* y *unmarshalling*.
7. Haz una copia del programa desarrollado en la actividad 3 anterior y modifícala para que la comprobación de si existe algún cliente con el DNI proporcionado se haga mediante XPath, en lugar de recorriendo el árbol DOM. También se podría hacer esta mejora sobre una copia del programa desarrollado para la actividad 4.
8. Crea un fichero tmx a partir del utilizado para el ejercicio 5 que contenga varias decenas de traducciones. Crea una plantilla XSL que genere un documento de HTML que las muestre en una tabla con dos columnas. En la cabecera de las columnas aparecerán los idiomas, y en cada fila aparecerá un texto en el idioma original y su traducción al otro idioma.

ACTIVIDADES DE AUTOEVALUACIÓN

1. Los datos en un documento de XML están estructurados de manera:
 - ☐ a) Secuencial, al tratarse de ficheros de texto.
 - ☐ b) Jerárquica, dado que pueden existir elementos dentro de otros elementos, y la información que contienen está asociada a ellos, como un texto o como valores para sus atributos.
 - ☐ c) Estricta, y de acuerdo a su esquema de XML asociado.
 - ☐ d) Ninguna de las respuestas anteriores es correcta.
2. Un documento de XML es válido si:
 - ☐ a) Está bien formado, es decir, es conforme a la sintaxis del lenguaje XML.
 - ☐ b) Está bien indentado, de manera que se pueda leer con claridad.
 - ☐ c) Es correcto sintácticamente y además cumple una serie de restricciones adicionales, con respecto a su estructura y contenidos, que se pueden especificar en un esquema de XML.
 - ☐ d) Todas las etiquetas de apertura tienen su correspondiente etiqueta de cierre.
3. El modelo DOM:
 - ☐ a) Es una especificación detallada de las estructuras de datos que hay que utilizar para representar en memoria un documento de XML utilizando un lenguaje orientado a objetos como Java.
 - ☐ b) Es la especificación formal de un tipo de *parser* para documentos de XML que permite un rápido acceso a sus contenidos y, una vez terminado el proceso de *parsing*, permite su modificación.
 - ☐ c) Especifica un conjunto de interfaces de programación para el acceso a los contenidos de documentos de XML y para su modificación.
 - ☐ d) Ninguna de las respuestas anteriores es correcta.
4. Un *parser* DOM:
 - ☐ a) En general es el más apropiado para documentos muy grandes porque permite un acceso muy rápido a sus contenidos.
 - ☐ b) Está basado en eventos.
 - ☐ c) No es apropiado para documentos muy grandes porque almacena en memoria todos los contenidos del documento.
 - ☐ d) Permite representar aspectos de un documento de XML que no contempla un *parser* SAX.
5. Un *parser* SAX:
 - ☐ a) Almacena por defecto los contenidos del documento en memoria. Una vez terminado el *parsing*, permite modificar los contenidos del documento y volverlo a grabar en un fichero con los cambios realizados.
 - ☐ b) No se puede utilizar para implementar un *parser* DOM porque el modelo de eventos de SAX es incompatible con el modelo de datos de DOM.
 - ☐ c) Está basado en eventos. Un programa que utilice un *parser* SAX debe proporcionar métodos manejadores para diversos tipos de eventos que pueden suceder durante el *parsing*.
 - ☐ d) Es un tipo de *parser* disponible solo para el lenguaje Java.

6. Un esquema de XML:
- ☐ a) No permite especificar restricciones que sí se pueden especificar con un DTD.
 - ☐ b) Es un documento de XML.
 - ☐ c) Es necesario para un documento para poder aplicarle una plantilla de XSL.
 - ☐ d) Utiliza expresiones de XPath para indicar qué elementos pueden ir dentro de otros.
7. En un documento de XML:
- ☐ a) Es obligatorio especificar un valor para todos los atributos, entre comillas, y no es admisible el valor "".
 - ☐ b) En un mismo elemento no puede aparecer más de una vez el mismo nombre de atributo.
 - ☐ c) En un mismo elemento puede aparecer más de una vez el mismo nombre de atributo, siempre que no se le asigne más de una vez el mismo valor. Esto es útil para asignar una lista de valores a un atributo.
 - ☐ d) Se puede especificar un valor nulo para un atributo de la forma `atrib=null`.
8. Un JAXB es:
- ☐ a) Un mecanismo de *binding* que se puede aplicar con cualquier documento de XML, pero solo si es válido según el esquema utilizado para el *binding*. Además, impide cualquier modificación sobre los objetos que contravengan las restricciones impuestas por el esquema.
 - ☐ b) Un mecanismo que permite traducir de objetos a XML, pero no a la inversa.
 - ☐ c) Un mecanismo de *binding* que requiere un esquema de XML. Permite *marshalling* (serialización) a documentos de XML y *unmarshalling* (deserialización) desde documentos de XML.
 - ☐ d) Un mecanismo de *binding* que puede utilizarse con Java y otros lenguajes, siempre que sean orientados a objetos.
9. Una expresión XPath aplicada a un documento devuelve:
- ☐ a) Un conjunto de textos.
 - ☐ b) Un conjunto de nodos de un árbol DOM.
 - ☐ c) Un conjunto de nodos que pueden ser atributos también.
 - ☐ d) Ninguna de las opciones anteriores es correcta.
10. El lenguaje de transformación XSL:
- ☐ a) Solo permite generar documentos de XML.
 - ☐ b) Solo permite transformar documentos de XML.
 - ☐ c) Solo permite transformar documentos de XML válidos según algún esquema.
 - ☐ d) Ninguna de las respuestas anteriores es correcta.

SOLUCIONES:

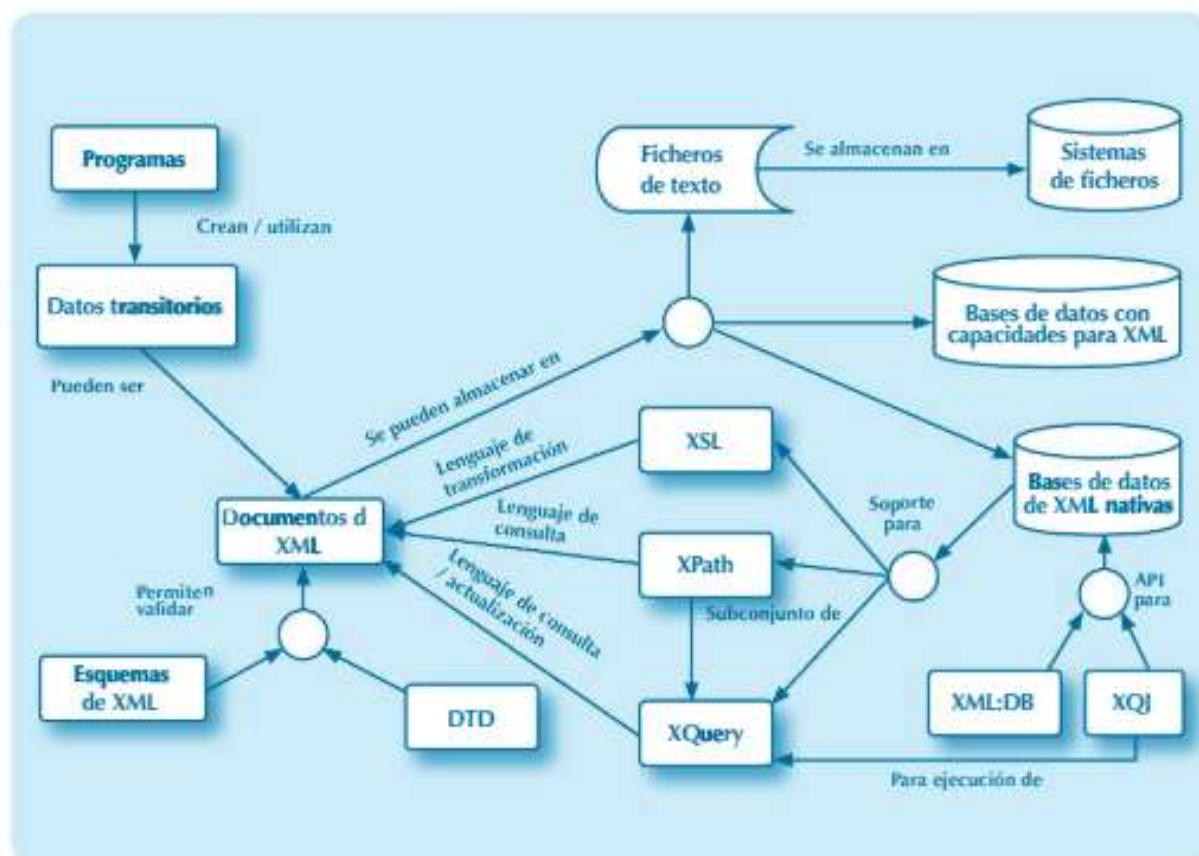
- | | | |
|---|---|--|
| 1. ☐ a ☒ b ☐ c ☐ d | 5. ☐ a ☐ b ☐ c ☐ d | |
| 2. ☐ a ☐ b ☒ c ☐ d | 6. ☐ a ☒ b ☐ c ☐ d | 9. ☐ a ☐ b ☒ c ☐ d |
| 3. ☐ a ☐ b ☒ c ☐ d | 7. ☐ a ☒ b ☐ c ☐ d | 10. ☐ a ☒ b ☐ c ☐ d |
| 4. ☐ a ☐ b ☒ c ☐ d | 8. ☐ a ☐ b ☒ c ☐ d | |

Bases de datos de XML

Objetivos

- ✓ Conocer los distintos tipos de aplicaciones basadas en XML: *data-centric* y *document-centric*.
- ✓ Comprender el propósito y los fundamentos de las bases de datos de XML nativas.
- ✓ Entender las diversas formas en que distintas bases de datos de XML nativas permiten organizar los documentos en colecciones.
- ✓ Aprender los lenguajes estándares XPath y XQuery para consulta y actualización de documentos.
- ✓ Utilizar la base de datos de XML nativa eXist para almacenar, consultar y modificar documentos de XML.
- ✓ Estudiar las API estándares para interacción con bases de datos de XML: XML:DB y XQJ.
- ✓ Trabajar con XML:DB para gestionar colecciones y documentos.
- ✓ Realizar operaciones de consulta y modificación de documentos con XQJ.
- ✓ Utilizar transacciones con XQJ.

Mapa conceptual



Glosario

Base de datos nativa de XML. Base de datos para documentos de XML que, para su almacenamiento, utiliza estructuras diseñadas y optimizadas para este tipo de documentos.

XML:DB. API para bases de datos de XML que permite realizar todo tipo de operaciones.

XML. Lenguaje que permite representar cualquier tipo de información en una estructura jerárquica dentro de un documento de texto.

XPath. Lenguaje de consulta para documentos de XML que permite recuperar un conjunto de nodos de su estructura jerárquica.

XQJ (XQuery for Java). API para bases de datos de XML que permite realizar operaciones de consulta utilizando XQuery, y operaciones de modificación utilizando extensiones de XQuery.

XQuery. Lenguaje de consulta para documentos de XML, más potente que XPath, y del que XPath es un subconjunto. Se han propuesto varias extensiones de XPath que permiten modificar documentos de XML. Entre ellas está XQF, que es un estándar de W3C.

7.1. XML como soporte para almacenamiento e intercambio de datos

XML es un formato muy sencillo y flexible que permite representar cualquier tipo de información en documentos de texto con una estructura jerárquica. Su uso se ha extendido cada vez más desde su creación como estándar de W3C en 1998, y en torno a él ha ido surgiendo un abundante cuerpo de estándares. Desde su introducción, el desarrollo de nuevos servicios y aplicaciones sobre internet se ha basado cada vez más en XML. XML se utiliza a menudo como soporte para almacenamiento de datos. Sus aplicaciones son cada vez más diversas, pero se pueden distinguir a grandes rasgos dos áreas de aplicación:

1. *XML centrado en documentos (document-centric XML)*. Corresponde al uso inicial de los lenguajes de marcado antecesores de XML, como HTML. Se utilizan documentos de XML que no tienen una estructura fija y regular, producidos mediante procesos manuales por personas para su utilización en procesos manuales realizados por personas. Suelen contener textos en lenguaje natural.
2. *XML centrado en datos (data-centric XML)*. Se utilizan documentos XML con una estructura bien definida y regular. Contienen datos. Son normalmente producidos por máquinas y consumidos por máquinas mediante procesos automáticos que utilizan lenguajes estándares como XML Schema, XPath, XQuery, XSL, etc.

Estos son dos casos extremos, y puede haber planteamientos intermedios. Por ejemplo: dentro de un documento de XML centrado en documento puede existir una sección con una estructura bien definida y regular, o bien dentro de un documento de XML centrado en datos puede haber una sección donde se admita un formato más flexible.

7.2. Alternativas para el almacenamiento de documentos de XML

Existen varias posibilidades para la persistencia de documentos de XML. Cada una puede ser más o menos apropiada dependiendo del tipo de documentos que almacenar y su uso previsto.

1. *Sistemas de ficheros*. Cada documento de XML se almacena como un fichero en una jerarquía de directorios. Para el manejo de los documentos se dispone solo de las funcionalidades que para el manejo de ficheros proporciona el sistema operativo, entre las que no suele estar el control de accesos concurrentes, ni tampoco el soporte para transacciones. Cualquier tipo de consulta o de modificación que se realice sobre el contenido de los documentos debe hacerse manualmente o programarse a medida.
2. *Bases de datos con capacidades para XML (XML-enabled)*. Pueden ser de distintos tipos, como por ejemplo relacionales u orientadas a objetos. Se necesitan mecanismos de traducción entre XML y el esquema de almacenamiento de la base de datos, que no estará en general optimizado para documentos de XML, lo que puede conllevar un mayor consumo de espacio de almacenamiento y un menor rendimiento en las operaciones realizadas sobre los documentos.
3. *Bases de datos nativas de XML*. El almacenamiento se realiza en estructuras diseñadas y optimizadas para documentos de XML. Proporcionan, además, soporte para lenguajes estándares que permiten realizar diversos tipos de operaciones sobre documentos de XML: consulta (XPath, XQuery), actualización (diversas extensiones de XQuery), y

transformación (XSL). Proporcionan implementaciones de API estándares para bases de datos de XML, tales como XML:DB y XQJ.

7.3. Almacenamiento de XML en SGBD relacionales

Es de especial interés plantear las distintas posibilidades para persistencia de documentos XML en bases de datos relacionales, por varios motivos:

1. Son, con diferencia, las más empleadas en la actualidad para todo tipo de aplicaciones. En principio es recomendable utilizarlas a no ser que el uso de una base de datos de XML nativa suponga una clara ventaja.
2. Pueden perfectamente almacenar documentos de XML, por grandes que sean, en columnas de tipo CLOB (*character large object*) o BLOB (*binary large object*). Si se trata de documentos muy pequeños (pocos *kilobytes*), podrían incluso almacenarse en columnas de tipo VARCHAR. Por tanto, son perfectamente válidas si lo único que se necesita es almacenar y recuperar documentos de XML enteros.
3. Es relativamente sencillo almacenar los datos que contienen los documentos de XML centrados en datos (*data-centric*) en esquemas relacionales, y generar los documentos a partir de ellos mediante procesos automáticos. Además, como se explica a continuación, las últimas versiones del estándar SQL incluyen características para XML que pueden facilitar mucho esta tarea.
4. El estándar SQL incluye, desde SQL:2003, SQL/XML en su parte 14. SQL/XML incluye, entre otras cosas:
 - a) El tipo de datos XML para almacenar documentos de XML.
 - b) Correspondencias entre tipos de datos de SQL y de XML para facilitar el almacenamiento y manipulación de XML en bases de datos con SQL.
 - c) Posibilidad de utilizar XQuery dentro de consultas en SQL. Como resultado de una consulta en XQuery, se puede obtener una relación, como con una consulta en SQL. Esto permite realizar consultas de SQL que combinen y relacionen información existente en esquemas relacionales y documentos de XML.
 - d) A la inversa, se pueden generar documentos de XML con el resultado de consultas en SQL. Esto permite realizar consultas en XQuery que combinen y relacionen información existente en documentos de XML y en esquemas relacionales.
5. Algunas bases de datos relacionales se pueden considerar *XML-enabled* porque implementan SQL/XML, lo que incluye soporte para XQuery, e incluso soporte para otros estándares de XML, como por ejemplo XSL.
6. Las bases de datos relacionales proporcionan, en general, un magnífico y completo soporte para transacciones, incluyendo en muchos casos transacciones distribuidas. No se puede decir tanto de las bases de datos de XML nativas, si bien es cierto que algunas bases de datos de XML nativas tienen soporte para transacciones.

La separación entre bases de datos relacionales y de XML nativas no es nítida. El estándar SQL incluye desde SQL:2003 muchas características para XML, y algunas bases de datos relacionales añaden soporte para otras tecnologías de XML como XSL, y la implementación de API estándar para bases de datos de XML. Si la funcionalidad y el rendimiento son satisfacto-

rios, lo de menos es que el almacenamiento se realice, en última instancia, sobre un esquema relacional. Todas las versiones de la base de datos de Oracle desde la 12c incluyen XML DB.

XML DB incluye un buen soporte para SQL/XML y diversas tecnologías de XML.

En conclusión, antes de optar por el uso de una base de datos de XML nativa conviene considerar el uso de una base de datos relacional.

Recurso web

www

El siguiente enlace proporciona información acerca de Oracle XML DB, incluyendo código de ejemplo y documentación:

<https://www.oracle.com/database/technologies/appdev/xmlldb.html>

7.4. Características de las bases de datos de XML nativas

Antes que nada, hay que aclarar que, en lo sucesivo, su utilizará el término *base de datos* (se sobreentiende de XML) con dos significados: el de almacenamiento de documentos de XML y el de SGBD (sistema gestor de bases de datos). El significado exacto será claro por el contexto. *Base de datos de XML nativa* significará siempre SGBD de XML nativo.

Las bases de datos de XML nativas deben permitir el almacenamiento directo de documentos de XML en estructuras diseñadas específicamente para XML, sean del tipo que sean: basados en contenido o en documentos, o cualquier otra posibilidad. Esto no excluye que puedan almacenar documentos que no sean de XML, si bien el conjunto de operaciones que se pueda realizar sobre ellos será más limitado. Por lo demás, suelen tener las siguientes características:

1. *Colecciones.* Dentro de una base de datos de XML existente en un SGBD de XML, se pueden organizar los documentos en colecciones. La manera de organizar los documentos en colecciones varía mucho de un SGBD de XML nativo a otro.
2. *Validación.* Se pueden usar esquemas o DTD para validar los documentos almacenados.
3. *Mecanismos estándares para consulta y transformación de documentos de XML.* A saber: XPath, XQuery y XSL (todos estándares de W3C). Pueden incluir la extensión XQuery Full Text, también estándar de W3C, para búsqueda en textos de lenguaje natural, tan frecuentes en los documentos de XML.
4. *Mecanismos estándares para modificación de documentos.* Debería ser posible modificar los contenidos de un documento sin reemplazarlo globalmente por otro. Han surgido diversos lenguajes para permitir esto, en general extensiones de XQuery.
5. *Soporte para API estándares.* Las dos API más importantes para la interacción con bases de datos de XML son XML:DB y XQJ. Pero cada base de datos de XML tendrá, en general, sus propias API, y las implementaciones de las API estándares se basarán en ellas.
6. *Indexación.* El uso de índices es fundamental para agilizar las consultas sobre los contenidos de los documentos de XML. Pero su indexación es mucho más problemática que la de las tablas de una base de datos relacional. Esto es debido a la mayor complejidad de los documentos de XML, con una estructura jerárquica en vez de tabular, ya que, en

una misma colección, puede haber documentos de distintos tipos. Se necesitan mecanismos de indexación muy flexibles y adaptativos. Las bases de datos de XML nativas suelen utilizar índices estructurales de todos los documentos, en los que se incluyen los valores de todos los elementos y todos los atributos. En ellos cada documento viene identificado por un identificador único asignado automáticamente por el sistema. Debido al mayor coste computacional de actualizar los índices, algunas bases de datos no lo hacen automáticamente tras cada cambio en los datos, lo que hay que tener muy en cuenta para el desarrollo de aplicaciones y para su administración. Las distintas bases de datos permiten crear distintos tipos de índices. La indexación de bases de datos de XML nativas es un aspecto poco o nada estandarizado.

7. *Transacciones.* Las diversas bases de datos de XML nativas proporcionan un cierto grado de soporte para transacciones, pero con frecuencia no ofrecen un completo soporte para transacciones ACID, como es la norma para las bases de datos relacionales. La API XQJ incluye métodos para gestión de transacciones, pero el soporte para transacciones es opcional para una implementación de XQJ. En cualquier caso, para que se puedan utilizar desde la API, el SGBD debe soportarlas.

7.5. Gestores comerciales y libres

Existen multitud de bases de datos de XML, tanto nativas como *XML-enabled*, y tanto comerciales como libres. Hay que tener en cuenta que, aunque XML se ha venido utilizando cada vez más, las bases de datos de XML nativas siempre han tenido, y siguen teniendo, un uso muy reducido. Se han desarrollado muchas desde finales de los años noventa, comerciales y libres, pero en pocas se invierte actualmente esfuerzo de soporte y desarrollo. En cambio, las bases de datos relacionales han ido cada vez más adoptando características para XML, incluso proporcionando implementaciones bastante completas de SQL/XML, parte del estándar SQL.

www

Recurso web

En la siguiente dirección se puede encontrar una descripción muy completa de las características más importantes de muchos productos de bases de datos de XML. Son de especial interés los enlaces titulados "Native XML Databases" y "XML-Enabled Databases":

<http://www.rpbouret.com/xml/XMLDatabaseProds.htm>

A continuación se reseñan brevemente varios SGBD de XML nativos comerciales y libres.

1. *Tamino.* Es un SGBD comercial de la empresa Software AG. Surgió durante el *boom* de XML a finales de los noventa. Es un producto muy caro y con muy buen rendimiento debido a su almacenamiento de datos diseñado específicamente para XML y a la indexación que realiza de los documentos. En el momento de la redacción de este libro, en la página web de Software AG no existe documentación ni información de producto para Tamino.

2. *Marklogic*. Este SGBD comercial comenzó como un SGBD de XML nativo a principios de los 2000 para evolucionar más recientemente hacia una base de datos multimodelo, es decir, que integra varios mecanismos de almacenamiento (en este caso, XML, JSON y tripletas semánticas de RDF), y ofrece una vista lógica unificada de todos los datos. Organiza los documentos en directorios y colecciones. Los directorios forman una jerarquía. Las colecciones agrupan documentos que tienen algo en común, como por ejemplo tema o autor. Un documento está en un directorio y puede estar en muchas colecciones. Existe una versión gratuita, descargable previo registro, para uso personal y no comercial. Se puede encontrar más información de Marklogic en su página web: <https://www.marklogic.com>
3. *eXist*. Es un SGBD de *software* libre. En una instalación de eXist solo existe una base de datos que contiene una jerarquía de colecciones, y dentro de cada una puede haber documentos de XML y de otros tipos. Asigna a cada documento un identificador único y crea y mantiene automáticamente índices estructurales para todos ellos. Se puede encontrar más información de eXist en su página web: <https://exist-db.org>.
4. *BaseX*. Es un SGBD de *software* libre. En una base de datos de BaseX no se crean las colecciones explícitamente. En lugar de ello, se crean y borran implícitamente, dependiendo de la existencia de documentos en *paths* específicos. Los documentos pueden ser de tipo XML o de tipo *raw* (que en este caso significa todo lo que no sea XML). Se puede encontrar más información de BaseX en su página web: <http://basex.org>
5. *Sedna*. Es un SGBD de *software* libre. Está escrito en C/C++, mientras que prácticamente todos los SGBD de XML nativos están escritos en Java. Es muy eficiente y está muy optimizado en todos los aspectos, como por ejemplo su esquema de almacenamiento para XML y su API propia basada en un protocolo propio binario, y no, como es habitual, en protocolos tales como SOAP, XML-RPC o similares. Fue un magnífico SGBD y un desarrollo muy prometedor en su momento, pero, de acuerdo con la información de su página web y de sourceforge.net, no parece haber actividad ni nuevos desarrollos desde 2012. Se puede encontrar más información de Sedna en su página web: <http://www.sedna.org>.

7.6. Instalación y configuración del SGBD de XML nativo eXist

Se ha elegido eXist como SGBD de XML nativo para trabajar por su facilidad de uso y funcionalidad. Como casi todas las bases de datos de XML nativas, está desarrollada en Java, por lo que funciona para Windows, Linux y cualquier sistema operativo para el que esté disponible una máquina virtual de Java.

Para instalarlo basta ejecutar el programa de instalación y aceptar todas las opciones por defecto. Se debería indicar la contraseña para el usuario administrador **admin**. Si no, se puede hacer más adelante. Con ello se crean opciones en el menú de inicio para lanzar y parar eXist, y para instalarlo o desinstalarlo como servicio del sistema.

Es recomendable instalarlo como servicio del sistema. Si no se hace, arrancará automáticamente, pero, como recuerda un mensaje que puede aparecer en ese momento, se pueden perder cambios realizados en la base de datos si se apaga el ordenador sin cerrar antes eXist.



Figura 7.1
Instalación de eXist-db como servicio de Windows

El *dashboard* (panel de control) de eXist está disponible en un servidor web que escucha por defecto por el puerto 8080.



TOMA NOTA

Hay diversos programas que suelen utilizar el puerto 8080, por ejemplo Tomcat y el servidor de base de datos Oracle. Si ya está en uso, se mostrará un error y se dará la opción de visualizar los *logs*, en los que aparecerá un mensaje similar al siguiente: **ERROR (JettyStart.java [run]:369) - ERROR: Could not bind to port because Address already in use: bind**. Se puede cambiar el puerto en el fichero `tools\jetty\etc\jetty-http.xml` en el directorio de instalación de eXist, en la línea donde dice `<SystemProperty name="jetty.port" default="8080"/>`. En esta instalación se ha cambiado a 8090, como se puede ver en algunos ejemplos siguientes.

Una vez iniciado eXist, se puede acceder al *dashboard* o panel de control en la dirección `http://localhost:puerto`. En él se pueden encontrar, entre otras cosas:

1. *Usermanager*. Para crear nuevos usuarios y cambiar sus propiedades. Si no se asignó contraseña para el usuario `admin` durante la instalación, se puede hacer aquí.
2. *Collections*. Para gestionar, crear y borrar colecciones. Una instalación de eXist gestiona una sola base de datos en la que pueden definirse colecciones dentro de otras colecciones, formando una jerarquía. En cada una puede haber documentos de XML y de otros tipos.
3. *eXide*. Magnífico IDE (entorno de desarrollo integrado) para XQuery. Permite navegar por los contenidos de la base de datos y visualizar los resultados de consultas con XQuery, y también ejecutar sentencias de XQuery que modifiquen los documentos. Permite modificar los contenidos de los documentos de XML mediante un editor de texto.
4. *Java Admin Client*. También se puede lanzar desde la bandeja del sistema. Además de consultas, permite realizar diversas tareas de gestión de los contenidos de la base de datos y administrativas, como por ejemplo:



Figura 7.2

Dashboard (panel de control) de eXist

- a) Importar ficheros y directorios seleccionados desde un sistema de ficheros.
- b) Gestionar colecciones.

- c) Crear, borrar, mover y renombrar documentos.
 - d) Reindexar colecciones.
 - e) Gestionar usuarios, incluyendo sus permisos sobre colecciones y documentos.
 - f) Realizar y restaurar copias de seguridad, en modo de mantenimiento.
5. *Package Manager*. Permite instalar muchos paquetes y utilidades adicionales. Se recomienda instalar “eXist-db Demo Apps”. Con ello se instalará alguna base de datos de ejemplo que se utilizará después.
 6. *Shutdown*. Para parar el SGBD eXist. Si no se ha arrancado mediante un servicio, hay que pararlo antes de apagar el ordenador. En caso contrario podrían no guardarse todos los cambios realizados, como se ve en la figura 7.1.

7.7. API para gestión de bases de datos nativas de XML

Cada base de datos nativa de XML suele incluir sus propias API. Aparte de ello, para Java se han desarrollado las API estándares XML:DB y XQJ, cuyo planteamiento es similar al de JDBC para bases de datos relacionales.

- *XML:DB*. Fue la primera API estándar para bases de datos de XML. Su última revisión es de 2001.
- *XQJ*. Son las siglas de XQuery for Java. Es una API más reciente: su última revisión es de 2009. Ha sido diseñada como un proyecto JCP (*Java Community Process*), pero no está incluida en la biblioteca estándar de clases de Java.

Como su propio nombre indica, XQJ es una API para XQuery, es decir, para operaciones de consulta en documentos de XML existentes, y de modificación con diversas extensiones de XQuery. XML:DB permite utilizar XQuery y además hacer operaciones no posibles con XQuery, en particular las realizadas sobre colecciones y sobre documentos como un todo (creación y borrado), y además diversas tareas administrativas y de gestión. En el resto de este capítulo se aprenderá a realizar operaciones de creación y borrado de colecciones y documentos con XML:DB, y de consulta y modificación de documentos con XQJ.

TOMA NOTA



Es posible que las implementaciones de XML:DB o XQJ para algunas bases de datos no proporcionen determinadas funcionalidades que sí están disponibles en las API propias, o que estas últimas proporcionen mejor rendimiento para algunas operaciones. En ese caso, el uso de las API propias puede ser una alternativa que considerar, dependiendo de la aplicación.

7.8. La API XML:DB

No existe ninguna nueva versión oficial de esta API desde 2001. A pesar de ello, sigue siendo ampliamente utilizada internamente por muchos SGBD de XML nativos, como es el caso de eXist. Esto, unido al hecho de que permite gestionar diversos aspectos como las colecciones, la

creación y borrado de documentos, los usuarios y sus permisos, y otras cosas que varían mucho de una base de datos a otra, hace que pueda haber diferencias significativas entre implementaciones para distintas bases de datos.

www

Recurso web

Se puede acceder a los Javadocs de XML:DB en la siguiente dirección: <http://xmldb-org.sourceforge.net/xapi/api/index.html>.

Para ilustrar el funcionamiento de esta API, se utilizará para realizar algunos tipos de operaciones sobre la base de datos que no son posibles con XQuery. En particular:

- Operaciones sobre colecciones.
- Operaciones de creación y borrado de documentos de XML dentro de colecciones.

Para utilizar el *driver* de XML:DB para eXist hay que añadir al proyecto algunos ficheros jar disponibles en la instalación de eXist. Siempre hay que añadir `exist.jar` (presente en el directorio base de la instalación) y un fichero jar cuyo nombre comienza por `xmldb-db-api`, y otros presentes en el directorio `lib/core`. Según las funcionalidades que utilice el programa, podrá ser necesario añadir más ficheros jar para que se puedan cargar en tiempo de ejecución todas las clases necesarias. Si falta alguna, se producirá una excepción `ClassNotFoundException`. Entonces habrá que localizar el fichero jar que proporciona dicha clase y añadirlo al proyecto. En caso de dificultades, puede ser de ayuda algún buscador de ficheros jar que permita obtener el nombre del fichero jar a partir del nombre de la clase, como www.findjar.com

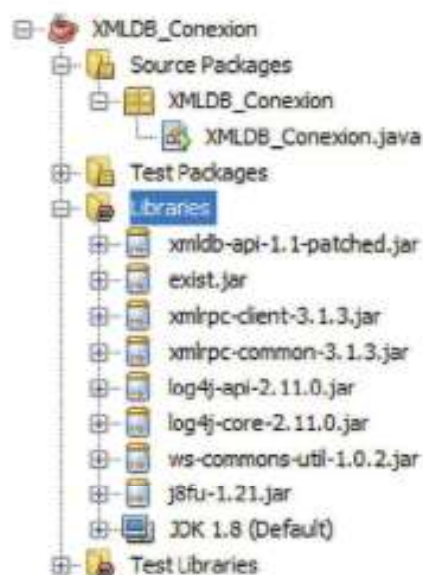


Figura 7.3
Proyecto que usa la API XML:DB

7.8.1. Establecimiento de conexiones y acceso a servicios con XML:DB

Para establecer una conexión con un SGBD con XML:DB, es necesario crear una instancia de una clase que implemente la interfaz `Database`, y proporcionar una cadena de conexión para la base de datos. El formato de la cadena de conexión varía según el SGBD. En la siguiente tabla se indica su formato para algunos SGBD de XML nativos. En ella se puede indicar el nombre de la base de datos y la colección, según el SGBD.

CUADRO 7.1

Propiedades para conexión para *drivers* de XML:DB para distintas bases de datos nativas de XML

SGBD	Clase que Implementa Database	Cadena de conexión
eXist	org.exist.xmldb.DatabaseImpl	xmlldb:exist://host:puerto/exist/ xmlrpc/db/colección
BaseX	org.basex.api.xmldb.BXDatabase	xmlldb:basex://localhost:puerto/ basedatos
Sedna	net.cfooster.sedna.DatabaseImpl	xmlldb:sedna://host:puerto/ basedatos/colección

Con eXist no se intenta abrir la conexión en el mismo momento en que se solicita, sino en el momento en que se intenta alguna operación que requiere acceso efectivo a la base de datos. Cualquier error en los datos de conexión pasará inadvertido hasta entonces. Además, si la conexión es a `localhost`, se ignora el puerto indicado y se toma de la configuración de eXist.

Una colección proporciona una serie de servicios, y cada uno de ellos puede proporcionar una serie de operaciones. El siguiente programa de ejemplo abre una conexión con una base de datos de eXist, y accede a una colección determinada dentro de ella, que siempre existe en una instalación típica, y muestra las colecciones y los documentos que hay en ella. También muestra una lista con los servicios proporcionados por la colección, y la versión de cada uno. Otros SGBD pueden proporcionar un conjunto distinto de servicios. Por ejemplo, la versión 3.5 de Sedna proporciona, además de los que ofrece eXist, los servicios `TransactionService`, `SednaUpdateService`, `XQueryService`, `IndexManagementService` y `ModuleManagementService`, cuyos nombres dan idea de su utilidad. Algunos de ellos están documentados en la especificación de XML:DB, y algunos de ellos no, al ser propios del SGBD Sedna.

```
// Muestra colecciones, documentos servicios disponibles en una colección
package XMLDB_Conexion;
import org.xmldb.api.base.Collection;
import org.xmldb.api.base.Database;
import org.xmldb.api.base.Service;
import org.xmldb.api.base.XMLDBException;
import org.xmldb.api.DatabaseManager;
public class XMLDB_Conexion {
    private static Collection obtenColeccion(String nomCol) throws Exception {
        Database dbDriver;
        Collection col;
        dbDriver = (Database)
            Class.forName("org.exist.xmldb.DatabaseImpl").newInstance();
        DatabaseManager.registerDatabase(dbDriver);
        col = DatabaseManager.getCollection(
            "xmlldb:exist://localhost:8090/exist/xmlrpc/db" + nomCol,
            "{usuario}", "{contraseña}");
        return col;
    }
    public static void main(String[] args) {
        Collection col = null;
    }
}
```

```

try {
    col = obtenColeccion("/apps/shared-resources");
    System.out.println("Colección actual: " + col.getName());
    int numHijas = col.getChildCollectionCount();
    System.out.println(numHijas + " colecciones hijas.");
    if (numHijas > 0) {
        String nomHijas[] = col.listChildCollections();
        for (int i = 0; i < numHijas; i++) {
            System.out.println("\t" + nomHijas[i]);
        }
    }
    int numDocs = col.getResourceCount();
    System.out.println(numDocs + " documentos.");
    if (numDocs > 0) {
        String nomDocs[] = col.listResources();
        for (int i = 0; i < numDocs; i++) {
            System.out.println("\t" + nomDocs[i]);
        }
    }
    Service servicios[] = col.getServices();
    System.out.println("Servicios proporcionados por colección " + col.
        getName() + ":");
    for (int i = 0; i < servicios.length; i++) {
        System.out.println("\t" + servicios[i].getName() + " - Versión:
            " + servicios[i].getVersion());
    }
} catch (Exception e) {
    e.printStackTrace();
} finally {
    try {
        if (col != null) {
            col.close();
        }
    } catch (XMLDBException e) {
        e.printStackTrace();
    }
}
}
}

```

7.8.2. Creación y borrado de colecciones con XML:DB

El siguiente programa crea dos colecciones en el nivel más alto de la jerarquía, y acto seguido borra una de ellas. Primero, obtiene un `CollectionManagementService` a partir de la colección raíz, y después, utiliza los métodos `createCollection` y `removeCollection` de este servicio para crear y eliminar colecciones dentro de ella. El método `obtenColeccion` en este programa y en los siguientes es igual que en el anterior y, por tanto, su código se omite.

```

// Creación y borrado de colecciones
package XMLDB_CreacionColecciones;
import org.xmldb.api.base.Collection;
import org.xmldb.api.base.Database;

```

```

import org.xmldb.api.base.XMLDBException;
import org.xmldb.api.DatabaseManager;
import org.xmldb.api.modules.CollectionManagementService;

public class XMLDB_CreacionColecciones {

    private static Collection obtenColeccion(String nomCol) throws Exception {
        (...)
    }

    public static void main(String[] args) {
        Collection col = null;
        try {
            col = obtenColeccion("");
            System.out.println("Colección actual: " + col.getName());
            System.out.println(col.getChildCollectionCount()+" colecciones
                hijas antes.");
            CollectionManagementService cmServ =
                (CollectionManagementService) col.getService(
                    "CollectionManagementService", "1.0"
                );
            cmServ.createCollection("pruebal");
            cmServ.createCollection("pruebas");
            cmServ.removeCollection("pruebal");
            System.out.println(col.getChildCollectionCount()+" colecciones
                hijas después.");
        } catch (Exception e) {
            e.printStackTrace();
        } finally {
            try {
                if (col != null) {
                    col.close();
                }
            } catch (XMLDBException xe) {
                xe.printStackTrace();
            }
        }
    }
}

```

7.8.3. Creación y borrado de documentos con XML:DB

El siguiente programa crea tres documentos en una colección creada por un programa anterior, y después borra uno de ellos. Los documentos se crean con `createResource` y se almacenan con `storeResource`. Para asignar y recuperar el contenido de un documento, la interfaz `XMLResource` tiene métodos `setContent` y `getContent`, y además `setContentAsDOM`, `getContentAsDOM`, `setContentAsSAX` y `getContentAsSAX`. eXist permite almacenar documentos que no son de XML (a los que denomina binarios). Para ello se utilizaría `BinaryResource` en lugar de `XMLResource`.

```

// Creación y borrado de documentos
package XMLDB_CreacionBorradoDocumentos;
import org.xmldb.api.base.*;
import org.xmldb.api.DatabaseManager;

```

```

import org.xmldb.api.modules.*;
import org.xml.sax.XMLReader;
import org.xml.sax.helpers.XMLReaderFactory;
import org.xml.sax.InputSource;
import org.xml.sax.ContentHandler;
import javax.xml.stream.XMLOutputFactory;
import javax.xml.stream.XMLStreamWriter;
import java.io.StringReader;
import java.io.StringWriter;
public class XMLDB_CreacionBorradoDocumentos {
    private static Collection obtenColeccion(String nomCol) throws Exception {
        (...)
    }
    public static void main(String[] args) {
        Collection col = null;
        try {
            col = obtenColeccion("/pruebas");
            XMLResource recurso;
            // Crear documento a partir de String
            recurso = (XMLResource) col.createResource("Clientes.xml",
                XMLResource.RESOURCE_TYPE);
            recurso.setContent("<clientes>\n"
                + "<cliente DNI=\"78901234X\">"
                + "<apellidos>NADALES</apellidos><CP>44126</CP></cliente>\n"
                + "<cliente DNI=\"89012345E\">"
                + "<apellidos>ROJAS</apellidos>"
                + "<validez estado=\"borrado\" timestamp=\"1528286082\"/>"
                + "</cliente>\n"
                + "<cliente DNI=\"56789012B\">\n"
                + "<apellidos>SAMPER</apellidos><CP>29730</CP></cliente>"
                + "</clientes>");
            col.storeResource(recurso);
            recurso = (XMLResource) col.createResource("Empresa_.xml",
                XMLResource.RESOURCE_TYPE);
            recurso.setContent("<empresa CIF=\"A34246801\">MegaExport"
                + "<sedes><sede>LEÓN</sede><sede>CÁCERES</sede></sedes>"
                + "</empresa>");
            col.storeResource(recurso);
            // Crear documento a partir de objeto SAX
            StringWriter out = new StringWriter();
            XMLOutputFactory xof = XMLOutputFactory.newInstance();
            XMLStreamWriter xsw;
            xsw = xof.createXMLStreamWriter(out);
            xsw.writeStartDocument();
            xsw.writeStartElement("empresa");
            xsw.writeAttribute("CIF", "A34246801");
            xsw.writeCharacters("MegaExport");
            xsw.writeStartElement("sedes");
            xsw.writeStartElement("sede");
            xsw.writeCharacters("LEÓN");
            xsw.writeEndElement();
            xsw.writeStartElement("sede");
            xsw.writeCharacters("CÁCERES");
            xsw.writeEndElement();
            xsw.writeEndElement();
            xsw.writeEndElement();
        }
    }
}

```

```

xsw.writeEndDocument();
xsw.flush();
xsw.close();
recurso = (XMLResource) col.createResource("DatosEmpresa",
    XMLResource.RESOURCE_TYPE);
ContentHandler ch = recurso.setContentAsSAX();
XMLReader reader = XMLReaderFactory.createXMLReader();
reader.setContentHandler(ch);
reader.parse(
    new InputSource(new StringReader(out.toString())));
col.storeResource(recurso);
col.removeResource(col.getResource("Empresa_.xml"));
} catch (Exception e) {
    e.printStackTrace();
} finally {
    try {
        if (col != null) {
            col.close();
        }
    } catch (XMLDBException xe) {
        xe.printStackTrace();
    }
}
}
}
}

```



Actividad propuesta 7.1

Creo un nuevo documento, en la misma colección que el anterior programa, utilizando `setContentAsDOM`. Primero hay que crear un documento DOM, de la manera en que se vio en el capítulo anterior dedicado a XML. Puedes copiar el código de un programa de ejemplo de este capítulo para crear el documento. Si es necesario, consulta los Javadocs de la API XML:DB.

7.9. El lenguaje XQuery

XQuery es un lenguaje estándar de W3C que se creó para realizar consultas sobre documentos de XML, lo mismo que su antecesor XPath. XPath se explicó en el capítulo dedicado a XML, que incluye numerosas consultas de ejemplo en XPath. XQuery 1.0 se definió de manera que incluye XPath 2.0. Es decir, cualquier sentencia de XPath 2.0 lo es de XQuery 1.0 y devuelve los mismos resultados. Lo mismo para XQuery 3.1 respecto de XPath 3.1.

Recurso web

www

Se puede encontrar información de XQuery en el sitio web de W3C:

<http://www.w3.org/TR/xquery>

Pronto se planteó la necesidad de disponer de un lenguaje para realizar operaciones de modificación sobre documentos de XML. Para ello surgieron diversas propuestas, como extensiones de XQuery, que se citan a continuación en orden cronológico:

- XUpdate. De 2000. Promovido por el grupo que desarrolló la API XML:DB.
- XQuery Update Extension. De 2001. Promovido por Patrick Lethi.
- XQuery Update Facility (XQUF). De 2011. Es un estándar de W3C.

Estas propuestas no solo tienen nombres muy parecidos, sino que también son similares entre sí. Si la base de datos proporciona soporte para ella, es preferible utilizar la última, porque es un estándar de W3C.



PARA SABER MÁS

Debido a la frecuencia de las operaciones de búsqueda por texto en documentos de XML, y a que los documentos de XML contienen muy a menudo lenguaje natural (es decir, texto redactado por humanos para ser leído por humanos), se desarrolló, como estándar de W3C, una extensión de XQuery para facilitar consultas sobre este tipo de textos: XQuery Full Text. Se puede encontrar una concisa descripción de sus posibilidades en la siguiente dirección:

<http://docs.basex.org/wiki/Full-Text>.

7.9.1. Consultas con XQuery

En este apartado se explica XQuery con unas cuantas consultas de ejemplo. Se recomienda ejecutarlas con eXide (eXist IDE o entorno de desarrollo integrado de eXist). eXide permite navegar por los contenidos de la base de datos, ejecutar consultas de XQuery y ver los resultados. Tiene características muy útiles, como verificación y autocompletado.

Una consulta de XQuery se puede ejecutar sobre todos los documentos de una colección, como se ve en la siguiente consulta, realizada sobre una colección de ejemplo, disponible si se ha instalado “eXist-db Demo Apps”. Cada uno de los resultados se obtiene de un documento diferente.



Figura 7.4

Consulta en XQuery sobre todos los documentos de una colección, ejecutada en eXide

Las siguientes consultas de ejemplo se harán en documentos de la colección **pruebas**, creada por un programa de ejemplo anterior. Se harán en general en **Clientes.xml**, pero al-

guna de ellas también en `productos.xml`, que debe crearse previamente con eXide. Se puede enlazar la información de ambos documentos utilizando el DNI.

Cientes.xml	<pre> <clientes> <cliente DNI="78901234X"> <apellidos>NADALES</apellidos> <CP>44126</CP> </cliente> <cliente DNI="89012345E"> <apellidos>ROJAS</apellidos> <validex estado="borrado" timestamp="1528286082"/> </cliente> <cliente DNI="56789012B"> <apellidos>SAMPER</apellidos> <CP>29730</CP> </cliente> </clientes> </pre>
productos.xml	<pre> <productos> <producto nombre="tuerca"> <precio>0.25</precio> <prov id="78901234X"/> </producto> <producto nombre="tornillo"> <precio>0.10</precio> <prov id="89012345E"/> <prov id="78901234X"/> </producto> </productos> </pre>

Las sentencias de XQuery son de tipo FLWOR (*for, let, where, order by, return*). La analogía con las cláusulas de una consulta de SQL (*select, from, where, group by, having, order by*) es evidente.

CUADRO 7.2
Cláusulas de una sentencia FLWOR de XQuery

FOR	Vincula variables a expresiones en XPath. Estas expresiones pueden devolver múltiples resultados, cada una de las cuales se tiene en cuenta para el resto de las cláusulas.
LET	Permite asignar a una variable el resultado de evaluar una expresión en XPath, y se suele utilizar para evitar repetirla más de una vez. Si la expresión devuelve más de un resultado, se concatenan todos para asignar el resultado a la variable.
WHERE	Especifica condiciones que permiten filtrar los resultados de las cláusulas FOR y LET.
ORDER BY	Permite ordenar los resultados de las cláusulas FOR y LET.
RETURN	Especifica una expresión que se evalúa para cada uno de los resultados proporcionados por el conjunto de cláusulas anteriores. En esta cláusula se pueden generar directamente XML y, en ese caso, se pueden incluir sentencias de XQuery entre llaves {}, lo que resulta un mecanismo muy potente.

Actividad propuesta 7.2



Utilizando eXide, crea en la colección `pruebas` el documento `productos.xml` con los contenidos indicados.

A continuación se explican varias sentencias de XQuery de ejemplo. Se sugiere ejecutarlas en eXide y experimentar con variaciones sobre ellas.

<pre>for \$n in doc('/db/pruebas/Cientes.xml') return \$n/clientes/cliente</pre>	Devuelve los datos de todos los clientes. Cada uno de los resultados es un elemento de nombre <code>cliente</code> del documento.
<pre>for \$n in doc('/db/pruebas/Cientes.xml') return \$n/clientes/cliente/apellidos/text()</pre>	Se utiliza el paso <code>text()</code> para obtener no los elementos, sino el texto dentro.
<pre>for \$n in doc('/db/pruebas/Cientes.xml')/ clientes/cliente where substring(\$n/CP,1,2)="29" return concat(\$n/apellidos, "-", \$n/string(@DNI))</pre>	Se utiliza la cláusula WHERE para seleccionar clientes de Málaga. Se concatena un XPath a la especificación del documento <code>doc(_)</code>. Se utiliza la función <code>string()</code> para obtener el valor de un atributo. Se utiliza la función <code>concat()</code> para concatenar cadenas.
<pre>for \$n in doc('/db/pruebas/Cientes.xml')/ clientes/cliente order by number(\$n/CP) return <cli dni="{ \$n/string(@DNI) }" ape="{ \$n/apellidos }"> { \$n/CP } </cli></pre>	Se utiliza la cláusula ORDER BY con la función <code>number()</code> para ordenar numérica y no alfabéticamente. Se devuelve XML, del que algunas partes se obtienen con XQuery entre llaves <code>{ }</code>. Los resultados son XML, no texto, y su representación textual puede diferir de lo que aparece literalmente en la sentencia de XQuery, como es el caso de un cliente sin el elemento CP, cuyos datos se muestran como <code><cli dni="89012345E" ape = "ROJAS"/></code>.
<pre><clientes> { for \$n in doc('/db/pruebas/Cientes.xml')/clientes/cliente return <cliente> <ident tipo="dni">{ \$n/string(@DNI) }</ident> <apell>{ \$n/apellidos/text() }</apell> </cliente> } </clientes></pre>	Esta consulta muestra cómo con XQuery se puede incluir una sentencia FLWOR dentro de un documento de XML, y cómo XQuery permite generar documentos de XML con total flexibilidad, utilizando XPath para obtener todos los datos que sean necesarios en cualquier lugar en que sean necesarios.

<pre>let \$cli:=doc('/db/pruebas/Clientes.xml')/clientes/ cliente return <nuestroscientes>{\$cli}</nuestroscientes></pre>	Uso de cláusula LET, no es el más habitual. El XPath devuelve varios resultados, pero se concatenan y esta consulta devuelve un único resultado.
<pre>for \$cli in doc('/db/pruebas/Clientes.xml')/ clientes/cliente let \$id_cli:=\$cli/string(@DNI) return <prod_cli dni="{ \$id_cli}" ape="{ \$cli/apellidos}"> { for \$prod in doc('/db/pruebas/productos.xml')/ productos/producto/prov[@id=\$id_cli]/ return <prod> { \$prod/string(@nombre) } </prod> } </prod_cli></pre>	Uso de cláusula LET en una consulta sobre dos documentos. Se utiliza LET para guardar el DNI y en la cláusula RETURN se hace una nueva consulta para obtener todos los productos suministrados al cliente.
<pre>for \$n in doc('/db/pruebas/Clientes.xml')/ clientes/cliente order by number(\$n/CP) return element cli { attribute dni { \$n/string(@DNI) }, data (\$n/apellidos), element cp { \$n/CP/text() } }</pre>	El uso de constructores para elementos, atributos y textos de XML permite obtener un código de XQuery más escueto, aunque más alejado de la sintaxis de XML.

Por lo demás, estos ejemplos solo dan una idea de las posibilidades de XQuery. XQuery tiene muchas funciones predefinidas y se pueden definir nuevas. XQuery tiene una sentencia condicional `if ... then ... else`. Con XQuery se dispone de mucha flexibilidad para añadir más de una vez las anteriores cláusulas y no necesariamente en ese orden. Existe una cláusula `GROUP BY` similar a la del mismo nombre en SQL, y funciones de grupo `count`, `sum`, `min`, `max`, etc.

Recurso web

www

El siguiente tutorial de la empresa Altova explora todas estas posibilidades:
<https://www.altova.com/training/xquery3>

7.9.2. Sentencias de modificación de datos con XQuery

En este apartado se explican, con algunos ejemplos, las sentencias de modificación de datos disponibles para eXist, pertenecientes a lo que en su documentación se denomina *XQuery Update*

Extension. Estos ejemplos modifican el documento `Clientes.xml` de la colección `pruebas`, y se recomienda ejecutarlos con eXide. Otras bases de datos pueden utilizar distintas extensiones de XQuery, pero el planteamiento y la sintaxis son similares.

<pre>update insert <cliente DNI="67890123C"> <apellidos> GAMBOA</apellidos> <CP>52351</CP> </cliente> into doc('/db/pruebas/Clientes.xml')/clientes</pre>	Inserta datos de un nuevo cliente. Los datos se insertan como último elemento bajo el elemento <code>clientes</code> indicado.
<pre>update insert <cliente DNI="23456789D"> <apellidos>DOLCE</apellidos> <CP>11895</CP> </cliente> following doc('/db/pruebas/Clientes.xml')/clientes/ cliente[@DNI="78901234X"]</pre>	Inserta datos de un nuevo cliente después de los de otro cliente especificado. Si no se especifica la posición de inserción, esta se realiza por defecto. Existe una cláusula <code>preceding</code> para realizar la inserción después de una posición determinada.
<pre>update value doc('/db/pruebas/Clientes.xml')/clientes/cliente[@DNI="23456789D"]/ apellidos with "DORCE"</pre>	Cambia el valor en elemento <code>apellidos</code> para el cliente antes insertado.
<pre>update replace doc('/db/pruebas/Clientes.xml')/clientes/cliente[@DNI="23456789D"] with <cliente DNI="12345678Z"> <apellidos>ARCOS</apellidos> </cliente></pre>	Cambia el elemento indicado por un nuevo elemento. Nótese la diferencia de esta sentencia con la anterior. En la anterior solo se reemplazaba el texto dentro de un elemento, en esta se reemplaza el elemento entero.
<pre>update rename doc('/db/pruebas/Clientes.xml')//validez as "valid"</pre>	Cambia el nombre de elementos con nombre <code>validez</code> y les asigna el nombre <code>valid</code>.
<pre>for \$cli in doc('/db/pruebas/Clientes.xml')/ clientes/cliente/valid[@estado="borrado"] return update delete \$cli</pre>	Borra clientes bajo los que exista un elemento <code>valid</code> con un atributo <code>estado</code> con valor <code>"borrado"</code>. Las sentencias que hacen cualquier cambio deben ejecutarse con mucha precaución. Igual que antes de ejecutar una sentencia que modifica los datos con SQL es conveniente ejecutar un SELECT con la misma cláusula WHERE para asegurarse de a qué datos va a afectar, con XQuery es conveniente ejecutar antes una consulta. En este caso, la consulta consistiría en eliminar <code>update delete</code> en la anterior sentencia.

7.10. La API XQJ

XQJ es a XQuery y las bases de datos de XML lo que JDBC a SQL y las relacionales. Si JDBC permite ejecutar sentencias de SQL desde programas en Java, XQJ hace lo propio con XQuery.

y sus extensiones. La API XQJ proporciona una serie de interfaces que los *drivers* para diferentes bases de datos deben implementar. XQJ no forma parte de la biblioteca estándar de clases de Java.

Para operaciones de consulta, el planteamiento de XQJ (y de XML:DB) es similar al de JDBC para bases de datos relacionales. Para operaciones de consulta con JDBC, se ejecuta una sentencia SELECT de SQL, y se obtiene un `ResultSet`, del que se consiguen los resultados mediante un iterador. Con XQJ se hace algo similar, pero las consultas se hacen con XQuery.

Las operaciones de actualización son más problemáticas porque, como ya se ha visto anteriormente, se han propuesto varios lenguajes para este tipo de operaciones, si bien todos son similares y se han planteado como extensiones de XQuery.

Recursos web

www

En la siguiente dirección están disponibles los Javadocs de XQJ:

<http://xqj.net/javadoc>

En las siguientes direcciones se pueden descargar *drivers* de XQJ para eXist, BaseX, Sedna y Marklogic. El fichero de la descarga es un fichero comprimido que contiene varios ficheros jar que hay que incluir en el proyecto. Entre ellos hay al menos uno con la API XQJ y otro con el *driver* para la base de datos que contiene clases que implementan las interfaces de XQJ:

<http://xqj.net/exist>

<http://xqj.net/basex>

<http://xqj.net/sedna>

<http://xqj.net/marklogic>

En cada una de las páginas anteriores hay a la derecha un enlace para descargar el *driver*. Hay también un enlace: Compliance Definition Statement. Como explica allí al principio, para la conformidad con el estándar XQJ, se requiere una declaración de cómo se han implementado todos los aspectos de la especificación XQJ que en ella se describen como dependientes de la implementación. Entre ellos está el soporte para transacciones.



Actividad propuesta 7.3

¿Cuáles de los *drivers* XQJ anteriores proporcionan soporte para transacciones?

7.10.1. Establecimiento de conexiones con XQJ

Para crear un programa en Java que utilice XQJ para trabajar con una base de datos, deben añadirse al proyecto los ficheros jar incluidos en la descarga del *driver* para la base de datos.

En la figura se muestra un ejemplo con la base de datos eXist. El fichero `xqjapi.jar` contiene la API XQJ. Para establecer una conexión debe, primero, crearse una instancia de la clase que implementa la interfaz `XQDataSource`. En este caso es `ExistXQDataSource`, en `exist-xqj-1.0.1.jar`, que contiene la implementación de XQJ para eXist. Después, se debe proporcionar un valor para los parámetros de conexión necesarios para la base de datos en particular, con `setProperty`. El nombre de la clase y los parámetros de conexión son aspectos dependientes de la implementación y, por tanto, están documentados en la Compliance Definition Statement del *driver*. Se indican en el cuadro 7.3 para varias bases de datos. No es siempre necesario proporcionar valor para todas las propiedades. La propiedad `description`, por ejemplo, es siempre opcional. En algunos casos, si no se indica un puerto, se puede asumir uno por defecto. Para eXist, por ejemplo, si se indica `localhost` para `serverName`, no es necesario indicar el puerto. Por último, al igual que con XML:DB, con eXist nunca se producirá ningún error ni excepción al abrir la conexión a la base de datos desde el programa, porque esta no se establece hasta que no se realiza alguna operación que requiere el acceso efectivo a la base de datos.

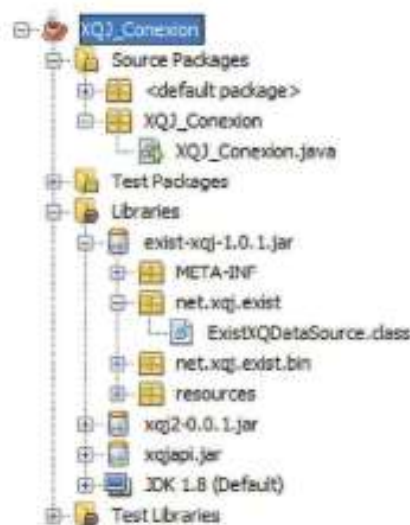


Figura 7.5
Proyecto que usa la API XQJ

CUADRO 7.3

Cadenas de conexión para *drivers* de XQJ para distintas bases de datos nativas de XML

SGBD	Clase que implementa XQDataSource	Propiedades para conexión
MarkLogic	<code>net.xqj.marklogic. MarkLogicXQDataSource</code>	<code>serverName</code> , <code>databaseName</code> , <code>port</code> , <code>user</code> , <code>password</code> , <code>description</code> , <code>mode</code>
eXist	<code>net.xqj.exist. ExistXQDataSource</code>	<code>serverName</code> , <code>port</code> , <code>user</code> , <code>password</code> , <code>description</code>
BaseX	<code>net.xqj.basex. BaseXXQDataSource</code>	<code>serverName</code> , <code>databaseName</code> , <code>port</code> , <code>user</code> , <code>password</code> , <code>description</code>
Sedna	<code>net.xqj.sedna. SednaXQDataSource</code>	<code>serverName</code> , <code>databaseName</code> , <code>port</code> , <code>user</code> , <code>password</code> , <code>description</code>

En el cuadro 7.3 se indica la clase que implementa la interfaz `XQDataSource` y las propiedades disponibles para la conexión. No es obligatorio proporcionar un valor para todas.

El siguiente programa establece una conexión con una base de datos eXist, indica si la conexión proporciona soporte para transacciones, y cierra la conexión. Para establecer la conexión, se ha creado el método `obtenConexion`, y para mostrar información específica de una excepción de tipo `XQException`, el método `muestraErrorXQuery`. Para averiguar si hay soporte para transacciones, se consultan los metadatos de la conexión, obtenidos con `getMeta-`

Data. Para que este programa funcione para otra base de datos, hay que asignar a `nomClaseDS` el nombre de la clase que implementa la interfaz `XQDataSource` para ella.

```
// Abre conexión con XQJ
package ConexionXQJ;

import javax.xml.xquery.XQDataSource;
import javax.xml.xquery.XQConnection;
import javax.xml.xquery.XQException;
import javax.xml.xquery.XQMetaData;
public class ConexionXQJ {

    private static String nomClaseDS = "net.xqj.exist.ExistXQDataSource";

    private static XQConnection obtenConexion() throws ClassNotFoundException,
        InstantiationException, IllegalAccessException, XQException {
        XQDataSource xqs=(XQDataSource) Class.forName(nomClaseDS).
            newInstance();
        xqs.setProperty("serverName", "localhost");
        xqs.setProperty("port", (puerto));
        xqs.setProperty("user", (usuario));
        xqs.setProperty("password", (contraseña));
        return xqs.getConnection();
    }

    private static void muestraErrorXQuery(XQException e) {
        System.err.println("XQuery ERROR mensaje: " + e.getMessage());
        System.err.println("XQuery ERROR causa: " + e.getCause());
        System.err.println("XQuery ERROR código: " + e.getVendorCode());
    }

    public static void main(String[] args) {
        XQConnection c = null;
        try {
            c = obtenConexion();
            XQMetaData xqmd = c.getMetaData();
            System.out.println("Conexión establecida como:"+xqmd.
                getUserName()+".");
            System.out.println(
                "Transacciones: "+(xqmd.isTransactionSupported() ? "Sí":
                    "No")+".\n"
            );
        } catch (XQException e) {
            muestraErrorXQuery(e);
        } catch (Exception e) {
            e.printStackTrace();
        } finally {
            try {
                if (c != null) {
                    c.close();
                }
            } catch (XQException xe) {
                xe.printStackTrace();
            }
        }
    }
}
```

7.10.2. Consultas con XQJ

La forma de realizar consultas con XQJ es similar a JDBC. A partir de una conexión `XQConnection` se obtiene una expresión de XQuery `XQExpression` sobre la que se ejecuta una consulta con `executeQuery(String query)`, y se obtiene un conjunto de resultados `XQResultSequence`, análogo a un `ResultSet` de JDBC. Sobre este se itera utilizando el método `next()`. Los resultados se pueden obtener en forma de `XmlStreamReader` con `getItemAsStream()`, o en forma de `Node` del modelo DOM con `getNode()`.

El siguiente programa de ejemplo recupera los apellidos de todos los clientes presentes en el documento `Clientes.xml` de la colección `/db/pruebas`.

```
// Consulta con XQJ
package XQJ_Consulta;

import javax.xml.xquery.XQDataSource;
import javax.xml.xquery.XQConnection;
import javax.xml.xquery.XQException;
import javax.xml.xquery.XQResultSequence;
import javax.xml.xquery.XQExpression;
import javax.xml.stream.XMLStreamReader;
import javax.xml.stream.XMLStreamConstants;

public class XQJ_consulta {

    private static String nomClaseDS = "net.xqj.exist.ExistXQDataSource";
    private static XQConnection obtenConexion() throws ClassNotFoundException,
        InstantiationException, IllegalAccessException, XQException {
        (...)
    }
    private static void muestraErrorXQuery(XQException e) {
        (...)
    }
    public static void main(String[] args) {
        XQConnection c = null;
        try {
            c = obtenConexion();
            String cad = "doc('/db/pruebas/Clientes.xml')/clientes/cliente/";
            String apellidos = "apellidos";
            XQExpression xqe = c.createExpression();
            XQResultSequence xqrs = xqe.executeQuery(cad);
            int i=1;
            while (xqrs.next()) {
                System.out.println("[Resultado " + (i++) + "]");
                XMLStreamReader xsr = xqrs.getItemAsStream();
                while (xsr.hasNext()) {
                    if (xsr.getEventType() == XMLStreamConstants.CHARACTERS) {
                        System.out.println(xsr.getText());
                    }
                }
                xsr.next();
            }
        } catch (XQException e) {
            muestraErrorXQuery(e);
        }
    }
}
```

```

    } catch (Exception e) {
        e.printStackTrace();
    } finally {
        try {
            if (c != null) {
                c.close();
            }
        } catch (XQException xe) {
            xe.printStackTrace();
        }
    }
}
}
}

```



Actividad propuesta 7.4

Modifica el programa anterior para obtener los resultados de la consulta como nodos del modelo DOM mediante el método `getNode()`. Escribe los resultados en forma de árbol utilizando el método `muestraNodo(Node nodo, int nivel, PrintStream ps)` de un programa de ejemplo del capítulo anterior dedicado a XML. Si es necesario, consulta los Javadocs de la API XQJ.

7.10.3. Modificaciones de documentos con XQJ

La ejecución de sentencias para modificación se hace también con una `XQExpression`, pero con el método `executeCommand`. El siguiente programa de ejemplo inserta un nuevo elemento con los datos de un nuevo cliente, en la posición anterior (cláusula `preceding`) a una determinada, indicada mediante un `XPath`.

```

// Operaciones de modificación con XQJ
package XQJ_Modificacion;

import javax.xml.xquery.XQDataSource;
import javax.xml.xquery.XQConnection;
import javax.xml.xquery.XQException;
import javax.xml.xquery.XQExpression;

public class XQJ_Modificacion {

    private static String nomClaseDS = "net.xqj.exist.ExistXQDataSource";
    private static XQConnection obtenConexion() throws ClassNotFoundException,
        InstantiationException, IllegalAccessException, XQException {
        (...)
    }
    private static void muestraErrorXQuery(XQException e) {
        (...)
    }
}

```

```

public static void main(String[] args) {
    XQConnection c = null;
    try {
        c = obtenConexion();
        String cad = "update insert "
            + "<cliente DNI=\"09876543K\">"
            + "<apellidos>LAMIQUIE</apellidos>"
            + "<CP>43001</CP>"
            + "</cliente>"
            + " preceding doc('/db/pruebas/Cientes.xml')/clientes/cliente/
              apellidos[text()='SAMPER']/..";
        XQExpression xqe = c.createExpression();
        xqe.executeCommand(cad);
    } catch (XQException e) {
        muestraErrorXQuery(e);
    } catch (Exception e) {
        e.printStackTrace();
    } finally {
        try {
            if (c != null) {
                c.close();
            }
        } catch (XQException e) {
            e.printStackTrace();
        }
    }
}
}
}

```

Actividades propuestas



- 7.5.** Escribe un programa que cambie el nombre del cliente añadido por el programa de ejemplo anterior. La sentencia utilizada debe localizar el cliente que se va a borrar mediante su DNI. Precaución: escribe una consulta en XQuery para recuperar el nombre de este cliente y después transfórmala en una sentencia que lo cambie.
- 7.6.** Escribe un programa que borre el cliente añadido por el programa de ejemplo anterior. La sentencia utilizada debe localizar el cliente que se va a borrar mediante su DNI. Precaución: escribe una consulta en XQuery para recuperar la información de este cliente y después transfórmala en una sentencia que lo borre.

7.10.4. Transacciones con XQJ

Una transacción es un conjunto de operaciones que se ejecutan como un todo, y que cumplen los requisitos ACID (atomicidad, consistencia, aislamiento y durabilidad). El concepto de transacción se explicó en detalle en el capítulo anterior dedicado a bases de datos relacionales. Para que en una aplicación se puedan utilizar transacciones, debe proporcionar soporte para ellas el

SGBD, la API y el *driver*. Ya se ha comentado que, al contrario de lo que sucede con los SGBD relacionales, solo algunos SGBD de XML nativos proporcionan soporte para transacciones, y eXist no es uno de ellos. XQJ sí permite utilizar transacciones siempre que el SGBD y el *driver* lo permitan, y además de manera muy similar a JDBC.

Al igual que en JDBC, en XQJ, por defecto, los cambios realizados por cada operación realizada sobre una conexión se confirman en la base de datos al finalizar la operación. Este comportamiento se puede cambiar con `setAutoCommit(false)` si hay soporte para transacciones, y entonces solo se confirman los cambios cuando se ejecuta `commit()`. Se pueden deshacer todos los cambios realizados en el curso de una transacción ejecutando `rollback()`. De acuerdo con la especificación XQJ, si se hace `setAutoCommit(false)` sobre una conexión para la que no hay soporte de transacciones, la implementación (léase el *driver*) debe lanzar un error. Si se produce alguna excepción durante la ejecución de una transacción, esta debe capturarse y debe ejecutarse el método `rollback` para deshacer los cambios. Todo esto se refleja en el siguiente código de programa para implementar una transacción:

```
// Transacción con XQJ
package XQJ_Transaccion;

import javax.xml.xquery.XQDataSource;
import javax.xml.xquery.XQConnection;
import javax.xml.xquery.XQException;
import javax.xml.xquery.XQExpression;

public class XQJ_Transaccion {
    // Sedna soporta transacciones.
    private static String nomClaseDS = "net.cfooster.sedna.DatabaseImpl";

    private static XQConnection obtenConexion() throws ClassNotFoundException,
        InstantiationException, IllegalAccessException, XQException {
        (...)
    }

    private static void muestraErrorXQuery(XQException e) {
        (...)
    }

    public static void main(String[] args) {
        XQConnection c = null;
        try {
            c = obtenConexion();
            c.setAutoCommit(false);
            XQExpression xqe = c.createExpression();
            // Ahora se pueden ejecutar múltiples sentencias de modificación,
            // entre las que se pueden intercalar sentencias de consulta con
            // executeQuery
            xqe.executeCommand("update... ");
            xqe.executeCommand("update... ");
            (...)
            c.commit();
        } catch (XQException e) {
            muestraErrorXQuery(e);
            try {
                c.rollback();
                System.err.println("Se hace ROLLBACK");
            }
        }
    }
}
```

```

    } catch (XQException er) {
        System.err.println("ERROR haciendo ROLLBACK");
        muestraErrorXQuery(er);
    }
} catch (Exception e) {
    e.printStackTrace();
} finally {
    try {
        if (c != null) {
            c.close();
        }
    } catch (XQException e) {
        e.printStackTrace();
    }
}
}
}
}

```

Resumen

- XML es un formato muy flexible para el almacenamiento e intercambio de información. Se pueden distinguir, a grandes rasgos, dos áreas de aplicación de XML: XML centrado en documentos (*document-centric XML*) y XML centrado en datos (*data-centric XML*).
- Las bases de datos de XML nativas almacenan los documentos de XML en estructuras diseñadas y optimizadas para documentos de XML, y además proporcionan soporte para tecnologías de XML tales como XPath, XQuery y XSL.
- Existen también bases de datos no nativas de XML pero con capacidades para XML (*XML-enabled*). Un ejemplo de ellas son las bases de datos relacionales que implementan SQL/XML, añadido al estándar SQL en SQL:2003. Pueden ser la mejor opción para aplicaciones de XML basadas en datos (*data-centric XML*).
- Las bases de datos de XML nativas suelen organizar los documentos en colecciones. Algunas permiten definir una jerarquía de colecciones, y algunas permiten almacenar otros tipos de documentos además de documentos de XML.
- Las bases de datos de XML nativas implementan mecanismos de indexación estructural, en los que se tienen en cuenta los valores almacenados en todos los elementos y atributos.
- El principal lenguaje de consulta para bases de datos de XML es XQuery, un estándar de W3C, que es un superconjunto de XPath, otro estándar anterior de W3C. Se han propuesto varias extensiones de XQuery para permitir operaciones de modificación sobre el contenido de los documentos de XML; la última es XQUF, también un estándar de W3C.
- Las API estándares existentes para bases de datos nativas de XML son XML:DB y XQJ. XQJ está concebida para ejecutar consultas de XQuery y también sentencias de modificación de documentos de XML expresadas en alguna de las extensiones de XQuery propuestas para ello, entre ellas el estándar XQUF de W3C.
- Algunas bases de datos nativas de XML proporcionan un buen soporte para transacciones, pero este no es el caso en general, muy al contrario que con las bases de datos relacionales.

Ejercicios propuestos



Todos los documentos que se creen para estas actividades deben crearse en una colección con nombre `ejercicios_bd_XML`, a no ser que se indique otra cosa.

1. Crea un programa genérico al que se le pasen por línea de comandos el nombre de una colección, el nombre de un documento y una consulta en XQuery. El programa ejecutará la consulta y devolverá una lista de resultados. Si no se indica un nombre de documento, la consulta debe realizarse sobre todos los documentos de la colección. Los resultados se deben mostrar en formato XML. Si cada resultado es un nodo de un árbol DOM, debe mostrarse el fragmento del documento por debajo del nodo obtenido. Prueba este programa con las consultas de ejemplo de este capítulo.
2. Crea un programa genérico al que se le pasen por línea de comandos el nombre de una colección, el nombre de un documento y una sentencia de modificación de XQuery para ejecutar sobre el documento en cuestión. El programa debe crear una copia del documento original en la misma colección, en cuyo nombre debe incluirse el nombre del documento original y una marca de tiempo incluyendo la fecha y la hora. Una vez ejecutada la consulta, deben mostrarse los contenidos tanto del documento original (es decir, de la copia que se ha hecho previamente) como del documento resultante tras ejecutar la sentencia. Debe pedirse confirmación al usuario para confirmar los cambios. Si la da, debe borrarse la copia creada. Si no, debe borrarse el documento y renombrar la copia, para que todo quede como al principio. Si se produce cualquier excepción, todo debe quedar como al principio. Prueba este programa con las sentencias de modificación de datos de este capítulo.
3. Crea una clase con varios métodos para consultar datos de un cliente, crear un cliente, borrar un cliente y modificar datos de un cliente (al menos, nombre, código postal y DNI). Los datos de los clientes se deben almacenar en un documento de XML con un formato como el del documento de ejemplo utilizado durante este capítulo. A los métodos que realizan cualquier cambio sobre un cliente se les debe indicar el cliente mediante el DNI. Debe haber un método para obtener el número de clientes almacenados y otro para mostrar todos los datos de todos los clientes, que se puede limitar a escribir los contenidos del documento. El constructor de la clase debe crear el documento para almacenar los datos de los clientes, en caso de que no exista, conteniendo solo un elemento con nombre `clientes`. Debe crearse un programa de prueba en el método `main()` que realice varias operaciones de creación, borrado y modificación de clientes, y por último muestre todos los datos de todos los clientes.
4. Crea una clase con la misma interfaz que la anterior, pero que internamente funcione con un esquema de almacenamiento diferente. A saber, para cada cliente debe haber un documento de XML con sus datos, y todos ellos deben estar en una misma colección, que inicialmente estará vacía. No debería ser necesario que, tras cambiar el DNI de un cliente, sigan coincidiendo el valor del DNI almacenado y el nombre del fichero, pero, opcionalmente, se puede hacer que se cambie el nombre del documento al modificar el DNI. En su defecto, se puede crear un método estático de la clase para renombrar correctamente todos los documentos para que su nombre coincida con el del DNI almacenado. El método `main()` debe ser el mismo que para la anterior actividad.

5. Crea una aplicación que gestione los datos de los empleados de una empresa almacenados en un fichero XML. El fichero debe tener la estructura siguiente:

```
<empresa cif="_" nombre="_">
  <departamento código="_" nombre="_">
    <empleado dni="_" nombre="_" />
    <puesto>...</puesto>
  </empleado>
  <empleado dni="_" nombre="_" />
  ...
  <empleado dni="_" nombre="_" />
  <puesto>...</puesto>
</departamento>
...
<departamento código="_" nombre="_">
  <empleado dni="_" nombre="_">
  ...
  </empleado>
  ...
  <empleado dni="_" nombre="_">
  ...
  </empleado>
</departamento>
</empresa>
```

En resumen, dentro de la empresa hay departamentos y dentro de cada departamento, empleados. No hace falta hacer validaciones sobre el formato de ningún dato. No puede haber dos departamentos distintos con el mismo código, ni dos empleados distintos con el mismo DNI, y ambos atributos son obligatorios. También los atributos `cif` y `nombre` del elemento `empresa`. El elemento `puesto` es opcional. Un empleado no puede estar en más de un departamento.

Debe crearse una clase `GestorEmpresa` que gestione los datos del documento, almacenado en una base de datos de eXist. El documento debe estar en una colección con nombre `empresa_y_empleados`. El constructor debe tener como parámetros el CIF y el nombre de la empresa, y debe crear la colección si no existe, y el documento si no existe, con el CIF y el nombre de la empresa. El método `main()` de la clase contendrá un programa de prueba que utilice esta clase para crear varios departamentos, añadir varios empleados en distintos departamentos y cambiar algunos empleados de departamento, utilizando los métodos que se indican más adelante. Los métodos no deben gestionar ninguna excepción, esto debe hacerlo el programa principal en el método `main()`. Esta clase debe tener métodos para:

- Crear un departamento, dado su código y nombre. Si ya existe un departamento con ese nombre, se debe mostrar un mensaje de error y devolver `false`. En otro caso, se debe devolver `true`.
- Crear un empleado, dado su DNI y nombre y el código del departamento al que pertenece. Si ya existe un empleado con ese DNI, se debe mostrar un mensaje de error y devolver `false`. También si no existe el departamento. En otro caso, se debe devolver `true`.

- Eliminar un empleado dado su DNI. Debe devolver `false` y mostrar un mensaje de error si el empleado no existe. En otro caso, debe eliminar el empleado y devolver `true`.
- Eliminar un departamento dado su código. Debe devolver `false` y mostrar un mensaje de error si el departamento no existe o si tiene algún empleado. En otro caso, debe eliminar el departamento y devolver `true`.
- Cambiar un empleado de departamento, dado el DNI del empleado y el código del nuevo departamento. Debe devolver `true` si se puede hacer la operación. En caso contrario, deben mostrarse mensajes de error y devolver `false`. A saber, si el empleado o el departamento no existen, o si el empleado ya está en el departamento.

El método `main()` podría quedar más o menos así:

```
try {
    GestorEmpresa gEmpresa = new GestorEmpresa(nomFich);
    if(!gEmpresa.creaDepartamento(codDep1,nomDep1)) return;
    if(!gEmpresa.creaEmpleado(dniEmp1,nomEmp1,codDep1)) return;
    if(!gEmpresa.creaEmpleado(dniEmp2,nomEmp2,codDep1)) return;
    if(!gEmpresa.creaDepartamento(codDep2,nomDep2)) return;
    if(!gEmpresa.creaEmpleado(dniEmp3,nomEmp3,codDep2)) return;
    System.out.println("Cambios realizados correctamente");
}
catch(...) {
    ...
}
catch(...) {
    ...
}
```

ACTIVIDADES DE AUTOEVALUACIÓN

- XML es un formato de documento:
 - ☐ a) Con una estructura fija y regular.
 - ☐ b) Que tiene dos variantes: una para almacenar datos y otra para textos en lenguaje natural.
 - ☐ c) Muy sencillo y flexible, pero con el que se pueden también crear documentos con una estructura fija y regular.
 - ☐ d) Con una estructura que puede ser jerárquica o tabular.
- El almacenamiento de documentos de XML en bases de datos relacionales:
 - ☐ a) Es siempre una mala alternativa, al tener los documentos de XML una estructura jerárquica que no se presta al almacenamiento en estructuras tabulares.

- ☐ b) Es la única manera de poder disponer de transacciones en aplicaciones que trabajan con datos en formato XML.
 - ☐ c) Solo es posible si se almacenan los documentos como un todo en campos de tipo BLOB, CLOB o VARCHAR.
 - ☐ d) Ninguna de las respuestas anteriores es correcta.
3. Las bases de datos de XML nativas:
- ☐ a) Son todas de código abierto, al ser XML un estándar de W3C.
 - ☐ b) Son las únicas que permiten utilizar tecnologías de XML tales como XML Schema, XPath, XQuery y XSL, sobre los documentos de XML almacenados.
 - ☐ c) Suelen organizar los documentos en colecciones.
 - ☐ d) Solo permiten almacenar documentos en formato XML.
4. Las colecciones en las bases de datos de XML nativas:
- ☐ a) Son jerárquicas, es decir, puede haber colecciones dentro de colecciones.
 - ☐ b) Deben siempre crearse explícitamente antes de añadir documentos a ellas.
 - ☐ c) Deben contener siempre documentos de igual tipo para toda la colección.
 - ☐ d) Ninguna de las respuestas anteriores es correcta.
5. SQL/XML:
- ☐ a) Es un lenguaje de consulta para bases de datos de XML inspirado en SQL.
 - ☐ b) Es una extensión para XML incluida en todas las versiones de Oracle desde la 12c.
 - ☐ c) Es una parte del estándar SQL que añade soporte para XML.
 - ☐ d) Es un SGBD comercial multimodelo, a la vez relacional y de XML nativo.
6. Los índices de las bases de datos de XML nativas:
- ☐ a) Son más costosos de mantener que los de las bases de datos relacionales.
 - ☐ b) No son actualizados automáticamente en algunos SGBD de XML nativos.
 - ☐ c) Son estructurales, basados en los valores de todos los elementos y atributos.
 - ☐ d) Todas las respuestas anteriores son correctas.
7. Las transacciones en los SGBD de XML nativos:
- ☐ a) No siempre están soportadas, pero si no lo están, se pueden conseguir con un *driver* de XQJ, porque el soporte para transacciones es obligatorio en XQJ.
 - ☐ b) No están soportadas por muchos de ellos, al contrario de lo que sucede con los SGBD relacionales.
 - ☐ c) Están en general soportadas, pero, como son muy costosas computacionalmente, suelen estar deshabilitadas por defecto.
 - ☐ d) Solo están soportadas por la API XQJ, pero no por XML:DB.
8. El lenguaje XQuery:
- ☐ a) Es un subconjunto del lenguaje XPath.
 - ☐ b) Permite realizar consultas sobre varios documentos de XML, siempre que estén en la misma colección.
 - ☐ c) Obliga a incluir las cláusulas de una sentencia FLWOR en un orden fijo, al igual que sucede con las cláusulas de una sentencia SELECT de SQL.

- ☐ d) Permite la anidación de consultas FLWOR, es decir, el uso de una consulta FLWOR dentro de una cláusula de una consulta FLWOR.

9. La API XML:DB:

- ☐ a) No permite realizar consultas con XQuery.
☐ b) Está obsoleta y se utiliza cada vez menos, en especial para nuevos desarrollos.
☐ c) No proporciona servicios para transacciones.
☐ d) Permite realizar tareas administrativas y de gestión que XQJ no permite.

10. La API XQJ:

- ☐ a) No permite realizar operaciones de modificación sobre documentos de XML.
☐ b) Es parte de las bibliotecas estándares de Java (*Java standard libraries*).
☐ c) Permite obtener los resultados de una consulta como nodos de DOM.
☐ d) Permite borrar documentos enteros de una colección.

SOLUCIONES:

1. ☐ a ☐ b ☒ c ☐ d

2. ☐ a ☐ b ☐ c ☒ d

3. ☐ a ☐ b ☒ c ☐ d

4. ☐ a ☐ b ☐ c ☒ d

5. ☐ a ☐ b ☒ c ☐ d

6. ☐ a ☐ b ☐ c ☒ d

7. ☐ a ☒ b ☐ c ☐ d

8. ☐ a ☐ b ☐ c ☒ d

9. ☐ a ☐ b ☐ c ☒ d

10. ☐ a ☐ b ☒ c ☐ d

Componentes para el acceso a datos

Objetivos

- ✓ Comprender qué es un componente de *software* y su importancia en la ingeniería del *software*.
- ✓ Entender el modelo de componentes para Java SE, basado en JavaBeans.
- ✓ Conocer los elementos fundamentales para desarrollar aplicaciones web en la plataforma Java: *servlets* y JSP.
- ✓ Comprender el modelo MVC y la manera en que se pueden desarrollar con Java aplicaciones web según este modelo.
- ✓ Desarrollar aplicaciones web sencillas en la plataforma Java SE según el modelo MVC y que utilicen JavaBeans para el modelo y ORM para persistencia de objetos.
- ✓ Estudiar el modelo de componentes para Java EE basado en EJB.
- ✓ Trabajar con aplicaciones web sencillas en la plataforma Java EE que hagan uso de EJB para el modelo, de su soporte integrado de JPA para persistencia de objetos y de JTA para transacciones.

Modelo MVC. Del inglés *model, view, controller*, es decir, modelo, vista, controlador. Modelo que estructura las aplicaciones en tres partes diferenciadas y que interactúan entre sí: el modelo, formado por un conjunto de objetos que constituyen un modelo de los datos relevantes para la aplicación; la vista, que muestra el estado del modelo de una manera significativa para los usuarios; y el controlador, que responde a las acciones realizadas por los usuarios sobre la vista, realiza los cambios apropiados sobre el modelo y actualiza la vista para que refleje los cambios realizados sobre el modelo.

Servidor de aplicaciones. Servidor que proporciona determinados servicios a aplicaciones. También puede proporcionar contenedores para determinados modelos de componentes.

Servlet. Objeto que se despliega en un contenedor de *servlets* de la plataforma Java EE y con el que se puede interactuar mediante el protocolo HTTP o HTTPS.

8.1. Componentes de software

Los sistemas informáticos tienen una enorme complejidad, tanto en lo que respecta al *hardware* como al *software*. La modularización facilita el desarrollo de sistemas complejos. Un sistema modular no se desarrolla como un bloque monolítico, sino que integra un conjunto de módulos o componentes. Un componente queda perfectamente especificado por lo que hace (su funcionalidad) y por cómo interactúa con el mundo exterior (sus interfaces). Esto permite su reutilización en diversos sistemas. El diseñador de un sistema que utiliza componentes solo tiene que tener en cuenta sus especificaciones, es decir, su funcionalidad y sus interfaces, para integrarlo en el sistema. La posibilidad de diseñar y desarrollar un sistema basándose en componentes reutilizables es especialmente importante en la ingeniería del *software*. Los componentes se pueden desarrollar y probar de manera independiente, y se distribuyen como paquetes independientes que se integran en el sistema tal y como vienen, aunque requieran, posiblemente, una configuración inicial. Esto reduce la complejidad global del sistema y contribuye a mejorar la calidad y reducir el tiempo de desarrollo.

El planteamiento modular para el desarrollo de sistemas complejos se viene utilizando de manera general, desde hace mucho tiempo, para sistemas electrónicos, y en particular para *hardware* de ordenadores. A una placa base, por ejemplo, se pueden conectar diversos componentes o módulos, como memoria, microprocesador, dispositivos de almacenamiento y muchos otros. Pero es mucho más reciente y está menos generalizado para sistemas de *software*.

8.2. Modelos de componentes

Un modelo de componentes define unos requisitos formales para los componentes, así como mecanismos para la interacción con ellos. Una plataforma de componentes es una infraestructura de *software* sobre la que se pueden desplegar los componentes y que proporciona servicios básicos para ellos.

Los primeros modelos de componentes se crearon para componentes visuales. Microsoft fue pionera con OLE y COM. Pronto surgieron otros modelos de componentes, como los JavaBeans para la plataforma Java. Los IDE (entornos integrados de desarrollo) incluyeron soporte para su uso y desarrollo. Más tarde, se empezaron a utilizar componentes no visuales, como por ejemplo componentes para el acceso a datos (DAO o *data access objects*).

Algunos modelos de componentes son:

- .NET de Microsoft. Es el último paso en la evolución desde OLE, pasando por COM, VBX, OCX, ActiveX, COM+ y DCOM.
- JavaBeans para Java SE (Java Standard Edition).
- EJB para Java EE (Java Enterprise Edition).
- CORBA (Common Object Request Broker Architecture), un estándar definido por OMG (Object Management Group).

8.3. La plataforma Java: Java SE y Java EE

Java es un lenguaje de programación en torno al cual se ha desarrollado una plataforma de computación: la plataforma Java. Existen dos versiones de esta plataforma: Java SE y Java EE, y un modelo de componentes para cada una.

Java es un lenguaje compilado, pero con la peculiaridad de que no se compila para ningún *hardware* ni sistema operativo en particular, sino a *bytecode*, que puede ser directamente ejecutado por una máquina virtual de Java (o JVM, siglas en inglés de *Java virtual machine*). Los programas de Java, una vez compilados a *bytecode*, se pueden ejecutar sobre cualquier sistema operativo, siempre que para él exista una máquina virtual de Java. La máquina virtual de Java forma, junto con una biblioteca de clases estándares, el JRE (*Java runtime environment* o entorno de ejecución de Java). Hay dos versiones del JRE:

- Java SE. Incluye una biblioteca de clases de uso general. El modelo de componentes para Java SE está basado en JavaBeans.
- Java EE. Incluye una biblioteca de clases ampliada para dar soporte a aplicaciones empresariales. Incluye la especificación de un servidor de aplicaciones con diversos tipos de contenedores para desplegar diversos tipos de elementos, a saber: contenedores de *servlets* y contenedores de EJB. El modelo de componentes para Java EE está basado en EJB.

Java SE y Java EE son especificaciones. El lenguaje de programación Java, la máquina virtual de Java, Java SE y Java EE fueron desarrollados originalmente por Sun, pero sus especificaciones son públicas. El desarrollo de la plataforma Java ha estado siempre a cargo de una comunidad liderada, primero, por Sun y, después, por Oracle, que han desarrollado siempre implementaciones de referencia para cada nueva versión, cuyo código fuente se ha hecho público. De esta manera, cualquier empresa o institución puede desarrollar y vender su propia implementación. Esto es lo que han hecho empresas como BEA, IBM y otras, que han desarrollado y vendido servidores de aplicaciones conformes a la especificación Java EE.

Oracle siempre ha mantenido Java EE como un proceso de la comunidad Java (JCP o *Java community process*). En septiembre de 2017, Oracle anunció que cedería Java EE a la fundación Eclipse. Pero Oracle no ha cedido el nombre Java EE, por lo que su nuevo nombre se cambió a EE4J (Eclipse Enterprise for Java). En febrero de 2018 se anunció que el nuevo nombre sería Jakarta EE. Aquí se le seguirá llamando Java EE.

8.4. JavaBeans

El modelo de componentes de Java SE se basa en los JavaBeans. Estos se empezaron a utilizar como componentes visuales para interfaces gráficas de usuario. Más adelante se empezaron a utilizar JavaBeans no visuales con otras finalidades, como por ejemplo el acceso a datos.

RECUERDA

- ✓ No hay que confundir los JavaBeans con los Enterprise JavaBeans o EJB, que se verán posteriormente.

Los JavaBeans son clases de Java que cumplen una serie de requisitos formales. La mayoría de ellos son los que se exigían a las clases de Java (POJO) para poder ser clases persistentes con Hibernate. De hecho, estos POJO son, en general, JavaBeans, aunque los JavaBeans pueden tener algunas características adicionales relacionadas con la gestión de eventos. Los requisitos que tienen que cumplir los JavaBeans son:

1. Un constructor sin argumentos, que no tiene por qué ser el único constructor.
2. Implementar la interfaz `Serializable`, para lo que basta con incluir en su definición la opción `implements Serializable`.
3. Propiedades definidas como privadas (`private`) y accesibles mediante métodos públicos `getter` y `setter`. El prefijo del nombre de un método `getter` debe ser `get`, a menos que sea de tipo `boolean`, en cuyo caso puede ser `is`. Las propiedades pueden ser:
 - a) *Simples*: constan de un único valor. Su método `getter` es de la forma `Tipo getX()`, y el `setter` de la forma `void setX(Tipo valor)`, donde `Tipo` es el tipo de la propiedad y `x` su nombre pero con la inicial en mayúsculas.
 - b) *Indexadas*: constan de un conjunto de valores. Su métodos `getter` y `setter` tienen un parámetro de tipo `int`, a saber: `Tipo getX(int índice)`, y `void setX(int índice, Tipo valor)`.
 - c) *Compartidas*: son propiedades de cuyos cambios de valor se notifica a otros objetos. Para mantener la lista de objetos a los que notificar del cambio en el valor de una propiedad compartida, se deben implementar los métodos `addChangeListener` para añadir un objeto a la lista, y `removeChangeListener` para eliminar un objeto de la lista. Para ello se utiliza normalmente un objeto auxiliar de la clase `PropertyChangeSupport`. Los JavaBeans receptores deben implementar el método `propertyChange` de la interfaz `PropertyChangeListener`. Las notificaciones de cambios en el valor de la propiedad se hacen con los métodos `firePropertyChange` o `fireIndexedPropertyChange` de dicho objeto auxiliar.
 - d) *Restringidas*: son similares a las compartidas, pero los objetos notificados pueden vetar el cambio de valor. Para mantener la lista de objetos notificados se implementan `addVetoableChangeListener` y `removeVetoableChangeListener`. Para ello se utiliza normalmente un objeto auxiliar de la clase `VetoableChangeSupport`. Los JavaBeans receptores deben implementar el método `vetoableChange` de la

interfaz `VetoableChangeListener`. Las notificaciones de cambios en el valor de la propiedad se hacen llamando al método `fireVetoableChange` de dicho objeto auxiliar. El objeto receptor de la notificación puede lanzar una excepción de la clase `PropertyVetoException`, en cuyo caso el objeto emisor capturaría la excepción y restauraría el antiguo valor.

Para gestionar los JavaBeans se utilizan los siguientes mecanismos:

1. *Reflexión e introspección.* Java SE incluye una API de reflexión que permite a un programa en Java obtener información detallada acerca de cualquier clase, incluyendo sus atributos y métodos. También una API de introspección que permite obtener información acerca de los JavaBeans utilizando la reflexión e interpretando la información obtenida teniendo en cuenta los convenios de nomenclatura utilizados en los JavaBeans. Se puede proporcionar directamente información acerca de un JavaBean con una clase cuyo nombre sea el del JavaBean más `BeanInfo` y que implemente la interfaz `BeanInfo`. Una posibilidad, más sencilla, es que esta clase extienda la clase `SimpleBeanInfo`, que implementa esta interfaz.
2. *Persistencia.* La posibilidad de almacenar los contenidos actuales de un JavaBean para restaurarlo posteriormente. La interfaz `Serializable` implementada por las clases de JavaBeans proporciona un mecanismo básico de persistencia en ficheros. Pero se suele implementar la persistencia de JavaBeans mediante ORM (correspondencia objeto-relacional).
3. *Personalización.* La posibilidad de modificar la apariencia y la conducta del JavaBean durante el diseño. Esto se puede hacer, por ejemplo, utilizando editores de propiedades. Esto es especialmente importante para los componentes visuales.

Los atributos, métodos y eventos que se declaran como públicos constituyen la interfaz del JavaBean. La funcionalidad viene determinada por las operaciones que realizan sus métodos (lo que hacen), y no por su implementación (cómo lo hacen). La especificación de la funcionalidad de un JavaBean debe formar parte de la documentación del JavaBean y distribuirse junto con él. Los JavaBeans se pueden empaquetar para su distribución como parte de un fichero de tipo jar (Java Archive).

8.5. El modelo MVC para desarrollo de aplicaciones web con Java

En este capítulo se muestra el desarrollo en Java de una aplicación web de ejemplo basada en la base de datos para sedes, departamentos y empleados, utilizada en un capítulo anterior dedicado a ORM. Una aplicación web es una aplicación a la que se accede mediante el protocolo HTTP o HTTPS, normalmente utilizando un navegador web.

Se desarrollará la misma aplicación para Java SE y para Java EE. La aplicación se desarrollará conforme al modelo MVC. El modelo MVC es un modelo para el desarrollo de aplicaciones visuales, es decir, con una interfaz gráfica de usuario. Las aplicaciones MVC constan de tres partes que interactúan entre sí:

- *Modelo.* Constituido por objetos persistentes. En el caso de Java, el modelo se implementa mediante JavaBeans. Para aplicaciones basadas en Java EE se utilizan también EJB, que se verán en breve.

- **Vista.** Se encarga de la presentación del modelo en un formato apropiado y significativo para las personas que utilizan la aplicación. La vista es una interfaz de usuario interactiva, en la que se muestra información, y sobre la que el usuario puede actuar para realizar determinadas operaciones sobre el modelo. En el caso de Java, la vista se implementa mediante ficheros de tipo JSP, que se verán en breve.

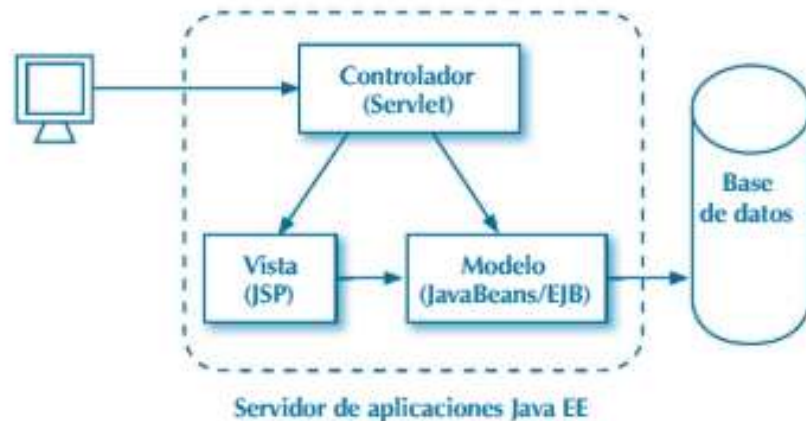


Figura 8.1
Modelo MVC para aplicaciones basadas en Java SE

- **Controlador.** Tiene como misión responder a las acciones del usuario sobre la vista y, si es necesario, realizar las modificaciones pertinentes sobre el modelo y actualizar la vista para reflejar estas modificaciones. En el caso de Java, el controlador se implementa mediante *servlets*, que se verán en breve.

Es muy recomendable utilizar el modelo MVC para el desarrollo de aplicaciones web. Las aplicaciones que siguen este modelo tienen una relativa complejidad intrínseca. Pero ello se compensa por su mayor fiabilidad, robustez, mantenibilidad y escalabilidad, especialmente importantes para aplicaciones empresariales de gran envergadura.

Existen distintos *frameworks* para el desarrollo de aplicaciones web según el modelo MVC con Java. Entre ellos se pueden citar JSF, Struts y Spring. Los IDE más importantes facilitan la creación de aplicaciones para estos *frameworks* mediante *plugins*, que proporcionan asistentes que automatizan la creación de la estructura fundamental de una aplicación. Los *frameworks* ayudan a evitar tareas repetitivas y complejas para la creación y puesta a punto de la estructura y los componentes fundamentales de las aplicaciones, así como de servicios básicos para ellas. Pero son relativamente complejos y requieren un tiempo significativo de aprendizaje, y por ello no se utilizarán para las aplicaciones de ejemplo.

Para desarrollar una aplicación web en Java conforme al modelo MVC, es necesario un conocimiento básico de las tecnologías que lo hacen posible. Este es un campo muy amplio, por lo que solo se proporcionarán breves explicaciones que permitan entender sus fundamentos y su aplicación en las aplicaciones desarrolladas. El objetivo es proporcionar una buena base a partir de la cual se puedan ampliar conocimientos, investigar y profundizar.

8.6. JSP (JavaServer Pages)

JSP se utiliza para la vista, lo que en el ámbito de las aplicaciones web significa la generación dinámica de páginas HTML. Una página JSP permite generar código HTML utilizando el lenguaje de programación Java. En este sentido, es similar a tecnologías como PHP y ASP.

Una página JSP tiene el aspecto de una página web, con etiquetas de HTML, pero además puede incluir directivas propias de JSP, fragmentos de código en lenguaje Java y referencias a constantes y variables de Java.

8.6.1. Directivas de JSP

Tienen la sintaxis `<%@ directiva atributo="valor" %>`. Se puede especificar valor para más de un atributo.

CUADRO 8.1
Directivas de JSP

<code><%@page import="clase"%></code>	Importa clases o paquetes de Java. Es el equivalente a <code>import clase</code> ; Se puede especificar más de una clase separándolas por punto y coma.
<code><%@page contentType="tipo_mime" pageEncoding="codificación"%></code>	Especifica el tipo MIME para la respuesta del protocolo HTTP, si es un tipo de texto, la codificación. Por defecto es <code>contentType="text/html; charset=ISO-8859-1"</code> . Se puede especificar UTF-8 con <code>contentType="text/html" pageEncoding="UTF-8"</code> .
<code><%@page errorPage="página_de_error"%></code>	Especifica la página de error que será invocada si ocurre alguna excepción.
<code><%@page isErrorPage="true"%></code>	Especifica que se trata de una página de error. Solo este tipo de páginas tienen acceso a una variable <code>exception</code> , que contiene la excepción.
<code><%@include file="fichero"%></code>	Incluye en el JSP el contenido del fichero especificado.

8.6.2. Scriptlets

Son fragmentos de código Java con la sintaxis:

```
<% código de Java %>
```

8.6.3. Variables implícitas

Son variables predefinidas cuyo valor proporciona información acerca del entorno de ejecución de JSP. Entre ellas se tienen las que se recogen en el cuadro 8.2.

CUADRO 8.2
Variables implícitas de JSP

<code>request</code>	Contiene la petición HTTP que se le ha hecho llegar al JSP al invocarlo.
<code>response</code>	Contiene la respuesta HTTP. El JSP puede acceder a este objeto y modificarlo, para proporcionar la respuesta HTTP apropiada.

[.../...]

CUADRO 8.2 (CONT.)

out	Permite escribir directamente en la salida del JSP, es decir, en el HTML generado. No es práctica recomendable utilizar directamente esta variable para escribir en la salida. En breve se explicará cómo se puede escribir de una manera alternativa, y preferible, en la salida del JSP.
exception	Ya se ha hablado de ella, está disponible solo para páginas definidas como páginas de error mediante una directiva apropiada, y contiene la excepción que se produjo en la página que invocó la página de error.

8.6.4. Referencias a variables de Java

Se puede escribir el valor de una variable en la salida del JSP de la siguiente manera:

```
<%=nombre_variable%>
```

En lugar de una variable, se podría incluir una expresión. Un bloque como el anterior se incluiría en medio de un fragmento de HTML, para insertar el valor de la variable o expresión. Esto sería equivalente, y preferible, a por ejemplo:

```
<%out.print(variable)%>
```

8.7. Servlets

Un *servlet* es un proceso servidor accesible mediante el protocolo HTTP (o HTTPS). Un *servlet* está todo el tiempo a la escucha de peticiones HTTP. Cuando recibe una petición, responde en el mismo protocolo. Los aspectos fundamentales acerca de los *servlets* son:

1. *Programación.* En lenguaje Java, un *servlet* es un objeto de una clase que implementa la interfaz **Servlet**.
2. *Despliegue.* En un contenedor de *servlets*, por ejemplo, Apache Tomcat. Un contenedor de *servlets* forma parte de un servidor de aplicaciones en la plataforma Java EE. El fichero de despliegue de una aplicación web para Java EE contiene toda la información acerca de los *servlets* que necesita el contenedor de *servlets* para trabajar con ellos.
3. *Interacción.* Los *servlets* reciben peticiones HTTP o HTTPS y responden en el mismo protocolo. El contenedor de *servlets* aloja los *servlets*, les proporciona servicios básicos, y hace de intermediario (o *proxy*) entre ellos y los clientes. Cuando recibe una petición de un cliente, la redirige al *servlet* apropiado y, cuando obtiene la respuesta del *servlet*, la envía de vuelta al cliente.

Las páginas JSP se implementan mediante *servlets*. JSP no es más que un mecanismo para facilitar el desarrollo de *servlets* que generan código HTML. Cuando se invoca una página JSP

mediante una petición HTTP por primera vez, un compilador de JSP genera un *servlet* a partir de ella, lo compila y le redirige la petición. A partir de ese momento, cada vez que se invoca la página JSP, la petición se redirige directamente al *servlet*.

Un contenedor de *servlets* como Tomcat funciona como servidor web para páginas HTML además de como proxy para los *servlets*. Cuando recibe una petición para una página HTML, la sirve directamente como haría cualquier servidor web.

NetBeans integra un servidor Tomcat y soporte para la creación y el despliegue de *servlets*.

8.8. Desarrollo de una aplicación web MVC basada en JavaBeans

Este apartado proporciona los conocimientos mínimos para el desarrollo de una aplicación web sencilla en Java basada en el modelo MVC. La atención se centra en la funcionalidad de acceso a datos. Para mayor simplicidad, no se utilizará ningún *framework*.

La aplicación se desarrollará para el modelo de datos utilizado para el capítulo anterior dedicado a persistencia de objetos mediante ORM. Permitirá crear sedes y listar las existentes, así como mostrar los detalles de una sede, incluyendo sus departamentos. Será sencillo añadir a esta aplicación el borrado y la modificación de sedes, con lo que se tendrá una aplicación CRUD para sedes. CRUD son las siglas de las operaciones básicas *create*, *read*, *update*, *delete* (crear, leer, actualizar, borrar). Será también muy sencillo añadir las mismas operaciones para los departamentos de una sede y los empleados de un departamento.

La aplicación representará los datos de una sede mediante JavaBeans. Se usará Hibernate para su persistencia en una base de datos relacional.

8.8.1. Creación de la aplicación web

Lo primero es crear una aplicación web en NetBeans. Esto se hace con la opción "File", "New Project...", y seleccionando después la opción "Java Web", "Web Application". Después se indica un nombre para el proyecto.

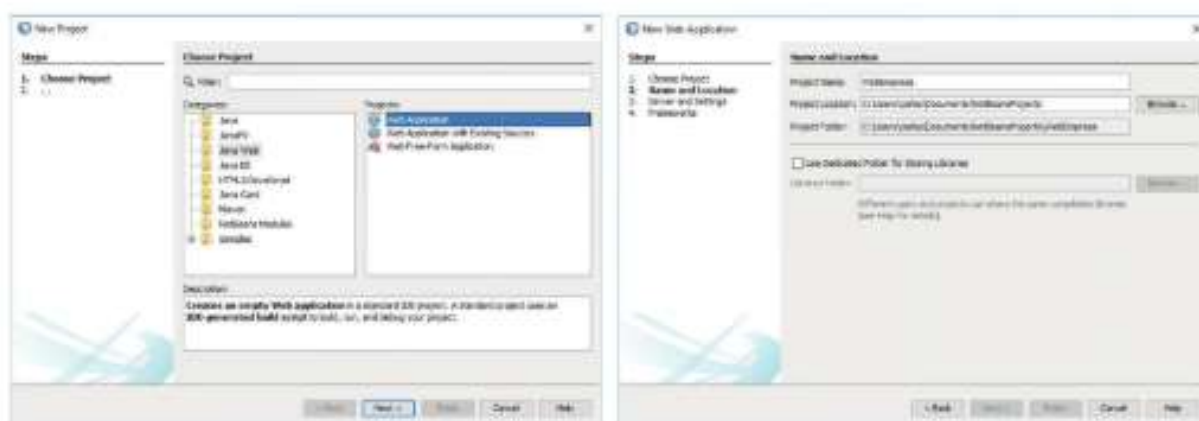


Figura 8.2
Creación de aplicación web con NetBeans

A continuación, es necesario especificar algunas opciones específicas de las aplicaciones web, y los *frameworks* que utilizar. Se selecciona Hibernate, que se utilizará para persistencia de los JavaBeans. El resto no hacen falta.



Figura 8.3

Opciones para la aplicación web y *frameworks* que se van a utilizar

Cuando se instala NetBeans se pueden instalar como servidores de aplicaciones Apache Tomcat, que incluye un contenedor de *servlets*, y GlassFish, que incluye soporte para EJB. Para esta aplicación no se necesitan EJB y basta con Apache Tomcat. Se puede elegir la versión de Java EE más reciente entre todas las disponibles.

El *context path* es la parte de la URL que viene después del servidor (nombre de *host* o IP y puerto, en su caso). Con él selecciona el servidor una aplicación de las que tiene desplegadas y le dirige la petición HTTP. Una vez obtenida la respuesta HTTP de la aplicación, la dirige al cliente. Como *context path* de esta aplicación se especifica **ADWebEmpresa**.

8.8.2. Persistencia de objetos con Hibernate

Es necesario hacer lo mismo que para los proyectos desarrollados en el capítulo anterior dedicado a ORM, a saber:

1. Crear una conexión con la base de datos **proyecto_orm**, con las mismas opciones que en el capítulo anterior dedicado a ORM. El nombre de la conexión será **proyecto_ORM**. Si ya está creada, se puede utilizar directamente.
2. Crear los ficheros **hibernate.cfg.xml** e **hibernate.reveng.xml** utilizando los asistentes de NetBeans para ello. Deben crearse en el paquete por defecto (**<default package>**).
3. Crear el fichero **HibernateUtil.java** con el asistente de NetBeans, en el paquete por defecto. En el capítulo anterior dedicado a ORM se vio que el fichero que se crea es para una versión antigua de Hibernate, y qué cambios hay que hacer en él.
4. Crear POJO para las tablas de la base de datos, con el asistente de NetBeans, en el paquete **ORM**.
5. En su caso, eliminar los **jar** para Hibernate incluidos por NetBeans, para sustituirlos por los de una versión reciente.

8.8.3. Creación del *servlet* controlador

Se crea el *servlet* controlador pulsando con el botón derecho sobre la aplicación y seleccionando "New ...", "Servlet ...". Se especifica su nombre (en este caso, **controlador**), y se configura el

servlet. Hay que marcar la opción para añadir esta información al descriptor de despliegue **web.xml**. Como patrón de url para el *servlet* se indica **/controlador**. El *servlet* se podrá invocar con **http://(host)/ADWebEmpresa/controlador**. **/ADWebEmpresa** indica al contenedor de *servlets* Tomcat la aplicación web, y **/controlador**, un *servlet* de la aplicación. Se indican los parámetros del *servlet* y sus valores por defecto. En este caso será solo uno con nombre **op** y valor por defecto nulo. El valor del parámetro **op** indica la acción que se va a realizar por el controlador, y un valor nulo indica que se debe ir a la página principal.

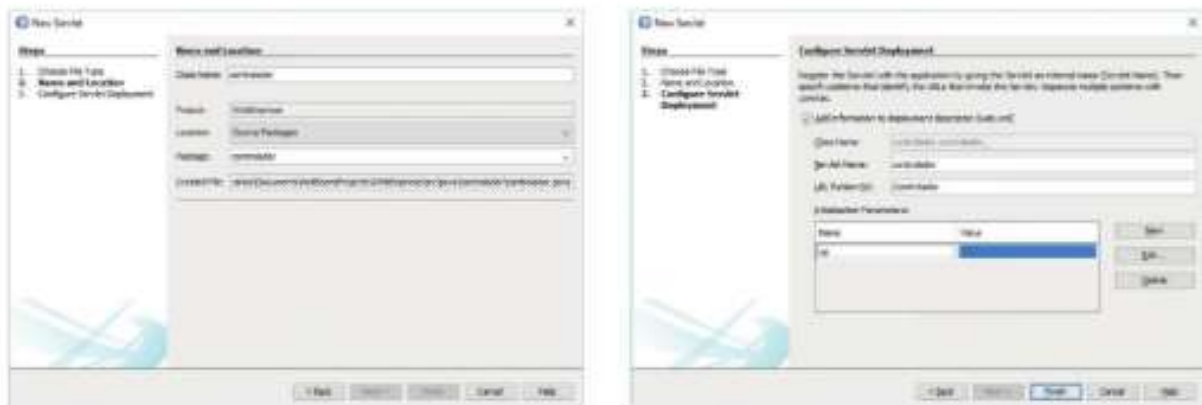


Figura 8.4
Creación y configuración del *servlet* controlador

A continuación, se muestra el contenido del descriptor de despliegue **web.xml**. En un elemento **<servlet>** se describe un *servlet*, y en **<init-param>**, los valores por defecto para cada parámetro (para el caso en que no se especifique ninguno en la petición HTTP con que se le invoca). En **<servlet-mapping>** se especifica la correspondencia entre patrones de URL y *servlets*. En los contenidos mostrados aquí se incluye el cambio, que habría que hacer ahora, de **index.html** por **controlador** en el elemento **<welcome-file>**, para que se invoque directamente el *servlet* **controlador**. El fichero **index.html** se puede borrar sin más.

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="3.1" xmlns="http://xmlns.jcp.org/xml/ns/
  javaee" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee http://xmlns.jcp.
  org/xml/ns/javaee/web-app_3_1.xsd">
  <servlet>
    <servlet-name>controlador</servlet-name>
    <servlet-class>controlador.controlador</servlet-class>
    <init-param>
      <param-name>op</param-name>
      <param-value></param-value>
    </init-param>
  </servlet>
  <servlet-mapping>
    <servlet-name>controlador</servlet-name>
    <url-pattern>/controlador</url-pattern>
  </servlet-mapping>
```

```

<session-config>
  <session-timeout>
    30
  </session-timeout>
</session-config>
<welcome-file-list>
  <welcome-file>controlador</welcome-file>
</welcome-file-list>
</web-app>

```

Después, se edita el código del *servlet* `controlador`, que debe quedar así:

```

// Servlet controlador (controlador.java)
package controlador;

import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.RequestDispatcher;

public class controlador extends HttpServlet {

    protected void processRequest(HttpServletRequest request,
        HttpServletResponse response) throws ServletException, IOException {

        String oper = request.getParameter("op");
        if(oper==null) oper = "";

        RequestDispatcher rd;

        switch(oper) {
            case "altaSede":
                response.sendRedirect("frmNuevaSede.jsp");
                break;
            case "insertSede":
                rd = request.getRequestDispatcher("procNuevaSede.jsp");
                rd.forward(request, response);
                break;
            case "muestraSede":
                rd = request.getRequestDispatcher("muestraSede.jsp");
                rd.forward(request, response);
                break;
            default:
                response.sendRedirect("home.jsp");
        }
    }

    (...)
}

```

El *servlet* `controlador` obtiene el valor del parámetro `op` de la petición HTTP y realiza la acción que se indica en él, que en general consistirá en pasar el control a un *servlet* o JSP, y redirigirle la petición HTTP. Si no se indica ninguna, se invoca la página inicial de la aplicación `home.jsp`, que se desarrollará en breve, al igual que los JSP a los que se hace referencia en este *servlet*.

8.8.4. Uso de JavaBeans asociado a formularios HTML con JSP

Esta aplicación hará uso extensivo del mecanismo disponible en JSP para utilizar JavaBeans para recopilar los datos enviados en los parámetros de una petición HTTP.

El siguiente código define un JavaBean (`jsp:useBean`) de nombre `sede` (`id="sede"`) y clase `ORM.Sede` (`class="ORM.Sede"`). Su ámbito es la petición HTTP, que invoca al JSP (`scope="request"`). Esto significa que solo está disponible para el propio JSP.

El valor de cualquier parámetro de la petición HTTP cuyo nombre coincida con el de cualquier atributo (`property="**"`) de este JavaBean (`name="sede"`) se asigna al atributo.

```
<jsp:useBean id="sede" scope="request" class="ORM.Sede"/>
<jsp:setProperty name="sede" property="**"/>
```

El JavaBean se obtiene mediante el siguiente código en Java:

```
ORM.Sede sedeBean = (ORM.Sede) request.getAttribute("sede");
```

Si no existe ningún parámetro de la petición HTTP cuyo nombre coincida con el de ningún atributo del JavaBean, entonces `sedeBean` toma valor `null`.

8.8.5. Creación de los JSP

Para crear un JSP se pulsa con el botón derecho del ratón sobre la aplicación y se selecciona la opción "New", "JSP ...". A continuación, se muestran los JSP de esta aplicación.

`home.jsp` muestra un listado de todas las sedes y un enlace para crear una nueva sede. Al lado de cada una se muestra un enlace para mostrar sus detalles. Todos estos enlaces dirigen al `servlet controlador` mediante peticiones HTTP de tipo POST, con los valores para los parámetros enviados desde el formulario `frm_muestra_sede`. Sería más sencillo emplear elementos `<a>` con la URL en el atributo `href` y la asignación de valores de los parámetros codificados en ella, pero esto generaría peticiones HTTP de tipo GET en las que serían visibles todos estos detalles en la URL mostrada por el navegador, lo que hay que evitar por seguridad. Para las clases de los JavaBeans, se ha implementado la persistencia con Hibernate, lo que hace muy sencillo recuperarlos de la base de datos y reflejar en ella los cambios sobre ellos.

```
<!-- home.jsp -->
<%%page import="java.util.List"%>
<%%page import="org.hibernate.Session"%>
<%%page import="org.hibernate.query.Query"%>

<%%page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>Empresa</title>
  </head>
```

```

<body>
  <form name="frm_muestra_sede" method="post" action="controlador">
    <input type="hidden" name="op" value="muestraSede">
    <input type="hidden" name="idSede">
    <table border="1">
      <tr><td colspan="2" align="center">Sedes [<a
        href="javascript:void(0)" onclick="javascript:document.
        frm_muestra_sede.op.value = 'altaSede';document.frm_muestra_
        sede.submit();">Nueva sede</a>]</td></tr>
    <%
    Session s = ORM.HibernateUtil.getSessionFactory().openSession();
    try {
      Query q = s.createQuery("FROM Sede ORDER BY
        nomSede,idSede").setReadOnly(true);
      List<ORM.Sede> listaSedes = (List<ORM.Sede>)
        q.getResultList();
      if (listaSedes.isEmpty()) {&#38;
        <tr><td colspan="2">No existen sedes</td></tr>
        <%
      } else {
        for (ORM.Sede unaSede: listaSedes) {&#38;
          <tr>
            <td><%=unaSede.getIdSede()&#38;</td>
            <td><a href="javascript:void(0)"
              onclick="javascript:document.frm_muestra_sede.
              idSede.value = '<%=unaSede.getIdSede()&#38;';
              document.frm_muestra_sede.submit();"><%=unaSede.
              getNomSede()&#38;</a></td>
          </tr>
          <%
        }
      }
    } catch (Exception e) {
      e.printStackTrace(System.err);
    }
    &#38;
  </table>
</form>
</body>
</html>

```

`frmNuevaSede.jsp` permite la creación de una nueva sede, y muestra un formulario para introducir sus datos, que envía los datos a `procNuevaSede.jsp` para crearla.

```

<!-- frmNuevaSede.jsp -->
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<jsp:useBean id="sede" scope="request" class="ORM.Sede"/>
<jsp:setProperty name="sede" property="*" />

<!DOCTYPE html>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>Alta de Sede</title>
  </head>

```

```

<body>
  <form method="post" action="controlador">
    <table>
      <tr>
        <td>
          <input type="hidden" name="op" value="insertSede"/>
          <input name="nomSede" required type="text" size="20"
            maxlength="20"/>
        </td>
      </tr>
      <tr>
        <td>
          <input type="submit" value="Crear"/>
          <input type="reset" name="cancelar" value="Cancelar"/>
        </td>
      </tr>
    </table>
  </form>
  <a href="controlador">Inicio</a>
</body>
</html>

```

`procNuevaSede.jsp` crea una nueva sede con los datos recogidos en el JavaBean llamado `sede`, que se hace persistente con el método `save()` de la interfaz `Session` de Hibernate.

```

<!-- // procNuevaSede.jsp -->
<%%page import="org.hibernate.Session"%>
<%%page import="org.hibernate.Transaction"%>
<%%page import="org.hibernate.query.Query"%>
<%%page contentType="text/html" pageEncoding="UTF-8"%>
<jsp:useBean id="sede" scope="request" class="ORM.Sede"/>
<jsp:setProperty name="sede" property="*/>
<!DOCTYPE html>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>Creación de sede</title>
  </head>
  <body>
    <%
      ORM.Sede sedeBean = (ORM.Sede) request.getAttribute("sede");
      if(sedeBean == null) {
    %>
      ERROR: no se proporcionaron datos de sede para crear.
    <%
      }
      else {
        Transaction t = null;
        try(Session s = ORM.HibernateUtil.getSessionFactory().openSession()) {
          t = s.beginTransaction();
          s.save(sedeBean);
          t.commit();
        }
    %>

```

```

Creada nueva sede: [<%=sedeBean.getIdSede()%>] <%=sedeBean.
    getNomSede()%><br/>
<a href="controlador">Volver</a>
<%
    } catch (Exception e) {
        e.printStackTrace(System.err);
        if (t != null) {
            t.rollback();
        }
    }
}>
</body>
</html>

```

Por último, `muestraSede.jsp` muestra los datos de una sede, incluyendo la lista de sus departamentos. Se usa un JavaBean de nombre `sede` en el que está disponible el identificador de la sede `idSede`. El resto de los datos se obtienen mediante el método `refresh()` de la interfaz `Session` de Hibernate.

```

<!-- // muestraSede.jsp -->
<%@page import="java.util.Iterator"%>
<%@page import="org.hibernate.Session"%>
<%@page import="org.hibernate.query.Query"%>
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<jsp:useBean id="sede" scope="request" class="ORM.Sede"/>
<jsp:setProperty name="sede" property="*" />
<%
    ORM.Sede sedeBean = (ORM.Sede) request.getAttribute("sede");
    if (sedeBean == null) {
%>
ERROR: no se especificó sede a mostrar.
<%
    } else {
        try (Session s = ORM.HibernateUtil.getSessionFactory().openSession()) {
            s.refresh(sedeBean);
%>
<!DOCTYPE html>
<html>
    <head>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
        <title>Sede <%=sedeBean.getIdSede()%> - <%=sedeBean.getNomSede()%></
            title>
    </head>
    <body>
        <p>Sede <%=sedeBean.getIdSede()%> - <%=sedeBean.getNomSede()%></p>
        <table border="1"><tr><td colspan="2" align="center">Departamentos</
            td></tr>
        <%
            if (sedeBean.getDepartamentos().isEmpty()) {%>
                <tr><td colspan="2">No existen departamentos en esta sede</td></tr>
                <%
            } else {

```

```

        Iterator itDeptos = sedeBean.getDepartamentos().iterator();
        while (itDeptos.hasNext()) {
            ORM.Departamento unDeppto = (ORM.Departamento) itDeptos.next();
            %>
            <tr>
                <td><%=unDeppto.getIdDeppto() %></td>
                <td><%=unDeppto.getNomDeppto() %></td>
            </tr>
            <%
            }
            %>
        </table>
    </body>
</html>
<%
    } catch (Exception e) {
        e.printStackTrace(System.err);
    }
    %>

```

El proyecto quedaría como se muestra en la figura 8.5. La vista está constituida por varias páginas JSP situadas bajo el apartado "Web Pages". El controlador consta de un único fichero de clases de Java dentro de un paquete `controlador`, y el modelo está constituido por varios POJO junto con sus ficheros de correspondencia de Hibernate, en el paquete `ORM`. En el proyecto están, además, los ficheros adicionales necesarios para Hibernate.

A continuación, se proponen varias actividades sucesivas para ampliar la aplicación anterior para convertirla en una aplicación CRUD completa, tanto para sedes como para departamentos y empleados.

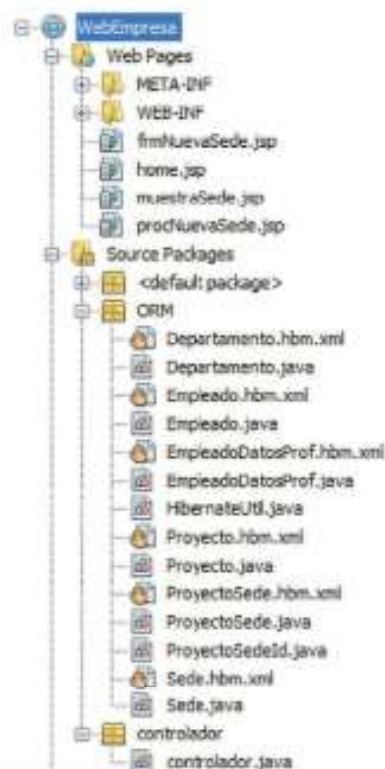


Figura 8.5
Proyecto para aplicación web

Actividad propuesta 8.1



Completa la aplicación anterior para que permita:

- Borrar sedes.
- Modificar los datos de una sede.
- Crear, ver, modificar y borrar departamentos. No olvides que los departamentos siempre pertenecen a una sede. Al mostrar los datos de un departamento se debe mostrar su lista de empleados, de manera análoga a como en el programa de ejemplo, al mostrar los datos de una sede se muestra su lista de departamentos.
- Crear, ver, modificar y borrar empleados. No olvides que los empleados siempre están adscritos a un departamento.
- Buscar empleados por DNI y por nombre.

8.9. Enterprise JavaBeans

El modelo de componentes de la plataforma Java EE está basado en Enterprise JavaBeans o EJB, que se despliegan en un contenedor de EJB, cuya existencia es obligatoria en un servidor para Java EE. Existen distintos tipos de EJB, pero en esta aplicación solo se utilizarán *session beans* de tipo *stateless*, es decir, EJB de sesión sin estado.

NetBeans incluye el servidor GlassFish para Java EE, que proporciona un contenedor de EJB. Tomcat proporciona un contenedor de *servlets*, pero no de EJB.

Se explicará lo mínimo indispensable para poder entender el uso de EJB en una aplicación para Java EE web con idéntica funcionalidad que la anterior para Java SE. Java EE ofrece algunas ventajas para el desarrollo de aplicaciones, a cambio de la necesidad de disponer de un relativamente complejo servidor de aplicaciones instalado y convenientemente configurado. Entre ellas:

- a) Disponibilidad de implementaciones de las API JPA para persistencia de objetos y JTA para transacciones, como parte de la especificación Java EE.
- b) Sincronización en el acceso a los datos. Solo un hilo de ejecución o hebra (*thread*) puede acceder a la vez a un *session bean*. Esto evita problemas que podrían darse si más de un *thread* accede simultáneamente a la interfaz `EntityManager` de JPA, que no es *thread-safe*, es decir, que no garantiza que funcionará correctamente en ese caso. Los *session beans* hacen innecesario el uso de métodos de sincronización de los que dispone Java, como la palabra clave `synchronized`.
- c) Disponibilidad del mecanismo CDI (*context and dependency injection*) para acceso directo y sencillo a servicios básicos de la plataforma Java EE. CDI se introdujo en Java EE 6, y se basa en el uso de anotaciones de Java para acceder a servicios tales como persistencia (con JPA) y transacciones distribuidas (con JTA).

Así pues, los *session beans* utilizan JPA para persistencia de objetos y garantizan la integridad de los datos mediante el uso de transacciones proporcionadas por JTA. Pero, además —y esta es una importante ventaja con respecto a la plataforma Java SE—, proporcionan sincronización en el acceso a dichos objetos persistentes.

En los *session beans* se implementan métodos de muy alto nivel para el acceso a los datos. Se puede decir que los *session beans* sin estado (*stateless*) se utilizan como objetos DAO. El patrón de diseño DAO consiste en el acceso a datos mediante clases que ofrecen una interfaz de alto nivel e independiente del tipo de almacenamiento de datos. En este caso particular, se utiliza JPA para el acceso a los datos que están en una base de datos relacional, pero esto es solo un detalle de la implementación, que no se refleja para nada en la interfaz de las clases DAO.

8.10. Desarrollo de una aplicación web MVC para Java EE con EJB

El planteamiento es muy similar al visto anteriormente para Java SE, pero se utilizan EJB de sesión sin estado (*stateless session EJB*) como DAO. Estos proporcionan métodos de muy alto nivel y próximos a la lógica de negocios. Se puede decir que los *session beans* permiten un acceso sencillo (mediante métodos de alto nivel), sincronizado (mediante el mecanismo de sincronización intrínseco de los *session beans*) y seguro (mediante el uso de transacciones) a las clases de entidad o *entity classes*, que son clases persistentes mediante JPA.

Se puede utilizar cualquier implementación de JPA de las disponibles. Se utiliza la implementación de Hibernate, no la de EclipseLink que se propone por defecto.

8.10.1. Creación de la aplicación web

Se crea también con “File”, “New Project”, “Java Web”, “Web Application”. Como nombre de aplicación se especifica, por ejemplo, **WebEmpresaEJB**.

Después, se especifica servidor. Para utilizar EJB hace falta un servidor con soporte para EJB como GlassFish, que viene integrado con NetBeans y se puede instalar con él. Como *context path* se puede dejar el mismo nombre del proyecto, que se propone por defecto. En el siguiente diálogo no se selecciona ningún *framework*, tampoco Hibernate, aunque, como se ha comentado, se utilizará la implementación de JPA que proporciona Hibernate, pero esto corresponde a un paso posterior.

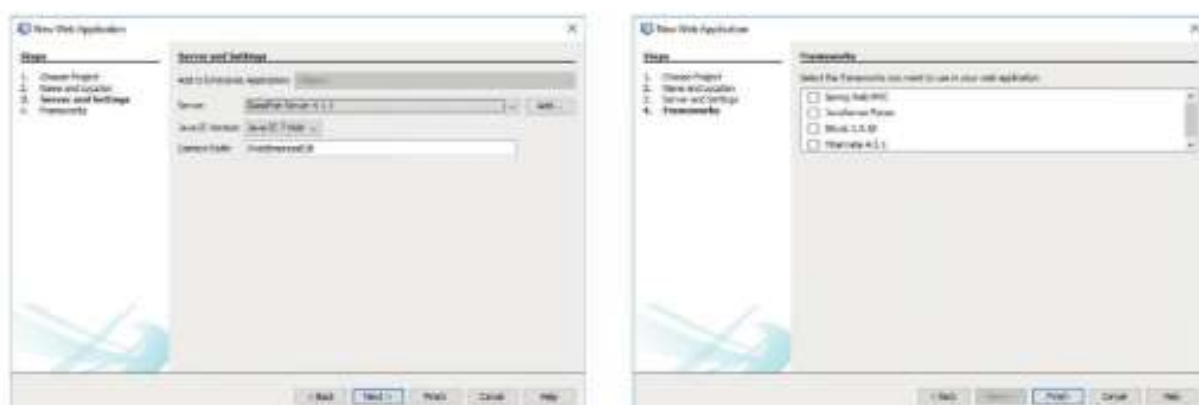


Figura 8.6
Creación de la aplicación web y selección de *frameworks*

8.10.2. Creación de la unidad de persistencia

La unidad de persistencia se encargará de la persistencia de un conjunto de clases de entidad (*entity classes*), utilizando JPA. Las clases de entidad se asociarán a ella en un paso posterior. Se pulsa con el botón derecho del ratón sobre el proyecto, se selecciona la opción “New...”, “Other...”, “Persistence”, “Persistence Unit”, y se indica **WebEmpresaEJBPU** como nombre. Se puede elegir una de las implementaciones disponibles de JPA. La opción por defecto es EclipseLink, pero se ha elegido Hibernate para explorar otra faceta de Hibernate, a saber, su implementación de JPA (en el capítulo dedicado a ORM se utilizó su API nativa y no su implementación de JPA). Por supuesto, se quiere utilizar Java Transaction API (JTA). Como “Table generation strategy” se elige “None” (las tablas ya existen, y las clases de entidad se generarán a partir de ellas). Se crea un nuevo *Data Source* **WebEmpresaEJBDS** basado en la conexión JDBC **proyecto ORM** para la base de datos **proyecto_orm**, creadas en el capítulo previo dedicado a ORM. Si no existieran, se pueden crear como se explicó en ese capítulo.

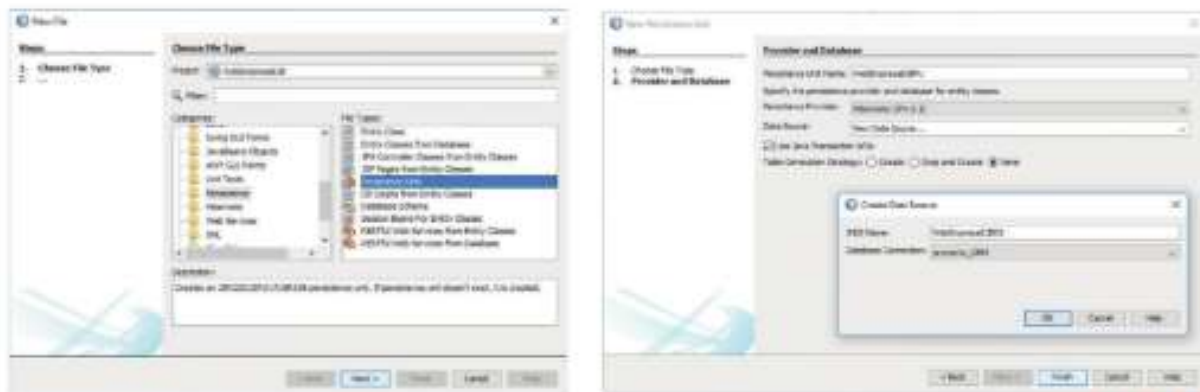


Figura 8.7

Creación de la unidad de persistencia

Con esto, la definición de la unidad de persistencia se graba en `persistence.xml`. En ella está incluido el nombre de la fuente de datos.

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence version="2.1" xmlns="http://xmlns.jcp.org/xml/ns/
  persistence" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/persistence http://xmlns.
  jcp.org/xml/ns/persistence/persistence_2_1.xsd">
  <persistence-unit name="WebEmpresaEJBPU" transaction-type="JTA">
    <jta-data-source>java:app/WebEmpresaEJBDS</jta-data-source>
    <exclude-unlisted-classes>false</exclude-unlisted-classes>
    <properties/>
  </persistence-unit>
</persistence>
```

Con esto, además, se añaden al proyecto las bibliotecas de Hibernate. Estas se pueden sustituir más adelante por las de una versión más reciente.

8.10.3. Creación de las clases de entidad

Las clases de entidad se crean a partir de las tablas, y quedarán asociadas a la unidad de persistencia. Para ello se pulsa con el botón derecho sobre el proyecto, se elige "New...", "Other...", "Persistence", "Entity Classes from Database", y se seleccionan todas las tablas.

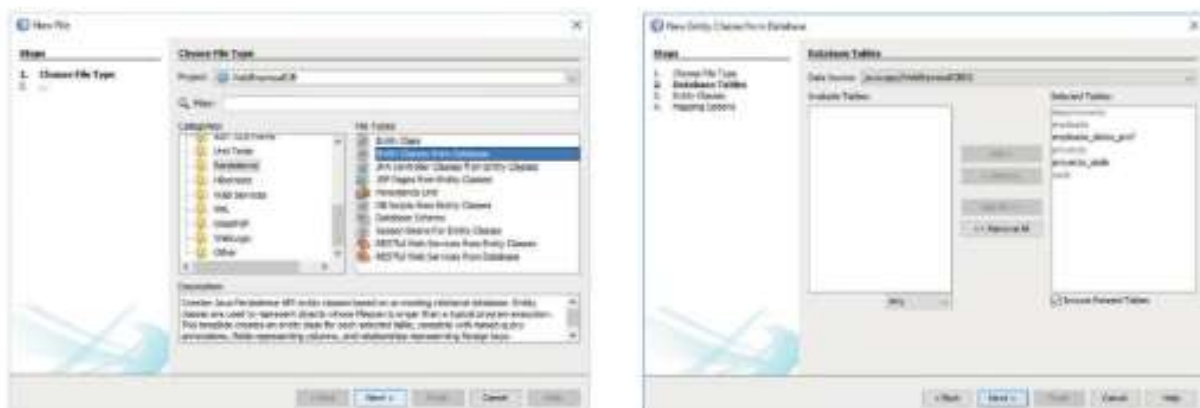


Figura 8.8

Selección de tablas para unidad de persistencia

Hay que especificar un paquete para las clases de entidad, que constituirán el modelo (M) de la aplicación MVC. Se puede especificar `com.miempresa.modelo`. No se necesitan anotaciones para JAXB. Como "Association Fetch" se selecciona "lazy", cuyo significado se vio en el capítulo dedicado a ORM.

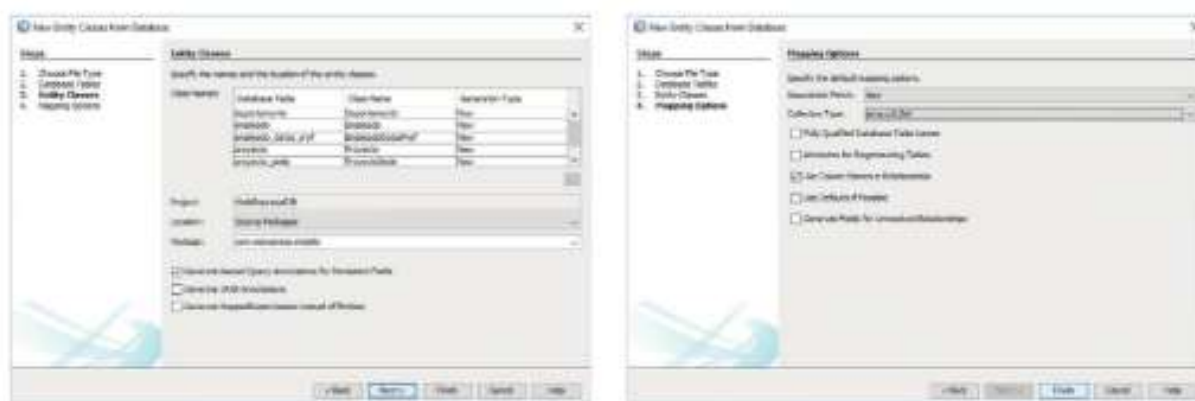


Figura 8.9
Opciones para la creación de las clases de entidad

Con esto están creadas las clases de entidad dentro del paquete `com.miempresa.modelo`. Estas clases son JavaBeans con anotaciones de JPA para implementar su persistencia.

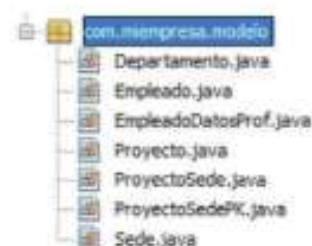


Figura 8.10
Clases de entidad

8.10.4. Creación de los EJB de sesión para las clases de entidad

Se crean pulsando con el botón derecho del ratón sobre el proyecto, y seleccionando "New...", "Other...", "Enterprise Java Beans", "Session Beans for Entity Classes". Se seleccionan todas las clases de entidad mostradas. Se selecciona la opción de crear interfaz local y, como paquete para los EJB de sesión, se selecciona `com.miempresa.dao`.

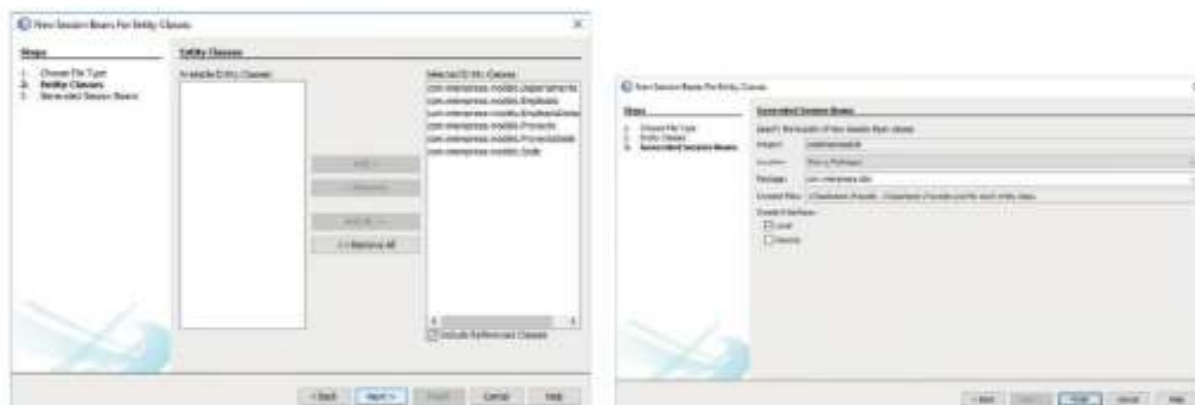


Figura 8.11
Creación de Session Beans sin estado para clases de entidad

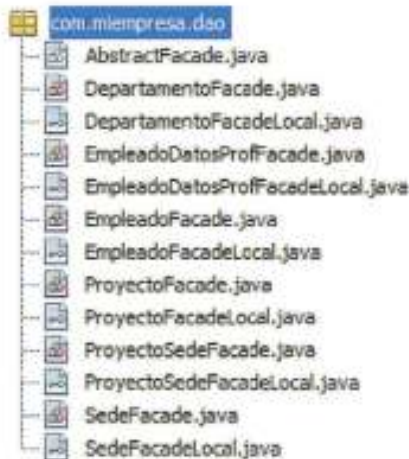


Figura 8.12
EJB de sesión

Con esto se crean los EJB de sesión para las clases de entidad. Por cada una se crea un *session bean* que consta de una interfaz local (etiquetada con `@Local`) y una clase que la implementa (etiquetada con `@Stateless` para indicar que se trata de un EJB de sesión sin estado). Por ejemplo, para la clase de entidad `Sede` se crea la interfaz local `SedeFacadeLocal` y la clase `SedeFacade` que la implementa. La interfaz local incluye, entre otras cosas, todos los métodos necesarios para una aplicación CRUD, que se implementan en la clase mediante métodos de JPA, según se explica en el cuadro 8.3.

CUADRO 8.3
Implementación con JPA de los métodos de la interfaz local

	Método	Método de JPA (EntityManager)
C(reate)	<code>create(T entidad)</code>	<code>persist(entidad)</code>
R(ead)	<code>find(Object id)</code>	<code>find(id)</code>
U(pdate)	<code>edit(T entidad)</code>	<code>merge(entidad)</code>
D(elte)	<code>remove(T entidad)</code>	<code>remove(entidad)</code>

Más en detalle, `SedeFacade` extiende la clase abstracta `AbstractFacade<Sede>`, que es la que proporciona la implementación de los métodos de `SedeFacadeLocal`, y `SedeFacade` solo añade la implementación del método `getEntityManager()`. De hecho, justo este método se debe cambiar, para dejarlo como sigue. Se indican los cambios realizados en negrita, con respecto al código generado por NetBeans. El resto de las clases *Facade* deben cambiarse de igual manera.

```
// SedeFacade.java
package com.miempresa.dao;

import com.miempresa.modelo.Sede;
import javax.ejb.Stateless;
import javax.persistence.EntityManager;
// import javax.persistence.PersistenceContext; // Eliminado
import javax.persistence.Persistence; // Añadido

@Stateless
public class SedeFacade extends AbstractFacade<Sede> implements
    SedeFacadeLocal {

// @PersistenceContext(unitName = "WebEmpresaEJBPU") // Eliminado
    private EntityManager em;

    @Override
    protected EntityManager getEntityManager() {
        em = Persistence.createEntityManagerFactory("WebEmpresaEJBPU").
            createEntityManager(); // Añadido
    }
}
```

```

        return em;
    }

    public SedeFacade() {
        super(Sede.class);
    }
}

```

Facade es una transcripción del francés *Façade* y significa que estas clases actúan como una fachada que oculta los detalles de las clases de entidad, y ofrecen una interfaz de más alto nivel para el acceso a los datos. Pero las clases *Facade* son solo un paso en esa dirección, y no se pueden considerar verdaderas clases DAO. Se crea *GestorSedes* como clase DAO para la clase de entidad *Sede*, basada en *SedeFacade*, pero que proporciona métodos de más alto nivel para el acceso a los datos. A estos métodos no se les pasan objetos de clases de entidad, sino tipos de datos sencillos. Por ejemplo, *createSede(String nomSede)* crea una sede con el nombre dado. Las operaciones realizadas por estos métodos se harán, además, dentro de una transacción. Esta clase se crea como un EJB de sesión, también en el paquete *com.miempresa.dao*, pero para ella no se crea una interfaz local. Se puede crear la clase pulsando con el botón derecho del ratón sobre el proyecto y seleccionando la opción “New...”, “Other...”, “Enterprise Java Beans”, “Session Bean”. Las definiciones de sus métodos se pueden crear como se indica en los comentarios incluidos en el código generado automáticamente (Right-click in editor and choose “Insert Code > Add Business Method”), pero esto no aporta realmente mucho valor ni ahorra mucho trabajo.

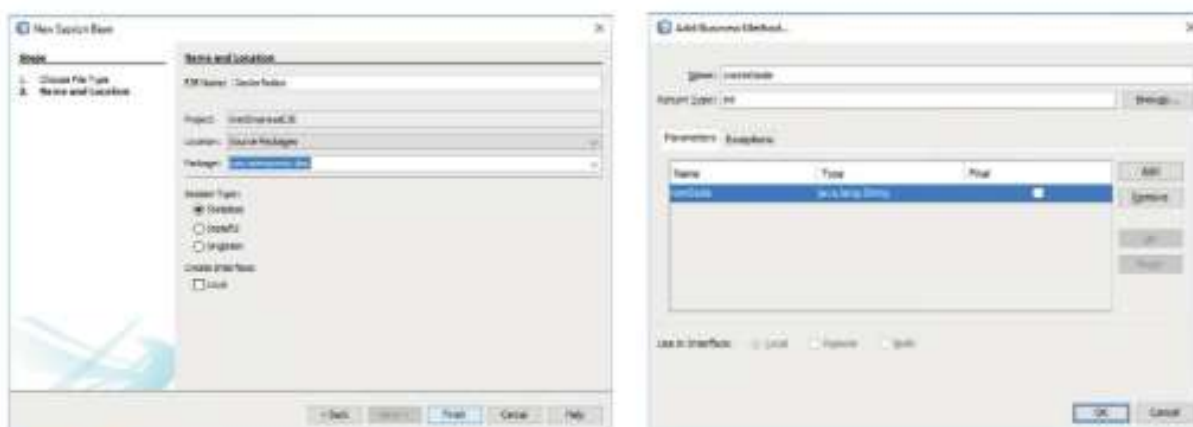


Figura 8.13
Creación de EJB de sesión sin estado que funcionan como clases DAO

El código de la clase *GestorSedes*, incluyendo el método *createSede*, es el siguiente:

```

// GestorSedes.java
package com.miempresa.dao;

import javax.ejb.Stateless;
import javax.persistence.EntityManager;
import javax.persistence.EntityTransaction;

import com.miempresa.modelo.Sede;

@Stateless
public class GestorSedes {

```

```

private final SedeFacade sedeFacade = new SedeFacade();

// Add business logic below. (Right-click in editor and choose
// "Insert Code > Add Business Method")

public int createSede(String nomSede) {
    Sede entidadSede = new Sede();
    entidadSede.setNomSede(nomSede);
    EntityManager em = sedeFacade.getEntityManager();
    EntityTransaction tx = em.getTransaction();
    try {
        tx.begin();
        sedeFacade.create(entidadSede);
        tx.commit();
        return entidadSede.getIdSede();
    } catch (Exception e) {
        try {
            tx.rollback();
        } catch (Exception e1) {
            e1.printStackTrace();
        }
        e.printStackTrace();
        return -1;
    }
}
}
}

```

8.10.5. Creación del *servlet* controlador

Ahora se crea el *servlet* controlador. En lugar de utilizar un único *servlet* controlador, se puede utilizar uno por cada clase de entidad. Para esta aplicación se creará solo el *servlet* controlador para sedes, que hará las mismas funciones que el creado para la aplicación anterior. Se crea pulsando con el botón derecho del ratón sobre el proyecto, con la opción "New...", "Other...", "Web", "Servlet". Como nombre del *servlet* se indica **ServletSede**, y como nombre del paquete, **com.miempresa.controlador**.

Se selecciona la opción de añadir información al descriptor de despliegue (**web.xml**), y se indica el patrón de URL asociado a él. Se añade un parámetro **op** con valor por defecto sin especificar (cadena vacía). En resumen, como para la aplicación anterior.

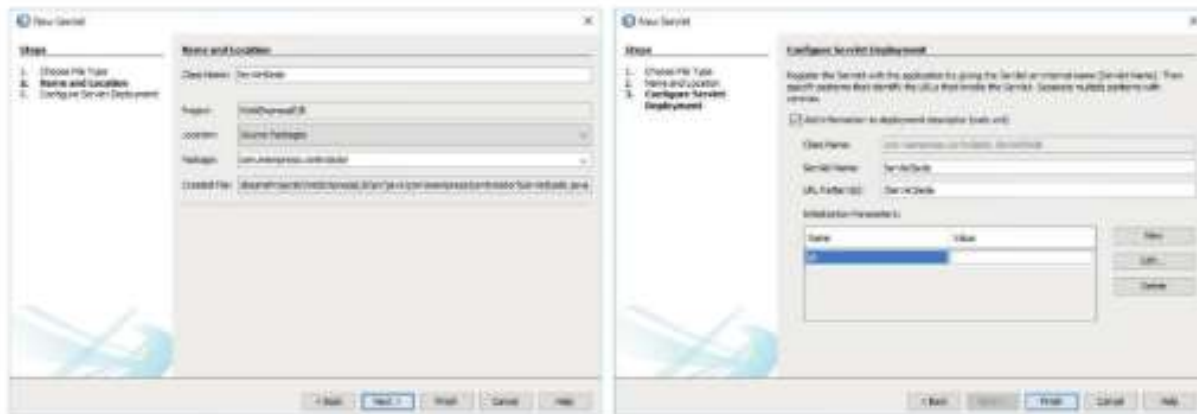


Figura 8.14
Creación del *servlet* controlador

El código del *servlet* es prácticamente idéntico al de la aplicación anterior:

```
// ServletSede.java
package com.miempresa.controlador;
import java.io.IOException;
import javax.servlet.RequestDispatcher;
import javax.servlet.annotation.WebServlet;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

@WebServlet(name="ServletSede")
public class ServletSede extends HttpServlet {
    protected void processRequest(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {
        String oper = request.getParameter("op");
        if(oper==null) oper = "";
        RequestDispatcher rd;
        switch(oper) {
            case "altaSede":
                response.sendRedirect("frmNuevaSede.jsp");
                break;
            case "insertSede":
                rd = request.getRequestDispatcher("procNuevaSede.jsp");
                rd.forward(request, response);
                break;
            case "muestraSede":
                rd = request.getRequestDispatcher("muestraSede.jsp");
                rd.forward(request, response);
                break;
            default:
                response.sendRedirect("home.jsp");
        }
    }
    (...)
}
```

Actividades propuestas



- 8.2.** Completa la clase `GestorSedes` con métodos `retrieveAll`, `retrieveSede`, `updateSede` y `deleteSede`. `retrieveAll` debe devolver un `List<Sede>` con todas las sedes. `retrieveSede` debe devolver un objeto de la clase `Sede` a partir de su identificador. `updateSede` permitirá cambiar los datos de la sede (ahora mismo, solo el nombre), y `deleteSede` borrará la sede. La sede se indicará a `updateSede` y `deleteSede` con su identificador numérico.

- 8.3. Crea nuevas clases DAO `GestorDepartamentos`, `gestorProyectos` y `gestorEmpleados` tomando como modelo `GestorSedes`. En general, tendrán parámetros adicionales. Por ejemplo, al crear un departamento debe indicarse la sede a la que pertenece, mediante el identificador numérico de la sede.
- 8.4. Amplia la aplicación web de manera similar a como se propuso en actividades anteriores para la anterior aplicación, de manera que se convierta en una aplicación CRUD para sedes, departamentos y empleados. Deben usarse las clases DAO `GestorSedes`, `GestorDepartamentos`, `gestorProyectos` y `gestorEmpleados`.
- 8.5. Añade métodos a las clases DAO anteriores para gestionar las relaciones entre ellas. Por ejemplo, la clase `GestorDepartamentos` puede tener un método para reasignar un empleado a otro departamento. También debe ser posible asignar un proyecto a un departamento si no está asignado a ninguno, o reasignarlo a otro departamento si está asignado a alguno. Esta es una actividad abierta, en la que no se trata tanto de dar con la solución correcta (hay más de una posible), sino de proponer una solución completa y con un planteamiento consistente, y de justificar dicho planteamiento y sus posibles ventajas e inconvenientes frente a otros posibles.
- 8.6. Amplia la aplicación web para utilizar los métodos creados para la actividad propuesta anterior.
- 8.7. ¿Qué cambios hay en este *servlet* controlador con respecto al de la anterior aplicación?

8.10.6. Configuración básica inicial de la aplicación web

Se trata de que, al invocar la aplicación, se invoque el *servlet* controlador para sedes. Esto se consigue mediante el siguiente cambio en `web.xml`. El fichero `index.html` se puede borrar sin más.

<code>web.xml</code>	<pre><?xml version="1.0" encoding="UTF-8"?> <web-app (<-)> (<-> <welcome-file-list> <welcome-file>ServletSede</welcome-file> </welcome-file-list>) </web-app></pre>
----------------------	---

8.10.7. Creación de los JSP

Los JSP son muy similares a los de la anterior aplicación, pero algo más sencillos, porque no tienen que ocuparse de los detalles de persistencia y las transacciones. De eso se encargan los métodos de los DAO implementados como *session beans*. Se muestran en negrita los cambios más significativos.

Los JSP que se muestran como ejemplo siguen utilizando la clase `FacadeSede`. Debería utilizarse en su lugar la clase DAO `GestorSedes`. Los cambios necesarios para ello se propondrán como ejercicios. Para llevarlos a cabo será necesario utilizar, a su vez, los nuevos métodos de `GestorSedes` que se propusieron en ejercicios anteriores.

```

<!-- home.jsp -->
<%@page import="java.util.List"%>
<%@page import="com.miempresa.dao.SedeFacade"%>
<%@page import="com.miempresa.modelo.Sede"%>

<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>Empresa</title>
</head>
<body>
<form name="frm_muestra_sede" method="post" action="ServletSede">
<input type="hidden" name="op" value="muestraSede">
<input type="hidden" name="idSede">
<table border="1">
<tr><td colspan="2" align="center">Sedes [<a
href="javascript:void(0)" onclick="javascript:document.frm_
muestra_sede.op.value='altaSede';document.frm_muestra_sede.
submit();">Nueva sede</a>]</td></tr>
<%
try {
SedeFacade daoSede = new SedeFacade();
List<Sede> listaSedes = daoSede.findAll();
for (Sede unaSede: listaSedes) {%>
<tr>
<td><%=unaSede.getIdSede()%></td>
<td><a href="javascript:void(0)"
onclick="javascript:document.frm_muestra_sede.idSede.
value='<%=unaSede.getIdSede()%>';document.frm_muestra_
sede.submit();"><%=unaSede.getNomSede()%></a></td>
</tr>
}
} catch (Exception e) {
e.printStackTrace(System.err);
}
%>
</table>
</form>
</body>
</html>

```

Actividad propuesta 8.8



Cambia el JSP anterior para utilizar el método `retrieveAll` de `GestorSedes` en lugar del método `findAll` de `SedeFacade`.

`frmNuevaSede.jsp` es casi idéntico al de la anterior aplicación, solo cambia el nombre del *servlet* controlador.

```

<!-- frmNuevaSede.jsp -->
<%@page contentType="text/html" pageEncoding="UTF-8"%>

<jsp:useBean id="sede" scope="request" class="com.miempresa.modelo.Sede"/>
<jsp:setProperty name="sede" property="*" />
<!DOCTYPE html>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>Alta de Sede</title>
  </head>
  <body>
    <form method="post" action="ServletSede">
      <table>
        <tr>
          <td>
            <input type="hidden" name="op" value="insertSede"/>
            <input name="nomSede" required type="text" size="20"
              maxlength="20" />
          </td>
        </tr>
        <tr>
          <td>
            <input type="submit" value="Crear"/>
            <input type="reset" name="cancelar" value="Cancelar"/>
          </td>
        </tr>
      </table>
    </form>
    <a href="ServletSede">Inicio</a>
  </body>
</html>

```

`procNuevaSede.jsp` es más sencillo que en la anterior aplicación gracias a la clase DAO `GestorSedes`.

```

<!-- procNuevaSede.jsp -->
<%@page import="com.miempresa.modelo.Sede"%>
<%@page import="com.miempresa.dao.GestorSedes"%>

<%@page contentType="text/html" pageEncoding="UTF-8"%>

<jsp:useBean id="sede" scope="request" class="com.miempresa.modelo.Sede"/>
<jsp:setProperty name="sede" property="*" />
<!DOCTYPE html>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>Creación de sede</title>
  </head>
  <body>
    <%
      Sede sedeBean = (Sede) request.getAttribute("sede");
      if(sedeBean == null) {
    %>

```

```

ERROR: no se proporcionaron datos de sede para crear.
<%
    }
    else {
        try {
            GestorSedes gestorSedes = new GestorSedes();
            gestorSedes.createSede(sedeBean.getNomSede());
        } catch (Exception e) {
            e.printStackTrace(System.err);
        }
    }
}
</body>
</html>

```

Por último, `muestraSede.jsp` es casi idéntico al de la anterior aplicación.

```

<!-- muestraSede.jsp -->
<%@page import="com.miempresa.dao.SedeFacade"%>
<%@page import="com.miempresa.modelo.Sede"%>
<%@page import="com.miempresa.modelo.Departamento"%>
<%@page import="java.util.Iterator"%>
<%@page contentType="text/html" pageEncoding="UTF-8"%>

<jsp:useBean id="sede" scope="request" class="com.miempresa.modelo.Sede"/>
<jsp:setProperty name="sede" property="*" />

<%
    Sede sedeBean = (Sede) request.getAttribute("sede");
    if (sedeBean == null) {
        ERROR: no se especificó sede a mostrar.
    } else {
        try {
            SedeFacade daoSede = new SedeFacade();
            sedeBean = daoSede.find(sedeBean.getIdSede());
        }
    }
<|DOCTYPE html>
<html>
    <head>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
        <title>Sede <%=sedeBean.getIdSede()%> - <%=sedeBean.getNomSede()%></title>
    </head>
    <body>
        <p>Sede <%=sedeBean.getIdSede()%> - <%=sedeBean.getNomSede()%></p>
        <table border="1">

```

```

<tr>
  <td colspan="2" align="center">Departamentos</td></tr>
  <%
    if (sedeBean.getDepartamentoSet().isEmpty()) {%>
<tr><td colspan="2">No existen departamentos en esta sede</td></tr>
  <%
    } else {
      Iterator itDeptos = sedeBean.getDepartamentoSet().iterator();
      while (itDeptos.hasNext()) {
        Departamento unDeppto = (Departamento) itDeptos.next();
        %>
<tr>
  <td><%=unDeppto.getIdDeppto() %></td>
  <td><%=unDeppto.getNomDeppto() %></a></td>
</tr>
  <%
    }
  }
  %>
</table>
</body>
</html>
<%
  } catch (Exception e) {
    e.printStackTrace(System.err);
  }
}
%>

```



Actividad propuesta 8.9

Cambia el JSP anterior para utilizar el método `retrieveSede` de `GestorSedes` en lugar del método `find` de `SedeFacade`.

8.10.8. Incluir los ficheros jar de una versión reciente de Hibernate

En pasos anteriores se han añadido al proyecto las bibliotecas de Hibernate. Antes de compilar y desplegar la aplicación, se pueden sustituir por los de una versión más reciente, como ya se ha hecho en anteriores proyectos.

8.10.9. Despliegue de la aplicación

Antes de desplegar la aplicación, se compila mediante la opción "Clean and Build" mostrada al pulsar con el botón derecho del ratón sobre la aplicación.

La vista "Files" (ficheros) es un buen lugar para entender los contenidos de la aplicación web una vez construida y lista para su despliegue. El fichero de la aplicación está en `build`, y es `WebEmpresaEJB.war`. Se pueden ver sus contenidos. En el paquete por defecto están los

JSP. En otros paquetes están las clases de sus distintos componentes. Se ven los ficheros `persistence.xml`, `glassfish-resources.xml`, y `web.xml`.

En `WEB-INF.lib` se ven los ficheros de biblioteca (jar) de esta aplicación, entre ellos los de Hibernate.

Durante el despliegue de la aplicación se pueden producir errores muy diversos. Cuando se despliega un módulo de EJB, se crean e inician todos los recursos asociados a él, entre ellos la unidad de persistencia y los *pools* de conexiones para la base de datos. Hay muchas cosas que podrían fallar, y entonces el módulo no se desplegaría.

La aplicación se despliega con la opción "Deploy", que aparece al pulsar con el botón derecho del ratón sobre el proyecto. Si hubiera cualquier error que impida el despliegue de la aplicación, hay que consultar los *logs* del servidor de aplicaciones (GlassFish), lo que se puede hacer desde el propio NetBeans.

GlassFish viene con unos cuantos *drivers* JDBC, y NetBeans establece automáticamente las configuraciones necesarias para utilizarlos. Si se necesitara utilizar un *driver* distinto, no hay que incluirlo como biblioteca (*library*) del proyecto, sino que hay que copiarlo manualmente en los directorios del servidor de aplicaciones GlassFish, porque este necesita establecer una conexión con el *DataSource* antes de desplegar la aplicación. Además, hay que establecer manualmente las opciones de configuración para dicho *driver*. A continuación, se explica cómo hacer esto para el *driver* JDBC de MySQL 8.0. Para otros *drivers*, habrá que consultar la documentación del *driver* en cuestión o de GlassFish.

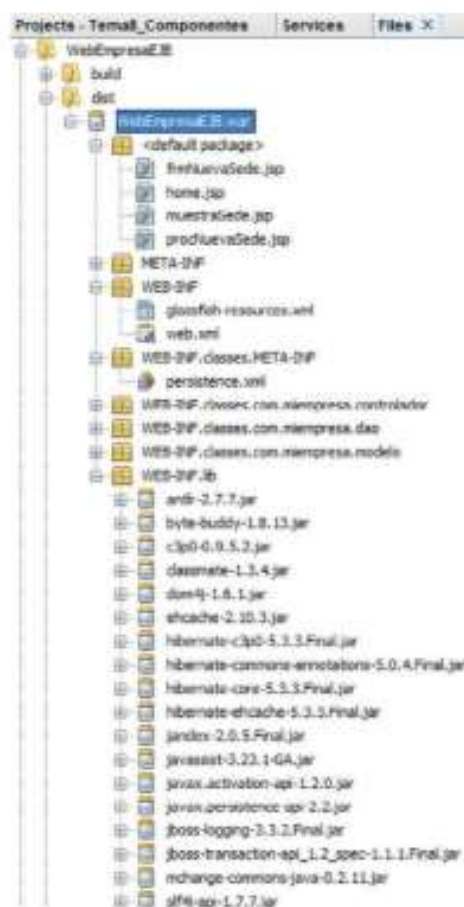


Figura 8.15
Contenidos de paquete war



PARA SABER MÁS

Configuración para un *driver* JDBC, por ejemplo, el de MySQL 8.0

El uso de un *driver* JDBC distinto a los que vienen incluidos con GlassFish requiere añadir a la instalación de GlassFish el fichero jar del *driver* y algunos cambios de configuración. El fichero jar hay que copiarlo en `glassfish/lib` o, según el caso, en `domains/domains1/lib`. El directorio de instalación de GlassFish depende del sistema operativo. En Linux es `~/glassfish-(versión)`, y en Windows es `C:\Archivos de Programa\glassfish-(versión)`.

Para utilizar el *driver* de MySQL 8.0, por ejemplo, hay que copiar en uno de estos directorios el fichero `mysql-connector-java-8.0.11.jar`. Además, hay que hacer los cambios que se indican a continuación en el fichero `glassfish-resources.xml`, que contiene configuraciones de la aplicación específicas para GlassFish. A saber, hay que indicar el nombre de dos clases necesarias para poner en marcha el *pool* de conexiones. Pero, además, se produce un error al intentar establecer una conexión segura, mediante SSL, con MySQL 8.0.

Ante eso se pueden hacer dos cosas: deshabilitar la conexión segura o hacer los cambios necesarios para establecer correctamente una conexión segura. La cuestión de cómo establecer una conexión segura con MySQL 8.0 queda fuera del alcance de este libro y se opta por la primera opción, la más sencilla.

```
<jdbc-connection-pool (...)  
datasource-classname="com.mysql.jdbc.jdbc2.optional.MysqlDataSource"  
(...)>  
  <property name="URL" value="...&useSSL=false"/>  
  <property name="driverClass" value="com.mysql.jdbc.Driver"/>  
</jdbc-connection-pool>
```

Después de estos cambios de configuración, hay que recompilar la aplicación y volver a desplegarla.

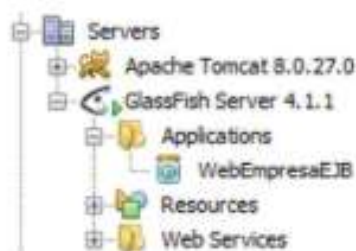


Figura 8.16
Aplicación web desplegada
en GlassFish

Una vez desplegada la aplicación, se puede ver bajo GlassFish (figura 8.16).

8.10.10. Solución de errores adicionales en tiempo de ejecución

Todavía se pueden producir algunos errores en el momento de ejecutar la aplicación. Es necesario leer atentamente los logs de GlassFish e intentar comprender la causa de los errores que se pueden producir y buscar solución para ellos.

Recurso digital



En el anexo web 8.1, disponible en www.sintesis.com y accesible con el código indicado en la primera página del libro, encontrarás más información sobre errores.

TOMA NOTA



La correcta instalación y configuración de un servidor de aplicaciones para Java EE, y el despliegue de aplicaciones en él, pueden ser tareas complejas. Parte de estas tareas, y en concreto la puesta a punto de los descriptores de despliegue y demás ficheros de configuración, corresponden más bien a un perfil profesional de administrador de sistemas que a uno de desarrollador, al menos si se trata de desplegar la aplicación en un entorno de producción o de preproducción (*staging*). Un perfil profesional de este tipo dará normalmente soporte a los desarrolladores para la correcta puesta a punto de los servidores de aplicaciones empleados para desarrollo y prueba de aplicaciones.

Resumen

- Los componentes de *software* facilitan el desarrollo de sistemas complejos mediante un diseño modular que permite reducir la complejidad global del sistema. Los componentes de *software* se pueden desarrollar y probar de manera independiente, y se distribuyen como paquetes independientes.
- En torno al lenguaje Java se ha desarrollado una plataforma de computación, de la que existen dos versiones o variantes: Java SE y Java EE. La especificación Java EE incluye, además de una biblioteca de clases ampliada para dar soporte a aplicaciones empresariales, un servidor de aplicaciones con contenedores para desplegar distintos tipos de elementos.
- El modelo de componentes de Java SE está basado en JavaBeans. Estas son clases de Java que cumplen determinados requisitos formales.
- El modelo de componentes de Java EE está basado en EJB.
- Para el desarrollo de aplicaciones web se suele emplear el modelo MVC. El modelo se implementa mediante JavaBeans o mediante EJB. En el primer caso, se necesita un contenedor de *servlets* tal como Apache Tomcat. En el segundo, se necesita un servidor Java EE completo, incluyendo contenedores para EJB, tal como GlassFish. La vista se implementa mediante JSP, y el controlador, mediante *servlets*.
- En los JSP se pueden utilizar JavaBeans para recopilar los valores para distintos parámetros de una petición HTTP, relativos al mismo objeto.
- Para la persistencia de JavaBeans se puede utilizar ORM mediante la API nativa de Hibernate o mediante JPA.
- Los EJB de sesión sin estado, o *stateless session beans*, se pueden utilizar como DAO que proporcionan una interfaz de alto nivel para el acceso a datos mediante clases de entidad. Los EJB sincronizan el acceso a los datos y proporcionan persistencia de objetos mediante JPA y transacciones mediante JTA.



Ejercicios propuestos

Se propone una aplicación web según el modelo MVC para gestionar los proyectos que desarrolla una empresa, sus empleados y las asignaciones de empleados a proyectos. Se propone realizar una aplicación para Java SE, utilizando JavaBeans, y una aplicación análoga para Java EE, utilizando *session beans* sin estado.

Se proporciona el diagrama entidad-relación para describir las entidades y las relaciones entre ellas, y las sentencias SQL necesarias para crear las tablas en MySQL en una nueva base de datos y proporcionar acceso a ellas a un nuevo usuario. Se puede utilizar otra base de datos distinta de MySQL, preferentemente Oracle.

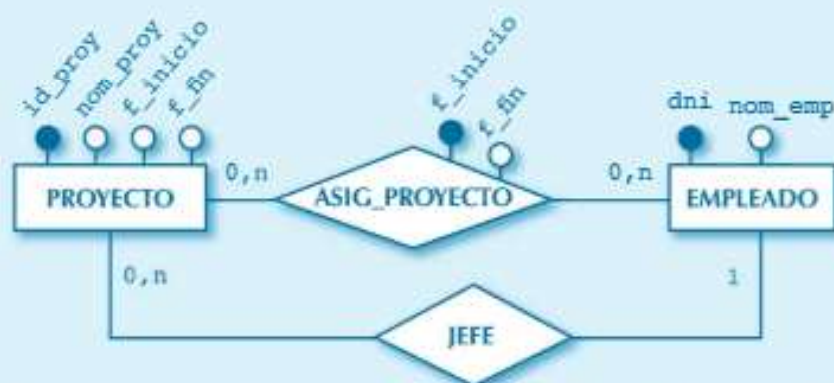
```
CREATE DATABASE proyectos_empresa;  
USE proyectos_empresa;  
CREATE TABLE empleado(  

```

```

    id_emp INTEGER AUTO_INCREMENT NOT NULL,
    dni CHAR(9) NOT NULL,
    nom_emp VARCHAR(32) NOT NULL,
    PRIMARY KEY(id_emp)
);
CREATE TABLE proyecto(
    id_proy INTEGER AUTO_INCREMENT NOT NULL,
    nom_proy VARCHAR(32) NOT NULL,
    f_inicio DATE NOT NULL,
    f_fin DATE,
    dni_jefe_proy CHAR(9) NOT NULL,
    PRIMARY KEY(id_proy),
    FOREIGN KEY fk_proy_jefe(dni_jefe_proy) REFERENCES empleado(dni)
);
CREATE TABLE asig_proyecto(
    id_emp INTEGER NOT NULL,
    id_proy INTEGER NOT NULL,
    f_inicio DATE NOT NULL,
    f_fin DATE,
    PRIMARY KEY(id_emp, id_proy, f_inicio),
    FOREIGN KEY f_asig_emp(id_emp) REFERENCES empleado(id_emp),
    FOREIGN KEY f_asig_proy(id_proy) REFERENCES PROYECTOS(id_proy)
);
CREATE USER 'proyectos_empresa'@'localhost' IDENTIFIED BY '(password)';
GRANT SELECT,INSERT,UPDATE,DELETE,CREATE,DROP,EXECUTE
ON proyectos_empresa.* TO 'proyectos_empresa'@'localhost';

```



Para cada una de las dos aplicaciones, se propone desarrollarla siguiendo estos pasos:

1. Listado de empleados. Creación, recuperación, modificación y borrado de empleados.
2. Listado de proyectos. Creación, recuperación, modificación y borrado de proyectos.
3. Creación de asignaciones de empleados a proyectos.
4. Visualización de datos de un empleado, incluyendo lista de asignaciones a proyectos. Enlace disponible en el listado de empleados.
5. Visualización de datos de un proyecto, incluyendo lista de empleados asignados al proyecto. Enlace disponible en el listado de proyectos. Mostrar todas las asignaciones, aunque

hayan concluido. Las asignaciones concluidas se deben mostrar con una apariencia distinta, por ejemplo, sobre fondo gris.

6. Borrado de asignaciones de empleados a proyectos, desde un enlace mostrado en el listado anterior, solo mostrado para asignaciones con fecha de inicio en el futuro.
7. Modificación de asignaciones de empleados a proyectos, desde un enlace mostrado en el listado anterior, solo mostrado para asignaciones vigentes (no concluidas). Debe poderse cambiar la fecha de inicio si está en el futuro, y la fecha de fin siempre, pero no debe poder ponerse la fecha de fin en el pasado. Si así se hiciera, se mostraría un mensaje de error.
8. Por último, como mejora, se podría crear una página JSP específica para gestión de errores. Como se indicó en el apartado correspondiente, habría que utilizar `<%%page errorPage="página_de_error" %>`, `<%%page isErrorPage="true"%>`, y la variable implícita `exception`.

ACTIVIDADES DE AUTOEVALUACIÓN

1. Los componentes de *software*:
 - ☐ a) Facilitan el desarrollo de sistemas modulares de *software*.
 - ☐ b) Se empaquetan y distribuyen de manera independiente.
 - ☐ c) Cumplen los requisitos establecidos en un modelo de componentes.
 - ☐ d) Todo lo anterior es cierto.
2. Los *JavaBeans*:
 - ☐ a) Solo sirven para desarrollar componentes visuales para interfaces gráficas de usuario.
 - ☐ b) No se pueden utilizar en la plataforma Java EE.
 - ☐ c) Son la base del modelo de componentes de Java SE.
 - ☐ d) Solo tienen utilidad para el desarrollo de aplicaciones web.
3. Las propiedades de los *JavaBeans*:
 - ☐ a) Deben ser públicas para que las puedan modificar las aplicaciones cliente.
 - ☐ b) Pueden ser simples, compuestas o indexadas.
 - ☐ c) Deben definirse como privadas y ser accesibles con funciones *getter* y *setter*.
 - ☐ d) Ninguna de las opciones anteriores es cierta.
4. Los *EJB*:
 - ☐ a) Se pueden desplegar en un servidor Apache Tomcat.
 - ☐ b) Forman parte de la plataforma Java SE.
 - ☐ c) No necesitan de ningún servidor de aplicaciones, se pueden desplegar en un contenedor autónomo.
 - ☐ d) Son un modelo de componentes para la plataforma Java EE.

5. El *context path*:

- ☐ a) Es un tipo particular de URL utilizada en la plataforma Java EE.
- ☐ b) Es una parte de una URL que identifica una aplicación web.
- ☐ c) Identifica un *servlet* desplegado en un servidor de *servlets*.
- ☐ d) Debe obligatoriamente redirigir a un *servlet* controlador.

6. Un JSP:

- ☐ a) Solo puede contener etiquetas de HTML.
- ☐ b) Debe realizar su tarea lo más rápidamente posible y ceder el control al *servlet* controlador.
- ☐ c) Puede utilizar *JavaBeans* para recopilar los valores que se le pasan en parámetros de la petición HTTP.
- ☐ d) Necesita un contenedor de EJB para funcionar.

7. Los *servlets*:

- ☐ a) Se despliegan en un servidor de *servlets* como por ejemplo, Apache Tomcat.
- ☐ b) Son accesibles mediante el protocolo HTTP o HTTPS.
- ☐ c) Se utilizan para implementar páginas JSP.
- ☐ d) Todo lo anterior es cierto.

8. Un EJB de sesión:

- ☐ a) No permite el acceso simultáneo por más de un *thread* (hebra de ejecución).
- ☐ b) Se puede utilizar para implementar una clase de tipo *Facade* para el acceso a las clases de entidad.
- ☐ c) Se puede utilizar como objeto DAO para el acceso a los datos representados por las clases de entidad.
- ☐ d) Todo lo anterior es cierto.

9. JPA:

- ☐ a) Es un componente opcional de la plataforma Java EE.
- ☐ b) Es una alternativa a Hibernate para persistencia en la plataforma Java EE.
- ☐ c) Utiliza anotaciones para las clases de Java.
- ☐ d) Solo se puede utilizar en la plataforma Java EE, no con Java SE.

10. Una aplicación web MVC para la plataforma Java:

- ☐ a) Puede empaquetarse en un fichero de tipo WAR.
- ☐ b) Utiliza *JavaBeans* para el modelo.
- ☐ c) Normalmente necesita un servidor de *servlets* para el controlador y para la vista.
- ☐ d) Todas las respuestas anteriores son correctas.

SOLUCIONES:

1. ☐ a ☐ b ☐ c ☒ d2. ☐ a ☐ b ☒ c ☐ d3. ☐ a ☐ b ☒ c ☐ d4. ☐ a ☐ b ☐ c ☒ d5. ☐ a ☒ b ☐ c ☐ d6. ☐ a ☐ b ☒ c ☐ d7. ☐ a ☐ b ☐ c ☒ d8. ☐ a ☐ b ☐ c ☒ d9. ☐ a ☐ b ☒ c ☐ d10. ☐ a ☐ b ☐ c ☒ d

